Miyabi システム
利用手引書

Version 1.6

2025 年 9 月 29 日

富士通株式会社

Copyright 2025 FUJITSU LIMITED

改版履歴

版	発行年月日	章	改版理由
1.0	2025.01.14		初版
1.1	2025.01.27	1.2.1 5.6.1	表 1 1 Supermicro ARS-111GL-DNHR-LCC の主な仕様の説明を更新 Miyabi-G での mpirun コマンドのオプション-x *env* について、指定する際の注意事項を追記
1.2	2025.04.04	1.3 5.3.1 5.7.1.2 5.7.1.3	Miyabi-C の利用可能なソフトフェアに MATLAB を追加 ジョブ投入オプション(-o/-e)の説明に指定したファイルには、追記モードで出力される旨を記載 「複数 MIG インスタンスの利用」を追加 「MPS 利用」を追加
1.3	2025.05.28	3.3 4.6.8.2.1 4.6.9.2.1	「計算ノードの内蔵ディスク領域(/tmp 領域、/local 領域)の使用について」を追加 例 3 の実行例に nvfortran コマンドのオプション-I${PHDF5_INC}を追加 例 2 の実行例に nvfortran コマンドのオプション-I${NETCDF_FORTRAN_INC}を追加
1.4	2025.07.23	8.7	「Shell 機能」を追加
1.5	2025.08.29	5.7.1.2	複数 MIG インスタンスの利用時の注意事項を追記
1.6	2025.09.29	2.7 5.8.8	「ワンタイムパスワード(OTP)の初期化」を追加 qstat –-limit コマンドの表示形式を修正

Copyright 2025 FUJITSU LIMITED

目次

はじめに 1

1. Miyabi システムの概要 2

1.1. システム構成 2
1.2. ハードウェア概要 3
1.3. ソフトウェア構成 5
1.4. システム利用概要 7

2. Miyabi システムへのアクセス 8

2.1. アカウントと認証方式 8
2.2. 接続先アドレス 8
2.3. システム初回ログイン時の設定 9
2.4. SSH ログイン 21
2.5. ファイル転送 29
2.6. ログイン環境 33
2.7. ワンタイムパスワード(OTP)の初期化 37

3. ファイルシステム環境 40

3.1. ファイルシステムの概要 40
3.2. ディスク使用状況と上限値の表示 42
3.3. 計算ノードの内蔵ディスク領域(/tmp 領域、/local 領域)の使用について 43

4. コンパイル/リンク 44

4.1. 環境設定 44
4.2. NVIDIA HPC SDK のコンパイル/リンク概要 45
4.3. NVIDIA CUDA Toolkit のコンパイル/リンク概要 50
4.4. Intel oneAPI のコンパイル/リンク概要 52
4.5. GCC のコンパイル/リンク概要 55
4.6. ライブラリ 58

5. バッチジョブ・インタラクティブジョブ 78

5.1. ジョブ管理システム概要 78
5.2. ジョブ実行リソース 79
5.3. ジョブ投入オプション 82
5.4. バッチジョブ実行 84
5.5. インタラクティブジョブ実行 109
5.6. MPI プログラムの実行 112
5.7. 高度なジョブ実行 114
5.8. ジョブ状態表示 121
5.9. ジョブキャンセル 130
5.10. ジョブ保留 131
5.11. ジョブ保留の解除 131

Copyright 2025 FUJITSU LIMITED

5.12. ジョブ履歴の参照 132
5.13. 計算ノードから外部リソースへのアクセス 133

6. マニュアル参照 134

7. コンテナ環境 135

7.1. Singularity の利用方法 135

8. Open OnDemand 136

8.1. 認証方式 136
8.2. Open OnDemand への接続 137
8.3. ログアウト 138
8.4. ファイル操作機能の利用 139
8.5. Desktop 環境の利用 140
8.6. Jupyter 環境の利用 142
8.7. Shell 機能 145
8.8. アクティブなジョブの確認機能の利用 146

Copyright 2025 FUJITSU LIMITED

はじめに

本利用手引書は、東京大学情報基盤センターと筑波大学計算科学研究センターが共同運営する、最先端共同 HPC 基盤施設（JCAHPC：Joint Center for Advanced High Performance Computing）が運用するスーパーコンピューターシステム Miyabi システムの利用方法について説明した資料です。システムを利用する方は、必ずお読みください。

本利用手引書の内容は、不定期に更新いたします。最新の利用手引書は、利用支援ポータルより入手することが可能です。

https://miyabi-www.jcahpc.jp

本利用手引書に記載しているシェルスクリプト等は、Miyabi システム ログインノードの以下のディレクトリに格納していますので、併せてご利用ください。

/usr/local/example 配下

Miyabi システムの運用状況は、東京大学情報基盤センターの下記 URL や利用支援ポータル等でアナウンスします。

https://www.cc.u-tokyo.ac.jp/
https://www.cc.u-tokyo.ac.jp/supercomputer/schedule.php

Miyabi システム利用に関する質問は Web ページまたはメールで受け付けています。質問の種別によって異なるメールアドレスを用意しておりますので下記 URL を参照ください。

https://www.cc.u-tokyo.ac.jp/supports/contact/

なお、よくあるご質問とその回答については、「FAQ」にまとめてあります。下記 URL を参照ください。

https://www.cc.u-tokyo.ac.jp/faq/miyabi.php

本書の一部、または全部を無断で複製、転載、再配布することを禁じます。

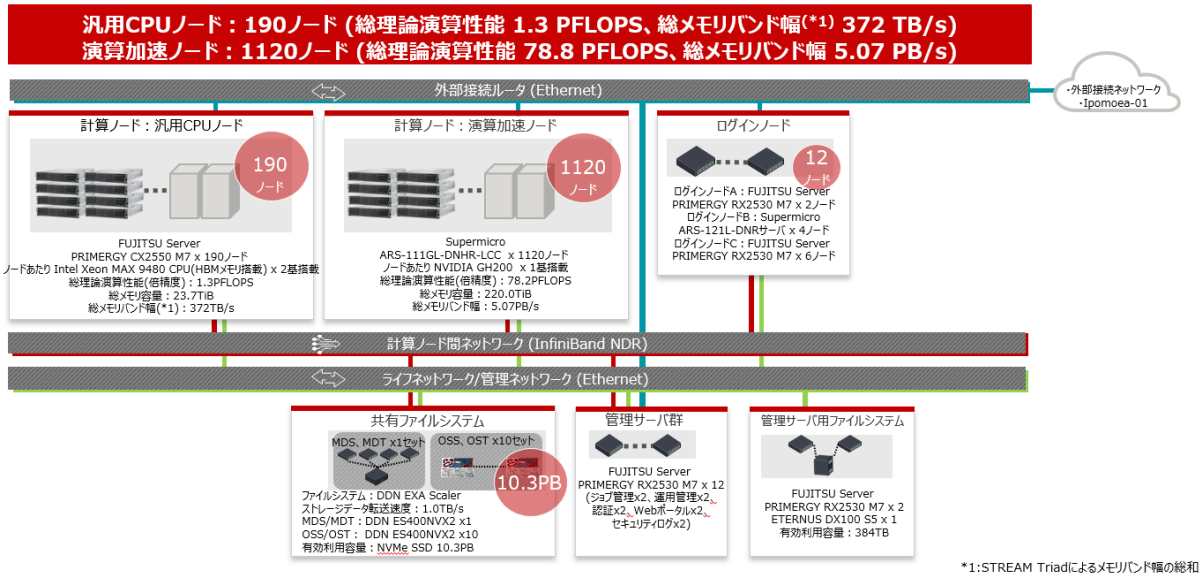
Copyright 2025 FUJITSU LIMITED

1. Miyabi システムの概要

1.1.　システム構成

Miyabi システムは、米国 NVIDIA 社による超高速 CPU-GPU 専用リンク NVLink-C2C で接続した GH200 Grace-Hopper Superchip を搭載した計算ノード 1,120 ノード(Miyabi-G)と、米国 Intel 社による Xeon Max 9480 を 2 基搭載した計算ノード 190 ノード(Miyabi-C)を InfiniBand NDR200 で結合した、倍精度演算性能 80.1 PFLOPS を有する超並列クラスタ型スーパーコンピュータであり、GH200 を搭載した国内初の汎用大規模システムです。さらに全ドライブに NVMe-SSD を採用した 11.3 PB の並列ファイルシステムを備えます。Miyabi システムの全体構成を図 1-1 に示します。

図 1-1 システム構成図



*1:STREAM Triadによるメモリバンド幅の総和

Miyabi-G(演算加速ノード)は 1,120 台の計算ノード「Supermicro ARS-111GL-DNHR-LCC」で構成され、総理論演算性能(倍精度浮動小数点演算性能)は 78.8 PFLOPS です。各ノードに NVIDIA 社製の CPU+GPU(NVIDIA GH200 Grace Hopper Superchip)を 1 基搭載します。

Miyabi-C(汎用 CPU ノード)は 190 台の計算ノード「FUJITSU Server PRIMERGY CX2550 M7」で構成され、総理論演算性能(倍精度浮動小数点演算性能)は 1.3 PFLOPS です。各ノードに Intel 社製の CPU(Intel Xeon MAX 9480)を 2 基搭載します。

ストレージ環境は、共有ファイルシステムで構成します。共有ファイルシステムの有効利用容量は 10.3PB です。各計算ノードは、InfiniBand NDR(25 GB/s)によりフルバイセクションバンド幅で接続され、ノード間通信とファイル共有を行います。

Miyabi システムへのアクセスは、ssh/scp による SSH アクセスとブラウザによる HTTPS アクセス(利用支援ポータル、Open OnDemand)が可能です。いずれのアクセスも、ワンタイムパスワード（OTP）認証を併用する 2 要素認証です。ユーザーはログインノード上にて、プログラムの編集、コンパイル/リンク、バッチジョブの操作、インタラクティブジョブの実行等の作業を行うことが可能です。インタラクティブジョブでは、デバッグやチューニングが可能です。

Copyright 2025 FUJITSU LIMITED

1.2. ハードウェア概要

1.2.1. Supermicro ARS-111GL-DNHR-LCC

Miyabi-G は、Supermicro ARS-111GL-DNHR-LCC で構成されます。

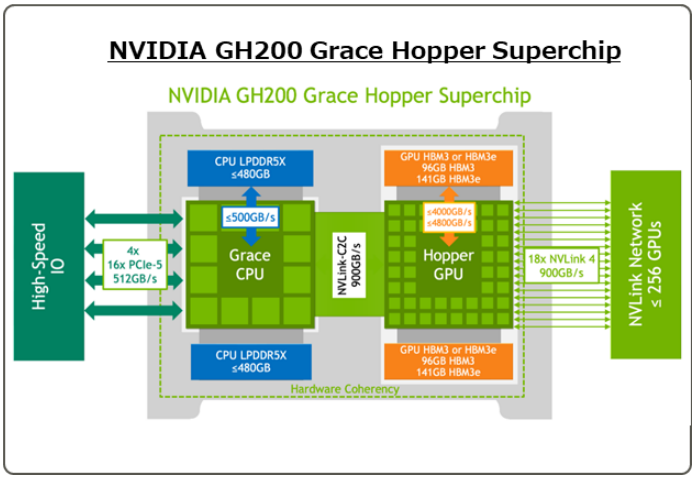


表 1-1 Supermicro ARS-111GL-DNHR-LCC の主な仕様

項目		説明
CPU	プロセッサ名	NVIDIA Grace CPU (Arm Neoverse V2 CPU)
	プロセッサ数(コア数)	1 (72)
	周波数	3.0 GHz
	論理演算性能	3.456 TFLOPS
	メモリ容量	120 GB
	メモリ帯域幅	512 GB/s
GPU	プロセッサ名	NVIDIA Hopper H100
	搭載数	1
	理論演算性能	66.9 TFLOPS
	メモリ容量	96 GB
	メモリ帯域幅	4.02 TB/s
	GPU-GPU 間接続	NVLink Chip-2-Ghip interconnect (片方向 450 GB/s)
内蔵ディスク		NVMe SSD 1.92 TB (PCIe Gen4 x4)
計算ノード間ネットワーク		InfiniBand NDR200 (200Gbps) x 1

Copyright 2025 FUJITSU LIMITED

1.2.2. Fujitsu Server PRIMERGY CX2550 M7

Miyabi-C は、PRIMERGY CX400 M7(シャーシ)、 Fujitsu Server PRIMERGY CX2550 M7 で構成されます。PRIMERGY CX400 M7 は、最大 4 台のサーバノードを搭載可能な 2U サイズのマルチノードサーバです。

表 1-2 Fujitsu PRIMERGY CX2550 M7 の主な仕様

項目		説明
CPU	プロセッサ名	Intel Xeon MAX 9480 CPU(開発コード名: Sapphire Rapids HBM)
	プロセッサ数(コア数)	2 (112)
	周波数	1.9 GHz
	論理演算性能	6.8096TFLOPS
	メモリ容量	128 GiB
	メモリ帯域幅	3.2 TB/s
計算ノード間ネットワーク		InfiniBand NDR200 (200Gbps) x 1

Copyright 2025 FUJITSU LIMITED

1.3. ソフトウェア構成

Miyabi システムで利用可能なソフトウェア一覧を以下に示します。

表 1-3 ソフトウェア一覧

<table>
<tr><th>項目</th><th>Miyabi-G(演算加速ノード)</th><th>Miyabi-C(汎用 CPU ノード)</th></tr>
<tr><td>OS</td><td colspan="2">Rocky Linux 9 (ログインノードは、Red Hat Enterprise Linux 9)</td></tr>
<tr><td>コンパイラ</td><td>NVIDIA HPC SDK
NVIDIA CUDA Toolkit
GCC</td><td>Intel OneAPI(インテルコンパイラ)
GCC</td></tr>
<tr><td rowspan="3">ライブラリ</td><td>NVIDIA 社製ライブラリ
(cuBLAS, cuSPARSE, cuFFT,MAGMA, cuDNN, NCCL)</td><td>インテル MPI ライブラリ
インテル MKL ライブラリ
(BLAS, CBLAS, LAPACK, ScaLAPACK)
NVIDIA 社製ライブラリ
(cuBLAS, cuSPARSE, cuFFT,MAGMA, cuDNN, NCCL)</td></tr>
<tr><td colspan="2">HDF5, FFTW, METIS, MT_METIS, ParMETIS, NetCDF, Parallel netCDF, PETSc, SuperLU, SuperLU_MT, SuperLU_DIST, Scotch, PT-Scotch, Xabclib, GSL, ppOpen-HPC, Trilinos</td></tr>
<tr><td colspan="2">Miniforge 注1</td></tr>
<tr><td>デバッガ</td><td>GDB/CUDA-GDB</td><td>Arm DDT</td></tr>
<tr><td>ツール</td><td></td><td>インテル VTune プロファイラー
インテル Advisor
インテル Inspector
インテル Trace Analyzer & Collector
Arm MAP</td></tr>
<tr><td rowspan="2">アプリケーション</td><td colspan="2">OpenFOAM, ABINIT-MP, PHASE, FrontFlow/blue, FrontISTR, REVOCAP, OpenMX, MODYLAS, GROMACS, BLAST, R Packages, Bioconductor, BioPerl, BioRuby, BWA, GATK, SAMtools, Quantum ESPRESSO, Xcrypt, ROOT, Geant4, LAMMPS, CP2K, NWChem, DeepVariant, ParaView, VisIt, POV-Ray, TensorFlow, Chainer, PyTorch, Keras, Horovod, MXNet</td></tr>
<tr><td>-</td><td>MATLAB 注2</td></tr>
<tr><td>ファイル
システム</td><td colspan="2">DDN Lustre</td></tr>
<tr><td>ジョブ管理
システム</td><td colspan="2">PBS Professional</td></tr>
<tr><td rowspan="3">フリーソフト
ウェア</td><td colspan="2">bash, tcsh, zsh, emacs, autoconf, automake, bzip2, cmake, cvs, gawk, gmake, gzip, perl, python, ruby, sed, tar, vim など</td></tr>
<tr><td colspan="2">Julia、CMake、Ninja、Java JDK</td></tr>
<tr><td colspan="2">Grid Community Toolkit、Gfarm、FUSE</td></tr>
<tr><td>コンテナ仮想化</td><td colspan="2">Apptainer、Singularity Community Edition、NVIDIA Enroot</td></tr>
</table>

Miyabi システムのオープンソースソフトウェアをご利用の際は、自己責任の範囲内でご活用ください。可能な範囲で動作確認は実施していますが、オープンソースソフトウェアのご利用方法等についてはお答えいたしかねますのでご了承ください。

Copyright 2025 FUJITSU LIMITED

注 1: Anaconda 社がサービス提供するリポジトリの利用規約が改定され、Anaconda の商用利用は有償になりました。

ただし、教育機関の学生または職員による教育活動での利用、非営利の研究機関による非商業的活動での利用等は継続して無償利用可能です。詳細については以下をご参照ください。

https://www.anaconda.com/terms-of-

可能であれば無償版である miniconda + conda forge の利用を推奨します。

注 2: Miyabi-C にて提供している MATLAB については、ライセンス上の制限により、東京大学情報基盤センターから発行されたユーザ ID でのみご利用が可能です。

Copyright 2025 FUJITSU LIMITED

1.4. システム利用概要

Miyabi システムでは、全ての計算ノードとログインノードにて、共有ファイルシステムを共有します。

ログインノードは、Miyabi-G 用のログインノード G と Miyabi-C 用のログインノード C があり、それぞれのログインノードで CPU アーキテクチャが異なります(ログインノード G: aarch64、ログインノード C: x86_64)。利用システムに応じたログインノードを選択しログインします。SSH によるログイン後、コマンドの対話的実行が可能であり、プログラムの作成・編集、コンパイル/リンク、ジョブ投入・表示などを行います。なお、ログインノードの CPU やメモリなどの資源は多くのユーザーで共有しますので、高負荷な処理は行わないようにしてください。高負荷な前処理、後処理を行う場合は、プリポストノードを利用してください。

計算ノード向けのプログラムを実行する場合、バッチジョブもしくはインタラクティブジョブとしてジョブ管理システムに処理を依頼します。インタラクティブジョブは、プログラムのデバッグや会話的なアプリケーション、可視化ソフトウェアの実行に使用します。

各ログインノードで使用できるメモリ量は、ユーザーあたりログインノード G は 14GB、ログインノード C は、16GB です。使用メモリ量合計が利用可能なメモリ量を越えた場合、使用メモリ量の大きいプロセスから順に強制終了されます。

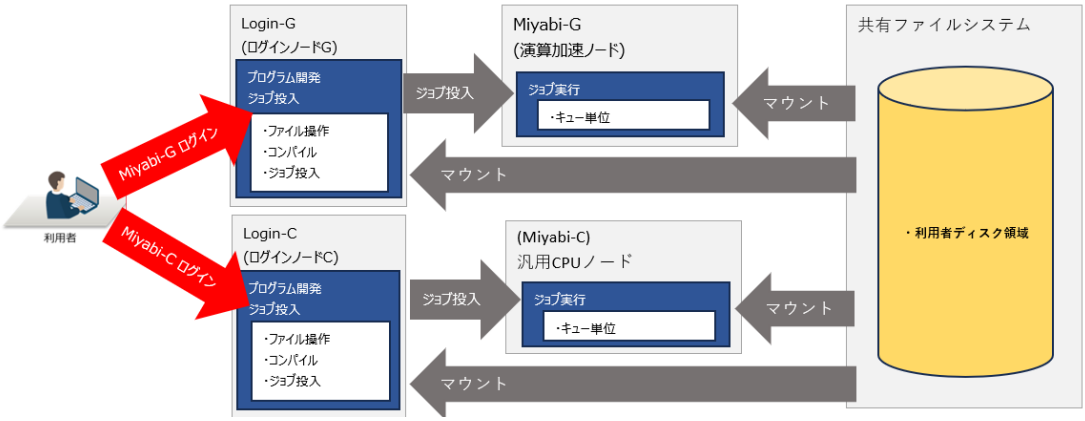


図 1-2 システム利用方法

Copyright 2025 FUJITSU LIMITED

2. Miyabi システムへのアクセス

2.1. アカウントと認証方式

Miyabi システムへのアクセスに使用するユーザー名は、申込時に通知されるユーザー名です。

Miyabi システムを初めて利用する場合、ユーザー名とパスワードを用いて利用支援ポータルにログインし、SSH 公開鍵を登録してください。Miyabi への SSH アクセス(ssh/scp)は、SSH version 2 の公開鍵認証および OAuth 認証のみ許可しています。

表 2-1 Miyabi システムへのアクセス方法と認証

サービス	アクセス	認証
仮想端末	ssh	・公開鍵認証(ユーザーが設定したパスフレーズ)+ワンタイムパスコード認証(OTP 認証) ・OAuth 認証(HPCI 利用)
ファイル転送	scp, sftp	
利用支援ポータル	https	LDAP 認証+ワンタイムパスコード認証(OTP 認証)
Open OnDemand	https	LDAP 認証+ワンタイムパスコード認証(OTP 認証)

2.2. 接続先アドレス

Miyabi システムがサービスを提供するノードで、ユーザーが外部から直接アクセスできるノードは、Miyabi-G ログインノードと Miyabi-C ログインノード、利用支援 WEB ノード、Open OnDemand サーバです。接続先アドレスを表 2-2 に示します。

Miyabi-G システムのログインノードは 3 台のノード、Miyabi-C のログインノードは 2 台のノードで構成されており、DNS ラウンドロビンにより、miyabi-g.jcahpc.jp にアクセスすると Miyabi-G のいずれかのログインノード、miyabi-c.jcahpc.jp にアクセスすると Miyabi-C のいずれかのログインノードにアクセスされます。

利用支援ポータル Web ノードは、利用支援ポータルサービスを提供しており、SSH 公開鍵登録やマニュアル参照等が可能です。

Copyright 2025 FUJITSU LIMITED

表 2-2 Miyabi 接続先一覧

ホスト名	プロトコル	用途
miyabi-g.jcahpc.jp	ssh	Miyabi-G ログインノード
miyabi-g1.jcahpc.jp		
miyabi-g2.jcahpc.jp		
miyabi-g3.jcahpc.jp		
miyabi-c.jcahpc.jp	ssh	Miyabi-C ログインノード
miyabi-c1.jcahpc.jp		
miyabi-c2.jcahpc.jp		
miyabi-www.jcahpc.jp	https	利用支援ポータル ● ドキュメント閲覧 ● パスワード変更 ● SSH 公開鍵登録
miyabi-c7.jcahpc.jp	https	Open OnDemand ● 仮想デスクトップ環境 ● JupyterLab

2.3. システム初回ログイン時の設定

Miyabi システムへの初回ログインまでの手順は以下のとおりです。

1. ssh 秘密鍵/公開鍵ペアを作成する （2.3.1 章）
2. 利用支援ポータルのトップ画面からパスワードを変更する （2.3.2.1 章）
3. 利用支援ポータルにログインする（2.3.2.2 章）
4. 利用支援ポータルから公開鍵を登録する （2.3.3 章）
5. ssh クライアントを使用して Miyabi システムへログインする （2.4 章）

2.3.1. ssh 秘密鍵/公開鍵ペアの作成

ssh 秘密鍵/公開鍵ペアは、作成済みの鍵を使用することも可能です。

Windows 環境で公開鍵認証方式用の公開鍵/秘密鍵のペアを作成する場合は「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」を、UNIX/Mac 環境で作成する場合は「2.3.1.2 ssh 秘密鍵/公開鍵ペアの作成(UNIX/Mac 環境)」を参照してください。

2.3.1.1. ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)

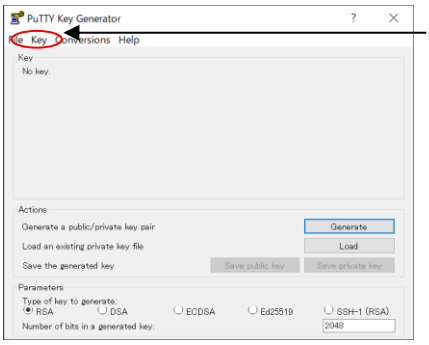
PC にて、ssh 秘密鍵/公開鍵ペアを作成します。鍵の作成は PuTTY をインストールした際に付属する PuTTYgen を用いる方法と OpenSSH の *ssh-keygen* コマンドを用いる方法があります。すでに鍵を作成済みの場合、作業を行う必要はありません。

2.3.1.1.1. PuTTYgen を用いた ssh 秘密鍵/公開鍵ペアの作成方法

PuTTY は以下のサイトからダウンロードすることができます。

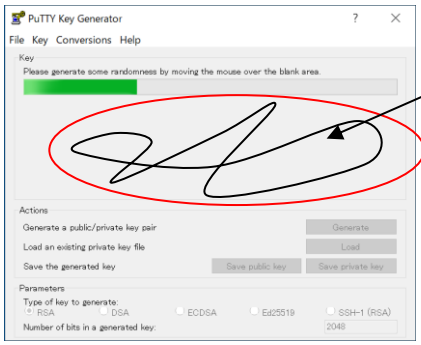
Copyright 2025 FUJITSU LIMITED

PuTTY:　http://www.chiark.greenend.org.uk/~sgtatham/putty/



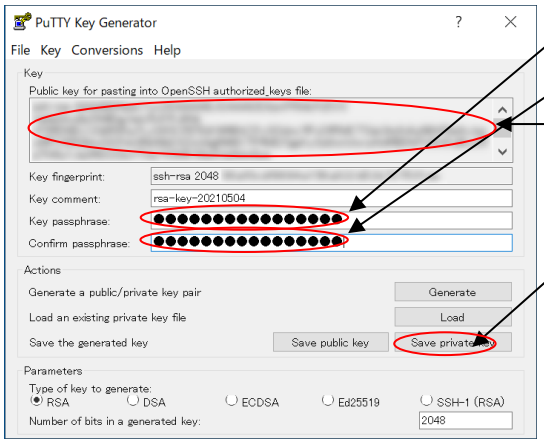
1. [Key]をクリック
2. 表示されるメニュー中の[Generate key pair]をクリック

図 2-1 PuTTY での秘密鍵/公開鍵作成画面 1



3. 鍵作成のための乱数を生成するため、マウスを動かす

図 2-2 PuTTY での秘密鍵/公開鍵作成画面 2



4. パスフレーズを入力
5. 再度パスフレーズを入力
6. 表示された公開鍵内容をコピーし、「2.3.3.2 公開鍵登録（直接入力）」で登録
   もしくは、テキストエディタ等でファイル
7. [Save private key]をクリック

図 2-3 PuTTY での秘密鍵/公開鍵作成画面 3

セキュリティ向上のため、パスフレーズは必ず入力してください。

Copyright 2025 FUJITSU LIMITED

2.3.1.1.2. ssh-keygen コマンドを用いた ssh 秘密鍵/公開鍵ペアの作成方法

Windows PowerShell を起動して *ssh-keygen* コマンドを実行します。

```
PS C:¥Users¥username> ssh-keygen ← ssh-keygen コマンドを入力
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:¥Users¥username/.ssh/id_ed25519): ← リターンを入力
Created directory 'C:¥Users¥username/id_ed25519'.
Enter passphrase (empty for no passphrase): ← パスフレーズを入力
Enter same passphrase again: ← 再度パスフレーズを入力
Your identification has been saved in C:¥Users¥username/.ssh/id_ed25519.
Your public key has been saved in C:¥Users¥username/.ssh/id_ed25519.pub.
The key fingerprint is:
SHA256:XXXXXXXXXXX username@machinename
The key's randomart image is:
+---[ED25519  256]----+
|                     |
|X X              X   |
|X X            X     |
|X X X        X X     |
|X X X        X XX    |
|     X X XX       XX |
|XXXX X    X      XX |
|XXX     X   X   X    |
|XXXXXXX     XX    |
+----[SHA256]-----+
PS C:¥Users¥username>
```

セキュリティ向上のため、パスフレーズは必ず入力してください。

Copyright 2025 FUJITSU LIMITED

2.3.1.2. ssh 秘密鍵/公開鍵ペアの作成(UNIX/Mac 環境)

アクセス元端末にて、ssh 秘密鍵/公開鍵ペアを作成します。鍵の作成は *ssh-keygen* コマンドを実行します。すでに鍵を作成済みの場合、作業を行う必要はありません。

- UNIX/Linux

  端末エミュレータを起動して、*ssh-keygen* コマンドを実行します。

- Mac

  ターミナルを起動して、*ssh-keygen* コマンドを実行します。

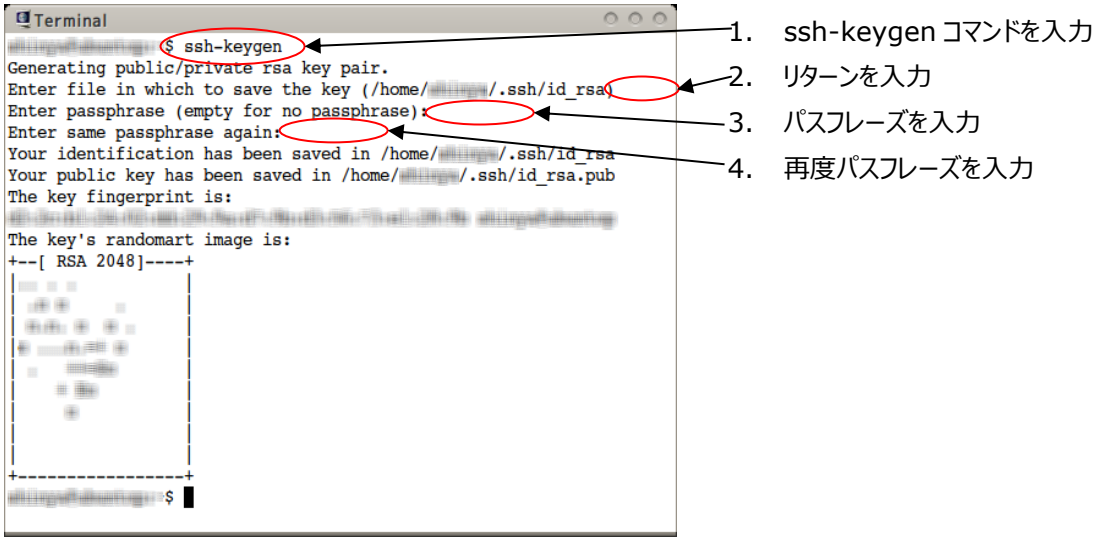


図 2-4 鍵ペアの作成

セキュリティ向上のため、パスフレーズは必ず入力してください。

Copyright 2025 FUJITSU LIMITED

2.3.2. 利用支援ポータルにログイン

公開鍵は利用支援ポータルから登録できるため、利用支援ポータルにログインします。

利用支援ポータルへのログインには、パスワード認証とワンタイムパスコード認証(OTP 認証)を併用する 2 要素認証を採用しています。

ご自身の端末(スマートフォンあるいは PC)に OTP 認証用のアプリケーションをインストールしてください。

対応している OTP 認証用アプリケーションは以下です。

OS	アプリケーション	配布元
Android	Google Authenticator	Google Play Store
	Microsoft Authenticator	Google Play Store
iOS	Google Authenticator	Apple App Store
	Microsoft Authenticator	Apple App Store
Windows	WinAuth	Github
macOS	Step Two	Apple App Store

2.3.2.1. パスワードの設定

(1) ブラウザで下記 URL にアクセスします。

https://miyabi-www.jcahpc.jp/

(2) 利用支援ポータルのトップ画面から「パスワード初期化」をクリックします。

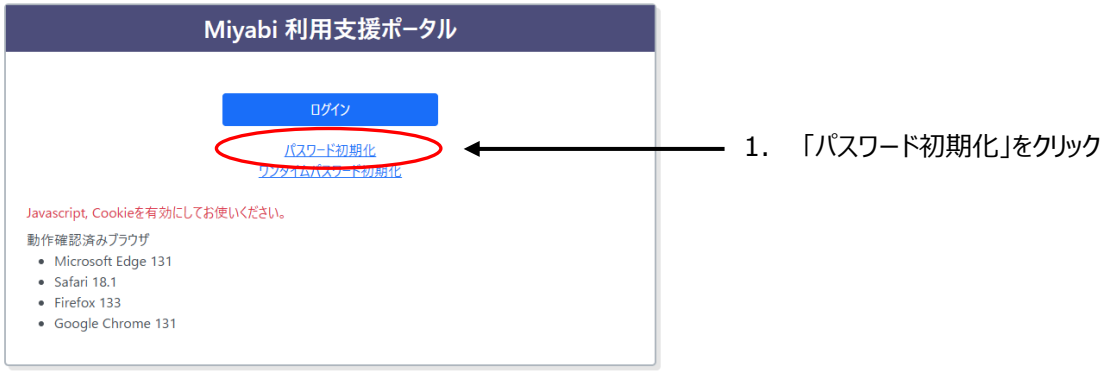


図 2-5 利用支援ポータルトップ画面

Copyright 2025 FUJITSU LIMITED

(3)　ユーザ ID と申請時のメールアドレスを入力します。

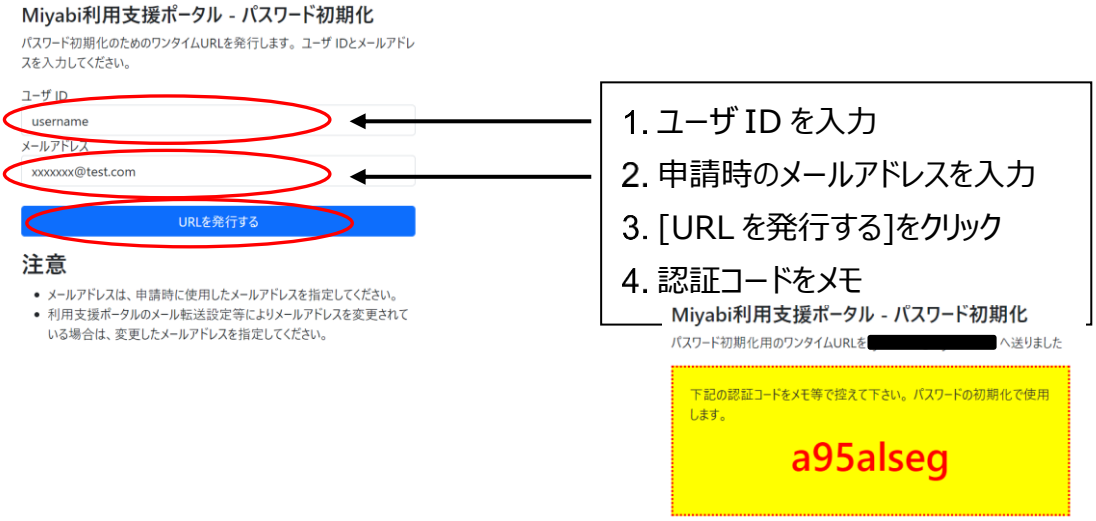


図 2-6 パスワード初期化画面

(4)　入力したメールアドレス宛に送られた URL にアクセスします。

メールの件名は“[Miyabi 利用支援ポータル] パスワード初期化用 URL”です。

Dear user username,

Miyabi 利用支援ポータルのパスワード初期化用 URL をお知らせします。
https://miyabi-www.jcahpc.jp/resetpassword//newpasswd/xxxxxx

(5)　新しいパスワードを設定します。上記(3)で控えた認証コードを入力し、「パスワードを初期化する」をクリックします。

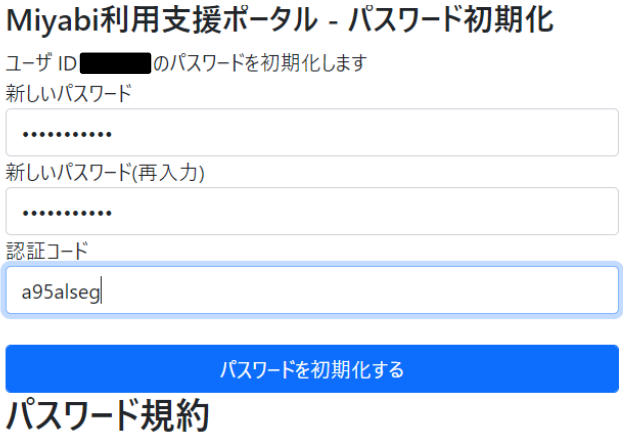


図 2-7 パスワード初期化画面

画面下部に表示されるパスワード規約を満たすパスワードを設定してください。

(6)　パスワード設定に成功すると、下記メッセージが出力されます。

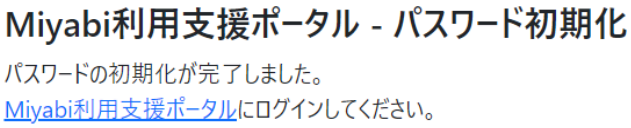


図 2-8 パスワード初期化完了画面

Copyright 2025 FUJITSU LIMITED

2.3.2.2. 利用支援ポータルへのログイン

(1) ブラウザで下記の URL にアクセスします。

https://miyabi-www.jcahpc.jp/

(2) 利用支援ポータルのトップ画面から、利用支援ポータルにログインします。

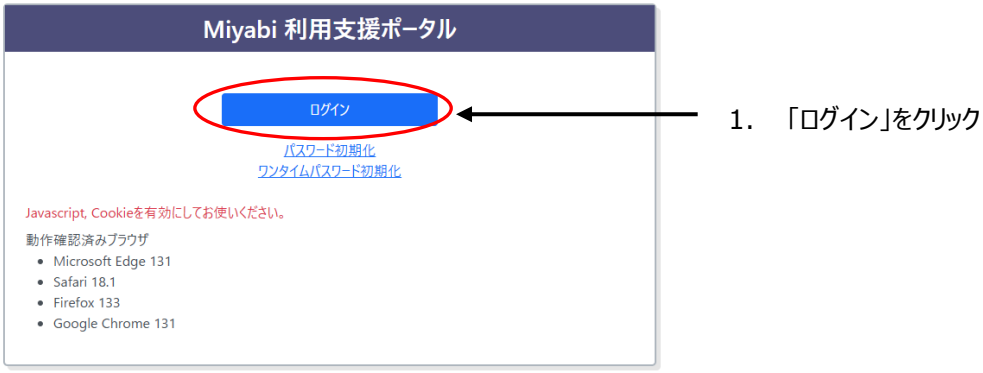


図 2-9 ログイン画面

(3) ユーザーID とパスワード(2.3.2.1 章にて設定したパスワード)を入力し、「ログイン」をクリックします。

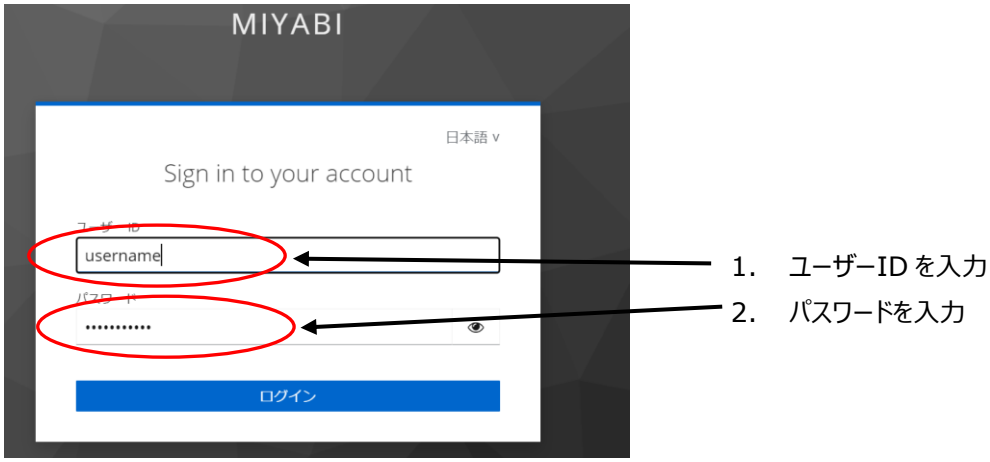


図 2-10 サインイン画面

Copyright 2025 FUJITSU LIMITED

(4)　ワンタイムパスワードの設定を行います。初回ログイン時のみ実施します。
パスワード認証が成功したら、ワンタイムパスワードの設定画面が表示されるため、OTP 認証用アプリケーションを起動し、QR コード(または、セットアップキー)を読み込み登録します。
※セットアップキーは、QR コード下の「スキャンできませんか？」をクリックすることで表示されます。

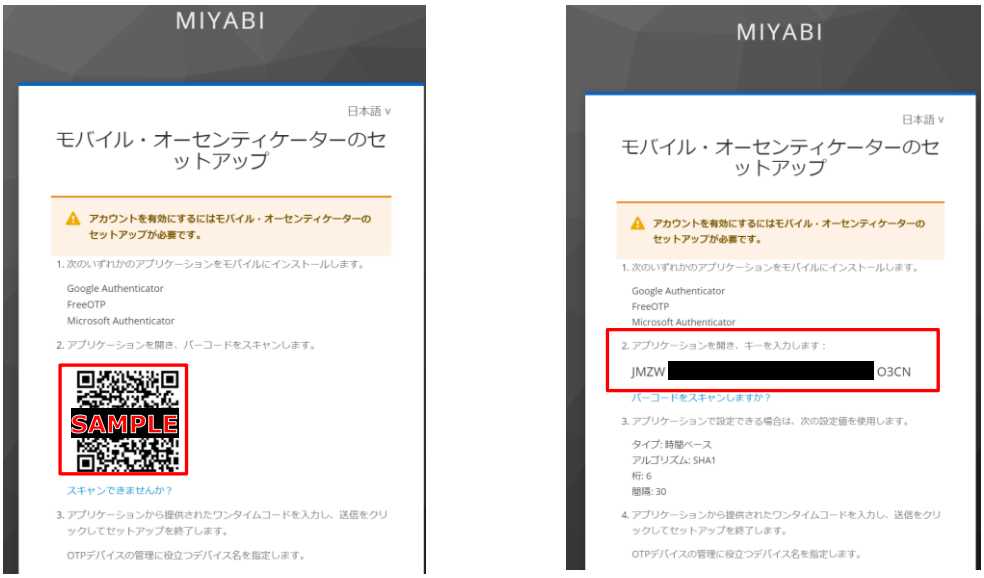


図 2-11 OTP 認証設定画面

(5)　ワンタイムパスワードを入力します。

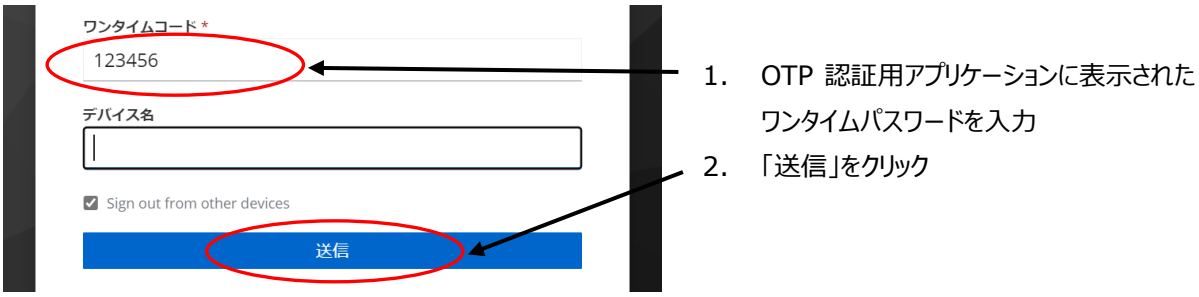


図 2-12 OTP 認証設定画面

OTP 認証用アプリケーションの削除や、デバイスの紛失等、登録した OTP アプリケーションが使用できない場合は、利用支援ポータルにログインできなくなるため、大切に保管してください。

Copyright 2025 FUJITSU LIMITED

2.3.3. SSH 公開鍵登録

Miyabi システムでは、ssh 接続を行う前に利用支援ポータルで公開鍵登録を行う必要があります。公開鍵を登録するには、[公開鍵登録]メニューを選択します。

1. [SSH 公開鍵登録]を選択

図 2-13 SSH 公開鍵登録画面への移動

公開鍵の登録は、作成した公開鍵ファイルをアップロードする方法（2.3.3.1 公開鍵登録（ファイルアップロード））と、公開鍵の内容をブラウザ内のテキストボックスに貼り付ける方法（2.3.3.2 公開鍵登録（直接入力））があります。公開鍵は複数登録可能です。

登録時は画面下部の注意事項をご確認の上、登録してください。

2.3.3.1. 公開鍵登録（ファイルアップロード）

(1) 公開鍵ファイルを指定します。

1. ファイルアップロードを選択
2. ファイルの選択から公開鍵を指定
3. [登録]をクリック

図 2-14 公開鍵ファイルのアップロード

Copyright 2025 FUJITSU LIMITED

(2)　公開鍵登録に成功すると、以下のメッセージが表示されます。

図 2-15 公開鍵登録の確認

2.3.3.2.　公開鍵登録（直接入力）

(1)　公開鍵の内容を表示し、コピーします。

Mac: Terminal を起動して、cat コマンドで公開鍵ファイルの内容を表示します。

UNIX/Linux: 端末エミュレータを起動し、cat コマンドで公開鍵ファイルの内容を表示します。

公開鍵ファイルは、デフォルトでは~/.ssh/id_rsa.pub です。
ご自身の環境に合わせて公開鍵の内容を表示してください。

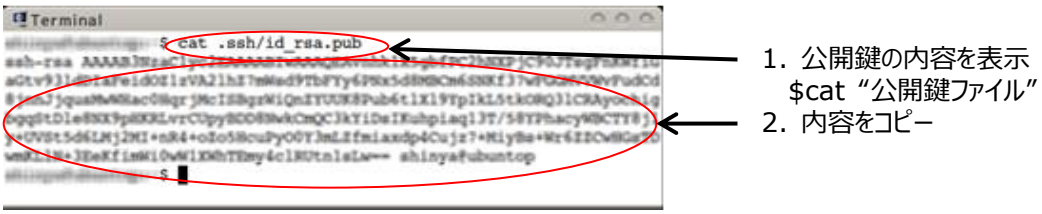


図 2-16 公開鍵の内容をコピー

(2)　公開鍵の内容を貼り付けます。

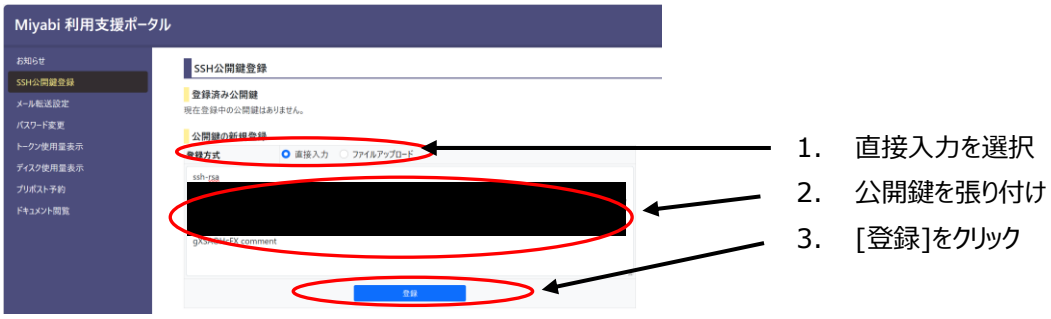


図 2-17 公開鍵の登録

Copyright 2025 FUJITSU LIMITED

(3)　公開鍵登録に成功すると、以下のメッセージが表示されます。

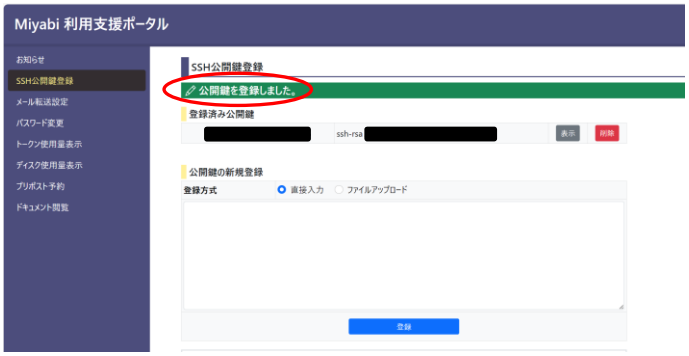


図 2-18 公開鍵登録の確認

2.3.3.3.　公開鍵削除

(1)　削除する公開鍵を指定します。

図 2-19 公開鍵の削除

Copyright 2025 FUJITSU LIMITED

2.4. SSH ログイン

Miyabi システムのログインノード(Miyabi-G 用のログインノード G と Miyabi-C 用のログインノード C)への SSH ログインには、公開鍵認証とワンタイムパスワード（OTP）認証を併用する 2 要素認証を採用しています。利用支援ポータルにて登録した OTP とは別のシークレットキーになるため、それぞれ別の管理となります。

SSH ログインするためには 2 要素認証に対応したターミナルソフトを利用する必要があります。Miyabi へのログインを確認できたターミナルソフトは以下です。

表 2-3 動作確認済ターミナルソフト

OS	ターミナルソフト/バージョン	配布元
Windows	Teraterm 5.3 以上	Github
	PuTTY 0.81 以上	PuTTY
	PowerShell 5.1.22621.4391 以上	-
	WSL Ubuntu 22.04.1 LTS 以上	-
macOS	terminal 2.4 以上	-

2 要素認証に対応していないターミナルソフト/バージョンを使用した場合、システムにログインできません。
対応しているソフト/バージョンに切り替えて、ログインしてください。

Copyright 2025 FUJITSU LIMITED

2.4.1. 初回ログイン

SSH 初回ログイン時に、OTP 認証の設定を行います。OTP 認証アプリケーションに登録する際,
ターミナル上に大きめの QR コードが表示されるため、事前にターミナルソフトの画面を上下に広げてください。

(1) ターミナルソフトから SSH ログインします。

```
$ ssh -l username miyabi-g.jcahpc.jp
The authenticity of host '
miyabi-g.jcahpc.jp (XX.XX.XX.XX)' can't be established. ← 初回ログイン時のみ表示される。
RSA key fingerprint is XX:XX:XX:XX:XX:XX:XX:XX:XX:XX:XX
Are you sure you want to continue connecting (yes/no)?  ← yes を入力
Warning: Permanently added 'XX.XX.XX.XX' (RSA) to the list of known hosts.
Enter passphrase for key '/home/username/.ssh/id_rsa':    ← パスフレーズ入力
```

(2) ワンタイムパスワードの設定

公開鍵認証に成功すると、QR コードがターミナル画面に表示されるため、OTP 認証用アプリケーションで、QR コードを読み込み登録してください。QR コードが読み込めない場合は、QR コード下部に出力される、[secret key]を OTP 認証用アプリケーションに入力してください。

図 2-20 OTP 認証用の QR コード(見本)

ターミナルの画面サイズが小さい時など、QR コードの表示が崩れる場合があります。
その場合は、Ctrl+C で一旦抜けることでターミナルが終了するため、画面サイズを大きくしてから再度ログインをお試しください。

Copyright 2025 FUJITSU LIMITED

(3) OTP 認証用アプリケーションが正しく動作しているのかを確認するため、表示されているワンタイムパスワードを入力します。

```
Enter code from app: ワンタイムパスワード
```

(4) 緊急用スクラッチコードが出力されるため大切に保管してください。OTP 認証できなくなった際に、2.4.3 にて示す方法で OTP の再登録ができるようになります。

```
Code confirmed
Your emergency scratch codes are:
14919875
64122242
77942233
34271823
16515648

[username@miyabi-g1 ~]$
```

緊急用スクラッチコードは、機密性が高い情報であるため、厳重に保管してください。

初回ログイン後に、OTP の設定ファイルとして、/home/<username>/.google_authenticator が作成されます。OTP 認証時に使用しますので、ファイルの削除等はしないでください。

Copyright 2025 FUJITSU LIMITED

2.4.2. 2 回目以降のログイン

（1） 初回ログイン時設定した OTP 認証用アプリケーションを起動します。

（2） ターミナルソフトから SSH ログインします。

```
username$ ssh -l username miyabi-g.jcahpc.jp
```

（3） ワンタイムパスワードを入力します。

```
Verification code: 123456 ← ワンタイムパスワード入力
[username@miyabi-g1 ~]$
```

デバイスの紛失等で OTP 認証用アプリケーションが使用できない場合は、システムにログインできなくなります。

2.4.3. スクラッチコードを使ったリカバリー(OTP 設定の初期化)

デバイスの紛失等で OTP 認証用アプリケーションでのログインができなくなった場合、初回ログイン時に表示される緊急用スクラッチコードを使い一時的にログインすることができます。

緊急用スクラッチコードを使い OTP 設定を初期化することで、OTP の再設定が可能になります。

（1） ターミナルソフトから SSH アクセスします。

```
username$ ssh -l username miyabi-g.jcahpc.jp
```

（2） 緊急用スクラッチコードを入力ログインします。スクラッチコードは一度しか使用できません。

```
Verification code: 1234568 ← 緊急用スクラッチコードを入力
[username@miyabi-g1 ~]$
```

（3） OTP の設定ファイルを削除します。

```
[username@miyabi-g1 ~]$ rm -f /home/<username>/.google_authenticator
```

（4） ログアウトします。

```
[username@miyabi-g1 ~]$ logout
```

（5） 再度ログインすることで、初回ログイン同様に QR コードが表示されるため、OTP 設定をします。

OTP 設定を初期化すると、以前のワンタイムパスワードは使用できなくなります。

Copyright 2025 FUJITSU LIMITED

2.4.4. SSH ログイン(Windows 環境)

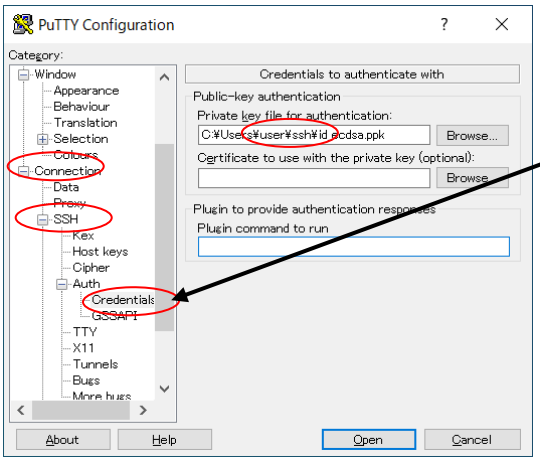
Windows では様々なターミナルソフトが利用できますが、ここでは PuTTY を用いた SSH 接続方法と Windows PowerShell を用いた SSH 接続方法を説明します。Cygwin を使用される方は「2.4.5 SSH ログイン(UNIX/Mac 環境)」を参照してください。

PuTTY は以下のサイトからダウンロードすることができます。

PuTTY: http://www.chiark.greenend.org.uk/~sgtatham/putty/

PuTTY のバージョンが古い場合、セキュリティ上の理由でログインに失敗することがあります。その場合は、最新版にアップデートしてください。

(1)PuTTY を起動し、「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」で作成した秘密鍵ファイル(.ppk ファイル)を指定します。



1. [接続(Connection)] - [SSH] - [認証(Auth)] – [証明書 Credentials]メニューを選択
2. [参照(Browse)] をクリックし、登録した公開鍵と対となる秘密鍵ファイルを指定

図 2-21 PuTTY での秘密鍵指定画面

Copyright 2025 FUJITSU LIMITED

(2)PuTTY から Miyabi システムにアクセスします。

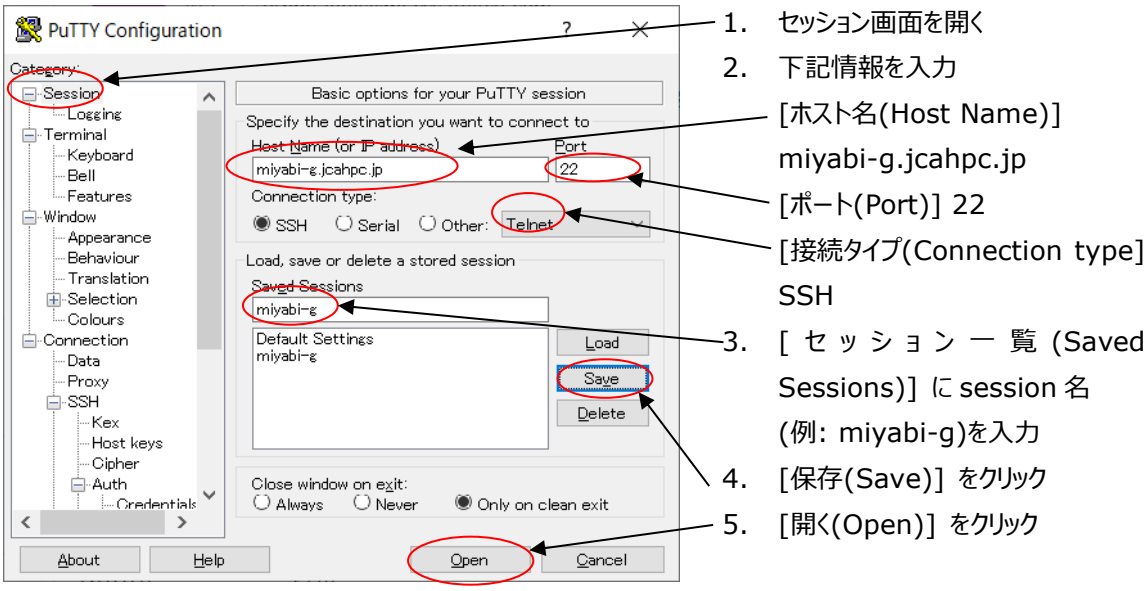


図 2-22 PuTTY セッション画面

ユーザー名と「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」にて入力したパスフレーズを入力します。入力すると QR コードが表示されるため、OTP アプリケーションを起動して、OTP 認証を行います。

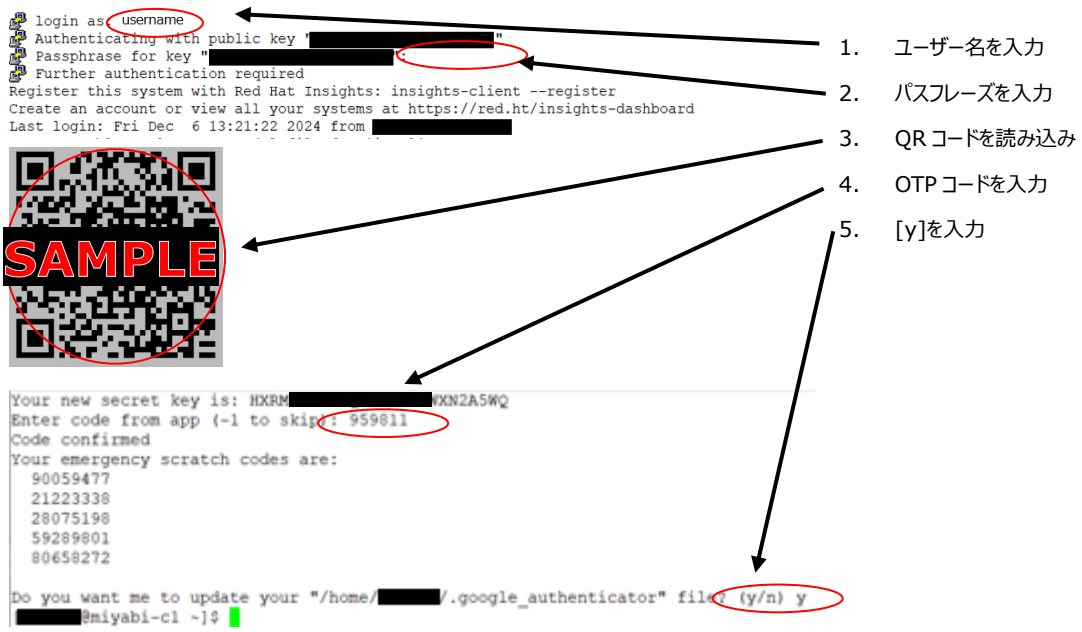


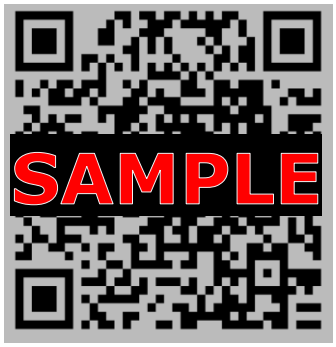
図 2-23 PuTTY ログイン完了画面

Copyright 2025 FUJITSU LIMITED

2.4.4.1. Windows PowerShell を用いた SSH 接続方法

Windows PowerShell から Miyabi システムへログインする場合は、*ssh* コマンドを利用します。

```
PS C:¥Users¥username> ssh -l username miyabi-g.jcahpc.jp ← ssh コマンドを入力
Enter passphrase for key 'C:¥Users¥username/.ssh/id_ed25519': ← パスフレーズを入力
Last login: Wed Dec  4 15:04:52 2024 from XXX.XXX.XXX.XX
```



← OTP アプリケーションで QR コードを読み取り

```
Your new secret key is: WN3E22XXXXXXXXXXXX4TAD34
Enter code from app:123456  ← ワンタイムパスワード入力

Code confirmed
Your emergency scratch codes are:
14919875
64122242
77942233
34271823
16515648

[username@miyabi-g1 ~]$
```

Copyright 2025 FUJITSU LIMITED

2.4.5. SSH ログイン(UNIX/Mac 環境)

アクセス元端末から Miyabi システムへログインする場合は、*ssh* コマンドを利用します。

```
% ssh -l username miyabi-g.jcahpc.jp
The authenticity of host
'miyabi-g.jcahpc.jp:22 (XX.XX.XX.XX)' can't be established.
ED25519 key fingerprint is XX:XX:XX:XX:XX:XX:XX:XX:XX:XX:XX
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes ← yesを入力
Warning: Permanently added 'XX.XX.XX.XX' (ED25519) to the list of known hosts.
Enter passphrase for key '/Users/username/.ssh/id_ed25519':  ← パスフレーズ入力
```

← OTP アプリケーションで QR コードを読み取り

```
Your new secret key is: WN3E22XXXXXXXXXXXX4TAD34
Enter code from app: 123456  ← ワンタイムパスワード入力
Code confirmed
Your emergency scratch codes are:
  39308745
  95687997
  23603117
  85315693
  72459926

[username@miyabi-g1 ~]$
```

Copyright 2025 FUJITSU LIMITED

2.5. ファイル転送

2.5.1. ファイル転送(Windows 環境)

2.5.1.1. WinSCP を利用する場合

PCとMiyabiシステム間のファイル転送について、WinSCPによるファイル転送方法を説明します。WinSCPは以下のサイトから入手できます。

WinSCP は転送するファイルをドラッグ＆ドロップすることでファイル転送することが可能です。

WinSCP: http://winscp.net/eng/docs/lang:jp

WinSCP を使用する場合は、事前に「2.4.1 初回ログイン」にて OTP 認証を設定する必要があります。

(1) WinSCP を起動して、ログインします。

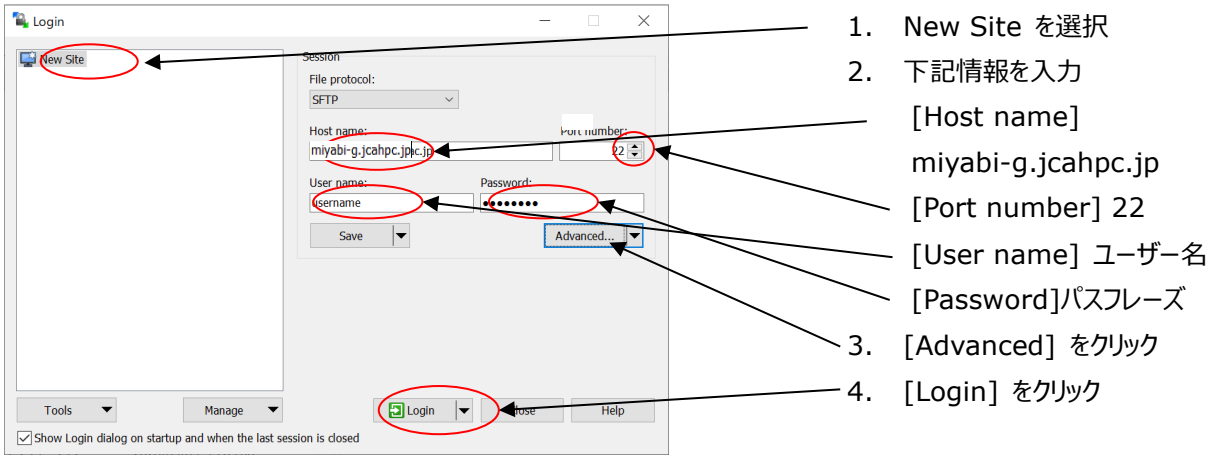


図 2-24 WinSCP Login 画面

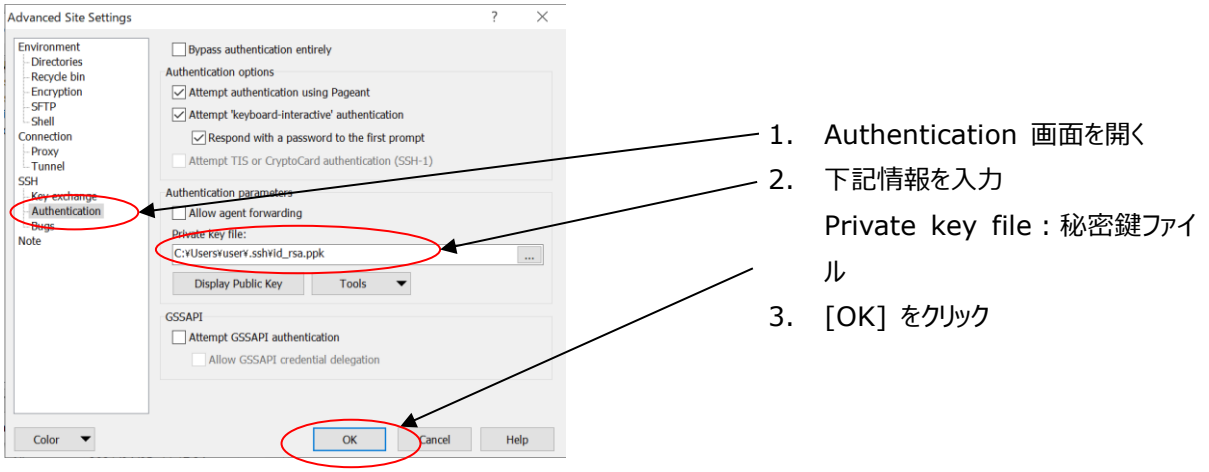


図 2-25 WinSCP Settings 画面

Copyright 2025 FUJITSU LIMITED

(2) ワンタイムパスワードを入力します。

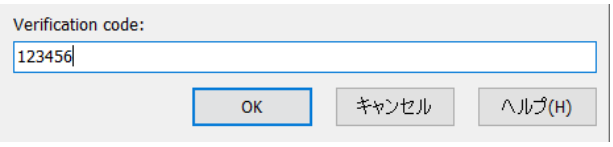


図 2-26 ワンタイムパスワード入力画面

(3) ドラッグ＆ドロップでファイル転送を行います。

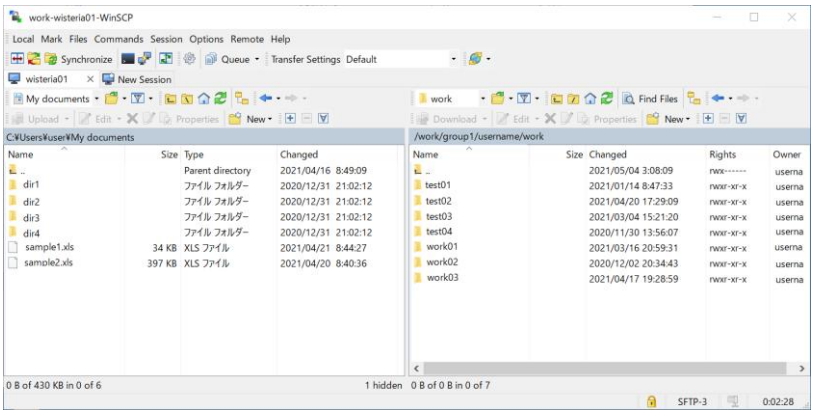


図 2-27 WinSCP ログイン後の画面

Copyright 2025 FUJITSU LIMITED

2.5.1.2. rsyncコマンドを利用する場合

WSL(Windows Subsystem for Linux)を使用して Linux 環境を構築し、*rsync* コマンドによりファイル転送を行う方法を説明します。以下では Linux のディストリビューションとして Ubuntu を使用しています。

(1) Windows PowerShell を起動して、*wsl --install* コマンドにより Linux 環境をインストールします。
※*wsl --list --online* と入力すると、利用可能な Linux ディストリビューションを一覧表示できます。

```
PS C:¥Users¥username> wsl --install -d Ubuntu
ダウンロード中: Ubuntu
インストール中: Ubuntu
Ubuntu はインストールされました。
Ubuntu を起動しています...
```

(2) インストール後、Windows PowerShell のターミナルとは別に新規に WSL のターミナルが起動されます。
起動された WSL ターミナルにて Linux 環境で利用するユーザ名/パスワードを入力します（ここで入力したユーザ名/パスワードは Linux 環境でのみ利用されます）。

```
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: XXXX          ← ユーザ名を入力
New password: ++++++++          ← パスワードを入力
Retype new UNIX password: ++++++++          ← パスワードを再入力
passwd: password updated successfully
Installation successful!
...
```

(3) *exit* コマンドで WSL ターミナルを閉じます（WSL ターミナル上でも以降の操作は可能です）。

(4) Windows PowerShell 上で *wsl* コマンドを実行し、構築した Linux 環境にログインします。

```
PS C:¥Users¥username> wsl -d Ubuntu
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
$
```

Copyright 2025 FUJITSU LIMITED

(5) *rsync* コマンドを実行します。以下は 2.3.1.1.2 項で PC に保存した秘密鍵を使用する場合の例です。

```
$ rsync -e "ssh -i /mnt/c/Users/username/.ssh/id_rsa" local-file ¥
  username@miyabi-g.jcahpc.jp:remote-dir
sending incremental file list

(username@miyabi-g.jcahpc.jp) Verification code: 13747332  ← ワンタイムパスワードを入力

sent XX bytes   received XX bytes   XX.XX bytes/sec
total size is XX   speedup is X.XX
```

WSL の Linux 環境から PC のファイルにアクセスする場合は、*wslpath* コマンドを使用して変換されたパスを指定する必要があります。上記 *rsync* コマンドの実行例では、以下のように表示されたパスを指定しています。

```
$ wslpath "C:¥Users¥username¥.ssh¥id_rsa"
/mnt/c/Users/username/.ssh/id_rsa
```

(6) WSL の Linux 環境からログアウトする場合は *exit* コマンドを実行します。

2.5.2. ファイル転送(UNIX/Mac 環境)

UNIX 系/Mac 系の PC と Miyabi システム間でファイル転送をする場合、*scp*(*sftp*)コマンドを利用します。

```
$ scp local-file username@miyabi-g.jcahpc.jp:remote-dir
Enter passphrase for key  : ++++++++      ← パスフレーズを入力
Verification code: 123456                 ← ワンタイムパスワードを入力

local-file      100% |**********************|   file-size   transfer-time
```

Copyright 2025 FUJITSU LIMITED

2.6. ログイン環境

2.6.1. ログインシェル

Miyabi システムのデフォルトログインシェルには、bash が設定されています。ログインシェルは *chsh* コマンドを使用して tcsh もしくは zsh に変更することができます。ログインシェルの変更は次回ログインから有効となります。また、ログインシェルの反映には 10 分程度時間がかかります。

```
$ chsh  [option]  <new_shell>
```

表 2-4 chsh コマンドオプション一覧

オプション	説明
-l	利用可能なログインシェル一覧を表示する [/bin/bash, /bin/tcsh, /bin/zsh]
-s	ログインシェルを変更する

例) ログインシェルを tcsh に変更する。

```
[username@miyabi-g1 ~]$ chsh -s /bin/tcsh
Enter LDAP Password:                    ← 利用支援ポータルのパスワードを入力します
modifying entry "uid=username,ou=People,dc=Miyabi,dc=jcahpc=ac,dc=jp"
```

Miyabi システムへログインすると、Miyabi システムを利用するための環境設定が自動で設定されます。環境変数 PATH や環境変数 LD_LIBRARY_PATH 等をカスタマイズする場合は、表 2-5 に示す個人用設定ファイルに設定してください。

表 2-5 個人用設定ファイル一覧

ログインシェル	個人用設定ファイル
bash	$HOME/.bash_profile
tcsh	$HOME/.cshrc
zsh	$HOME/.zshrc

設定ファイルに環境変数 PATH にパスを追加する際は、環境変数 PATH の最後に追加してください。先頭に追加した場合、システムを正常に使用できなくなる恐れがあります。

個人用設定ファイルのオリジナル（雛形）は/etc/skel 配下に格納しています。

Copyright 2025 FUJITSU LIMITED

2.6.2. メール転送設定

ジョブの開始時や終了時にメールを通知することができます。通知するメールアドレスを設定/変更する場合は、利用支援ポータルの「メール転送設定」を使用してください。

ログインノードからはメールを送信することはできません。

2.6.3. マスク値

新たに作成されるファイルやディレクトリのパーミッションはマスク値で設定することが可能です。マスク値のデフォルトは 0027 です。マスク値を変更する場合は、*umask* コマンドを使用してください。

例） 自分が所属しないグループに属するユーザの読み込み・書き込み・実行を抑止する場合

```
[username@miyabi-g1 ~]$ umask 0007
```

ホームディレクトリおよび.ssh ディレクトリ、公開鍵ファイル(authorized_keys)のパーミッションに他ユーザーに対する書き込み権があるとログインできなくなります。ご注意ください。

2.6.4. 実行パス/ライブラリ設定

ユーザー環境は、Environment Modules (http://modules.sourceforge.net/)を用いて設定されます。ユーザーは *module* コマンドを用いることで容易に環境を変更することが可能です。*module* コマンドは、アプリケーション実行やライブラリの利用時に必要となる環境変数 PATH や環境変数 LD_LIBRARY_PATH などを設定(load)・解除(unload)します。

```
$ module <sub-command>
```

Copyright 2025 FUJITSU LIMITED

module コマンドのサブコマンドを表 2-6 に示します。

表 2-6 *module* コマンドのサブコマンド

サブコマンド	説明
list	設定中のアプリケーション、ライブラリの一覧を表示します。
avail	設定可能なアプリケーション、ライブラリの一覧を表示します。
load <module>	アプリケーション、ライブラリ向けの環境変数を設定します。
unload <module>	アプリケーション、ライブラリ向けの環境変数を解除します。
switch <moduleA> <moduleB>	アプリケーション、ライブラリ向けの環境変数を moduleA から moduleB へ入れ替えます。
purge	設定中のすべてのアプリケーション、ライブラリを解除します。
help <module>	アプリケーション、ライブラリの使用方法を表示します。また、構築に使用したコンパイラのバージョンを表示します。

同一アプリケーションの複数バージョンの利用や、排他関係にあるアプリケーション、ライブラリは同時に利用することはできません。排他関係にあるアプリケーション、ライブラリを同時に利用しようとした場合、以下のメッセージが表示されます。

例）排他関係にある言語処理系を利用した場合

```
[username@miyabi-g1 ~]$ module load nvidia/24.5
Loading nvidia/24.5
  ERROR: Module cannot be loaded due to a conflict.
    HINT: Might try "module unload nvidia" first.
```

利用中のアプリケーション、ライブラリを入れ替えるには、以下を実行してください。

例）nvidia/24.9 を load した状態から、nvidia/24.5 に環境を切り替える場合(バージョン間で排他の関係にあります)

```
[username@miyabi-g1 ~]$ module switch nvidia/24.9 nvidia/24.5
```

アプリケーション、ライブラリの構築されたバージョンを確認するには、以下を実行してください。

例）gsl/2.8 の構築バージョンを確認する場合

```
[username@miyabi-g1 ~]$ module help gsl/2.8
-------------------------------------------------------------------
Module Specific Help for /work/opt/local/modules/modulefiles/LNG/compiler/nvidia/24.9/gsl/2.8:


            'gsl/2.8' is built by nvidia/24.9
-------------------------------------------------------------------
(以下略)
```

Miyabi システムで利用可能なアプリケーション/ライブラリの最新の情報については 2.6.5 利用可能なアプリケーション/ライブラリの一覧表示をご参照ください。

Copyright 2025 FUJITSU LIMITED

2.6.5. 利用可能なアプリケーション/ライブラリの一覧表示

Miyabi システムで利用可能なアプリケーション/ライブラリの一覧を表示するには、*show_module* コマンドを実行してください。

```
$ show_module [option]
```

show_module コマンドのオプションを表 2-7 に示します。

表 2-7 *show_module* コマンドのオプション一覧

オプション名	説明
なし	ログインした環境(Miyabi-C/Miyabi-G)で利用可能なアプリケーション/ライブラリ、およびコンパイラに依存しないソフトウェア一覧を表示
-a	Miyabi システム(Miyabi-C/Miyabi-G を問わない)で利用可能なすべてのアプリケーション/ライブラリの一覧を表示

例)アプリケーション/ライブラリの一覧を表示(Miyabi-G)

```
[username@miyabi-g1 ~]$ show_module
ApplicationName                ModuleName                NodeGroup    BaseCompiler/MPI
-----------------------------------------------------------------------------------------
Apptainer                      apptainer/1.3.5           Login-G      -
Apptainer                      apptainer/1.3.5           Miyabi-G     -
BLAST                          blast/2.16.0              Login-G      gcc/11.4.1
BLAST                          blast/2.16.0              Miyabi-G     gcc/11.4.1
(省略)
```

例)アプリケーション/ライブラリの一覧を表示(Miyabi-C)

```
[username@miyabi-c1 ~]$ show_module
ApplicationName                ModuleName                NodeGroup    BaseCompiler/MPI
-----------------------------------------------------------------------------------------
BOOST                          boost/1.86.0              Login-C      intel/2023.2.0
BOOST                          boost/1.86.0              Miyabi-C     intel/2023.2.0
BWA                            bwa/0.7.17                Login-C      intel/2023.2.0
BWA                            bwa/0.7.17                Miyabi-C     intel/2023.2.0
(省略)
```

Copyright 2025 FUJITSU LIMITED

show_module コマンドの表示項目を表 2-8 に示します。

表 2-8 *show_module* コマンドの表示項目

項目	説明
ApplicationName	アプリケーション・ライブラリ名、および説明
ModuleName	module avail で表示するアプリケーション/ライブラリ名
NodeGroup	アプリケーション・ライブラリを利用可能なノード (Login-G/Login-C： ログインノード、 Miyabi-G/Miyabi-C： 計算ノード)
BaseCompiler/MPI	アプリケーション・ライブラリを利用するためにロードが必要なコンパイラとバージョン、および依存ライブラリとバージョン

2.7. ワンタイムパスワード(OTP)の初期化

2.7.1. 利用支援ポータルと Open OnDemand の OTP の初期化

(1) ブラウザで下記 URL にアクセスします。

https://miyabi-www.jcahpc.jp/

(2) 利用支援ポータルのトップ画面から「パスワード初期化」をクリックします。

1. 「ワンタイムパスワード初期化」をクリック

図 2-28 利用支援ポータルトップ画面

Copyright 2025 FUJITSU LIMITED

(3)　ユーザ ID と申請時のメールアドレスを入力します。

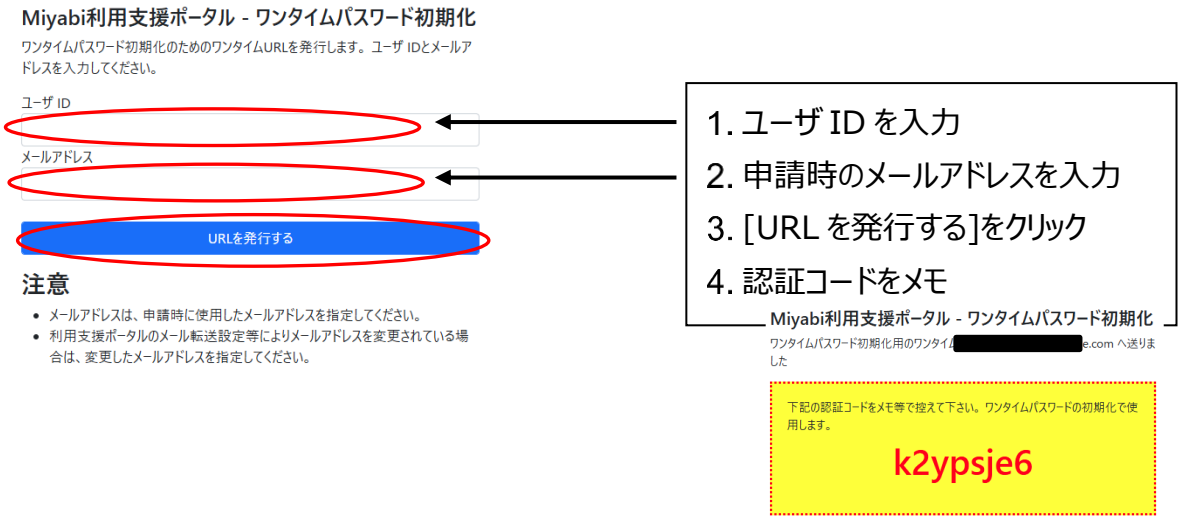


図 2-29 ワンタイムパスワード初期化画面

(4)　入力したメールアドレス宛に送られた URL にアクセスします。

メールの件名は“[Miyabi 利用支援ポータル] ワンタイムパスワード初期化用 URL”です。

Dear user username,

Miyabi 利用支援ポータルのワンタイムパスワード初期化用 URL をお知らせします。
https://miyabi-www.jcahpc.jp/resetotp//newotp/xxxx

(5)　新しいパスワードを設定します。上記(3)で控えた認証コードを入力し、「ワンタイムパスワードを初期化する」をクリックします。

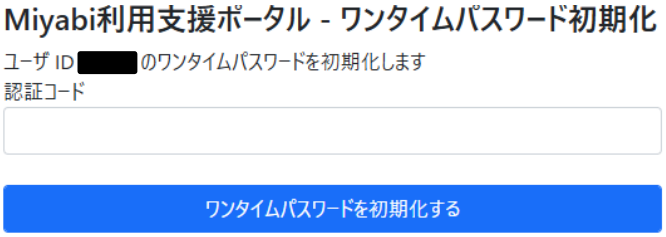


図 2-30 ワンタイムパスワード初期化画面

(6)　ワンタイムパスワード初期化に成功すると、下記メッセージが出力されます。

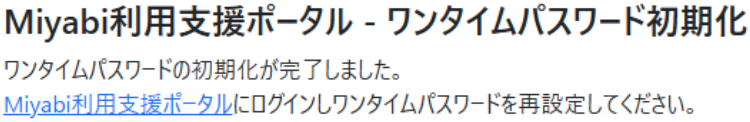


図 2-31 ワンタイムパスワード初期化完了画面

Copyright 2025 FUJITSU LIMITED

3. ファイルシステム環境

3.1. ファイルシステムの概要

Miyabi システムでは、以下のファイルシステム領域が利用可能です。

表 3-1 ファイルシステム領域一覧

ファイルシステム領域	ファイルシステム	サイズ	利用目的
共有ファイルシステム(/home)	Lustre	約 10PB	ホーム領域
共有ファイルシステム(/work)	Lustre	約 10PB	作業領域
Miyabi-G 計算ノードの内蔵ディスク (/local)	NVMe SSD	約 900GB	一時作業領域
Ipomoea-01 システムのストレージ	Lustre	約 23PB	データ保存領域

ホーム領域(/home)には、ユーザーあたり 50 GB のディスクが割り当てられます。

作業領域(/work)は利用申請の内容に応じたディスクがグループ単位で割り当てられます。プロジェクト内でファイルを共有する目的のために「/work/グループ名/share」が利用可能です。

表 3-2 割り当てディレクトリと割り当てサイズ

ファイルシステム領域	ディレクトリ	割り当てサイズ
共有ファイルシステム(/home)	/home/ユーザー名	50 GB
共有ファイルシステム(/work)	/work/グループ名/ユーザー名 /work/グループ名/share	5 TB（1 セットあたり）

ログインノード、および計算ノードから各ファイルシステムへのアクセス可否を以下に示します。

表 3-3 ファイルシステム領域へのアクセス可否一覧

ファイルシステム領域	Miyabi-G/Miyabi-C ログインノード	Miyabi-G/Miyabi-C 計算ノード
共有ファイルシステム	○	○
Ipoomea-01 システムのストレージ	○	×

○ ： 利用可能　　× ： 利用不能

Copyright 2025 FUJITSU LIMITED

Ipomoea-01 システムのストレージへのアクセスパスを表 3-4 に示します。

表 3-4 Ipomoea-01 システムのストレージへのアクセスパス

Ipomoea-01	Miyabi
/home/ユーザー名	/common1/home/ユーザー名
/work/グループ名	/common1/work/グループ名

Ipomoea-01 システムについての詳細は下記を参照ください。

https://www.cc.u-tokyo.ac.jp/supercomputer/ipomoea01/service/

3.1.1. ホームディレクトリの変更

chhome コマンドを実行しホームディレクトリを共有ファイルシステム(/work/グループ名/ユーザー名)に変更できます。chhome コマンドはログインノード上で実行してください。

```
$ chhome [option] <homedir>
```

chhome コマンドのオプションを表 3-5 *chhome* コマンドのオプション一覧に示します。

表 3-5 *chhome* コマンドのオプション一覧

オプション名	説明
なし	ホームディレクトリ($HOME)を homedir に変更します。 homedir に指定可能なのは以下の 2 種類です。 ● /home/ユーザー名 ● /work/グループ名/ユーザー名 .ssh などドット付きファイルも移動しますが、移動先に同名ファイルが存在する場合は上書きされません。
-f	移動先に.ssh などドット付きファイルがあっても上書きします。

例）ホームディレクトリを/work/groupname/username に変更する

```
[username@miyabi-g1~]$ chhome /work/groupname/username
Enter LDAP Password:   ← Miyabi システム(利用支援ポータル)のパスワードを入力
modifying entry "uid=username,ou=People,dc=Miyabi,dc=jcahpc,dc=jp"

[username@miyabi-g1 ~]$
```

ホームディレクトリの変更が反映されるまでに 10 分ほどかかります。変更後はしばらく時間をおいてから再度ログインしてください。

Copyright 2025 FUJITSU LIMITED

3.2. ディスク使用状況と上限値の表示

共有ファイルシステムのディスク使用状況と上限値を表示するには、*show_quota* コマンドを実行してください。

```
[username@miyabi-g1 work]$ show_quota
Disk quotas for user username
Directory                    used(MB)        limit(MB)        nfiles     nlimit
/home                        9,327           51,200           36,837     unlimited

Disk quotas for project group1 username
Directory          User      used(MB)        limit(MB)        nfiles     nlimit
/work/group1       -         200,000         204,800          1,000      unlimited
                   share           0         unlimited           10      unlimited
                   username  100,000         unlimited          500      unlimited
/work/group2       -         388,877,960     2,147,483,648    52,392,599   unlimited
                   share           0         unlimited           10      unlimited
                   username  100,000         unlimited          500      unlimited
```

表 3-6 show_quota の表示項目

項目	説明
Directory	割り当てディレクトリ /home： ログインノード用共有ファイルシステム /work/groupname： 共有ファイルシステム
used(MB)	ディスク使用量
limit(MB)	ディスク使用上限
nfiles	ファイル数
nlimit	ファイル数上限

共有ファイルシステムでは、1 ディレクトリ配下に作成できるファイル数に上限があります。上限値はファイル名の長さに依存し、約 4M～12M 個です。上限値を超えてファイルを作成しようとした場合、「No space left on device」のエラーが発生します。

Copyright 2025 FUJITSU LIMITED

3.3. 計算ノードの内蔵ディスク領域(/tmp 領域、/local 領域)の使用について

バッチジョブおよびインタラクティブジョブから内蔵ディスク領域を使用する場合は、以下の環境変数を使用ください。以下の環境変数は、ジョブ実行開始時に自動で設定されます。

表 3-7 /tmp 領域、/local 領域を使用するための環境変数

環境変数	設定値	利用システム
TMPDIR	/tmp/<ジョブ ID>.<4 桁の数字>	Miyaib-C Miyabi-G
LOCALDIR	/local/<ジョブ ID>.<4 桁の数字>	Miyabi-G

Copyright 2025 FUJITSU LIMITED

4. コンパイル/リンク

4.1. 環境設定

Miyabi システムの Miyabi-G 用のログインノード G と Miyabi-C 用のログインノード C では、各システムに対応するコンパイラを用いて、各システムの計算ノード向けに最適化されたロードモジュールを作成することが可能です。

表 4-1 ログインノードで利用可能なコンパイラ一覧

ソフトウェア名称		
NVIDIA HPC SDK	Miyabi-G	GPU 対応コンパイラ
NVIDIA CUDA Toolkit	Miyabi-G	GPGPU 統合開発環境
Intel OneAPI	Miyabi-C	インテルコンパイラ
GCC	Miyabi-G/Miyabi-C	GNU コンパイラ

module コマンドにて使用するコンパイラを load します。Miyabi-G のログインノード G と計算ノードでは、デフォルト設定として、NVIDIA HPC SDK の Environment Modules 環境(nvida と nv-hpcx)が設定されます。Miyabi-C のログインノード C と計算ノードでは、Intel OneAPI の Environment Modules 環境(intel と impi)が設定されます。

例1) NVIDIA HPC SDK のコンパイラおよび NVIDIA HPC SDK の MPI ライブラリ(HPC-X)を使用する設定

```
[username@miyabi-g1 work]$ module load nvidia
[username@miyabi-g1 work]$ module load nv-hpcx
[username@miyabi-g1 work]$ module list
Currently Loaded Modulefiles:
  1) nvidia/24.9     2) nv-hpcx/24.9
```

例2) NVIDIA CUDA Toolkit を使用する設定

```
[username@miyabi-g1 work]$ module purge
[username@miyabi-g1 work]$ module load cuda
[username@miyabi-g1 work]$ module list
Currently Loaded Modulefiles:
 1) cuda/12.6
```

例3) Intel OneAPI 及び IntelMPI 使用する設定

```
[username@miyabi-c1 work]$ module load intel
[username@miyabi-c1 work]$ module load impi
[username@miyabi-c1 work]$ module list
Currently Loaded Modulefiles:
 1) intel/2023.2.0     2) impi/2021.10.0
```

Copyright 2025 FUJITSU LIMITED

例4)　GCC および OpenMPI を使用する設定

```
[username@miyabi-g1 work]$ module purge
[username@miyabi-g1 work]$ module load gcc
[username@miyabi-g1 work]$ module load ompi
[username@miyabi-g1 work]$ module list
Currently Loaded Modulefiles:
 1) gcc/11.4.1     2) ompi/4.1.6
```

module コマンドの利用方法については、「2.6.4 実行パス/ライブラリ設定」を参照してください。

4.2. NVIDIA HPC SDK のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-2 NVIDIA HPC SDK のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション注 1	自動並列注 2	OpenMP注 2	OpenACC注 2
逐次 (非 MPI)	Fortran	nvfortran	-fast	-Mconcur	-mp	-acc
	C	nvc				
	C++	nvc++				
MPI 並列注 3	Fortran	mpifort				
	C	mpicc				
	C++	mpic++				

注 1: Miyabi-G 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: 自動並列、OpenMP、OpenACC オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は OpenMPI ライブラリの module のロードが必要です。

4.2.1. 逐次プログラムのコンパイル/リンク（NVIDIA HPC SDK）

逐次プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*nvfortran*/*nvc*/*nvc*++コマンドを使用します。

```
$ nvfortran/nvc/nvc++ [option] file [...]
```

例1) 逐次 Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvfortran -fast seq.f
```

例2) 逐次 C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvc -fast seq.c
```

Copyright 2025 FUJITSU LIMITED

4.2.2. スレッド並列プログラムのコンパイル/リンク（NVIDIA HPC SDK）

スレッド並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*nvfortran*/*nvc*/*nvc*++コマンドを使用します。

$ nvfortran/nvc/nvc++ thread-option [option] file [...]

例1) Fortran プログラムを Miyabi-G 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@miyabi-g1 work]$ nvfortran -fast -Mconcur para.f
```

例2) C プログラムを Miyabi-G 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@miyabi-g1 work]$ nvc -fast -Mconcur para.c
```

例3) OpenMP Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvfortran -fast -mp omp.f
```

例4) OpenMP C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvc -fast -mp omp.c
```

例5) OpenMP Fortran プログラムを Miyabi-G 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@miyabi-g1 work]$ nvfortran -fast -Mconcur -mp omp_para.f
```

例6) OpenMP C プログラムを Miyabi-G 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@miyabi-g1 work]$ nvc -fast -Mconcur -mp omp_para.c
```

Copyright 2025 FUJITSU LIMITED

4.2.3. OpenACC プログラムのコンパイル/リンク（NVIDIA HPC SDK）

OpenACC プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*nvfortran*/*nvc*/*nvc*++コマンドを使用します。

```
$ nvfortran/nvc/nvc++ -acc [option] file [...]
```

例1) OpenACC を使用した Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvfortran -fast -acc acc.f
```

例2) OpenACC を使用した C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvc -fast -acc acc.c
```

Copyright 2025 FUJITSU LIMITED

4.2.4. MPI 並列プログラムのコンパイル/リンク（NVIDIA HPC SDK + HPC-X）

MPI 並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合は、
mpifort/*mpicc*/*mpic*++コマンドを使用します。

```
$ mpifort/mpicc/mpic++ [option] file [...]
```

例1) MPI Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpifort -fast mpi.f
```

例2) MPI C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpicc -fast mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpifort -fast -mp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpicc -fast -mp mpi_omp.c
```

例5) MPI 並列と OpenACC を使用した Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpifort -fast -acc mpi_acc.f
```

例6) MPI 並列と OpenACC を使用した C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpicc -fast -acc mpi_omp.c
```

Copyright 2025 FUJITSU LIMITED

4.2.5. NVIDIA HPC SDK 推奨オプションによるプログラムの最適化（Fortran/C/C++）

計算ノード向けに実行性能を追求する最適化オプションとして、-fast を推奨します。
このオプションで翻訳すると、内部で以下のオプションが有効となります。

表 4-3 -fast で有効となる最適化オプション

オプション	説明
-O2	最適化レベル 2(最大 4)で翻訳します。
-Munroll=c:1	ループをアンロールします。
-Mlre	ループの冗長な処理を除去する最適化を行います。
-Mautoinline	関数をインライン展開します。
-Mvect=simd	SIMD 命令を生成します。
-Mflushz	flush-to-zero モードを使用します。
-Mcache_align	キャッシュ境界に合わせたオブジェクト割付けを行います。

上記の最適化オプションには副作用を伴うものが含まれているため、演算結果に影響を与える場合があります。使用する場合は、実行結果に問題がないかをご確認ください。

Copyright 2025 FUJITSU LIMITED

4.3. NVIDIA CUDA Toolkit のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-4 NVIDIA CUDA Toolkit のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション注 1	OpenMP 注 2
逐次（非MPI）	C	nvcc	-O3	-Xcomputer -fopenmp
	C++	nvcc -ccbin g++		
MPI 並列 注 3	C	nvcc -lmpi		
	C++	nvcc -ccbin g++ -lmpi		

注 1: Miyabi-G 計算ノードの CPU で実行されるコード部分の最適化オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は GCC および OpenMPI ライブラリの module のロードが必要です。

4.3.1. 逐次プログラムのコンパイル/リンク（NVIDIA CUDA Toolkit）

逐次プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*nvcc* コマンドを使用します。

```
$ nvcc [option] file [...]
```

例1) 逐次 CUDA プログラムを Miyabi-G 計算ノード（GPU）向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvcc -O3 cuda_seq.cu
```

4.3.2. スレッド並列プログラムのコンパイル/リンク（NVIDIA CUDA Toolkit）

スレッド並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*nvcc* コマンドを使用します。

```
$ nvcc thread-option [option] file [...]
```

例1) OpenMP 並列化された CUDA プログラムを Miyabi-G 計算ノード（GPU）向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvcc -O3 -Xcompiler -fopenmp cuda_omp.cu
```

Copyright 2025 FUJITSU LIMITED

4.3.3. MPI 並列プログラムのコンパイル/リンク（NVIDIA CUDA Toolkit + OpenMPI）

MPI 並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合は、*nvcc* コマンドを使用し、MPI のインクルードファイルやライブラリのパスをオプションで指定します。

```
$ nvcc [option] file [...] -lmpi
```

例1) MPI 並列化された CUDA プログラムを Miyabi-G 計算ノード（GPU）向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ nvcc -O3 -I$OMPI_INC -L$LIBRARY_PATH cuda_mpi.cu -lmpi
```

Copyright 2025 FUJITSU LIMITED

4.4. Intel oneAPI のコンパイル/リンク概要

Intel oneAPI の 2023.2.0(intel/2023.2.0)では、従来のインテルコンパイラである Intel oneAPI Compiler Classic(ICC)を利用することができます。コンパイル/リンクのコマンド一覧は以下です。

表 4-5 Intel oneAPI(Classic)のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション注 1	OpenMP 注 2
逐次 (非 MPI)	Fortran	ifort	-axSAPPHIRERAPIDS,CORE-AVX512	-qopenmp
	C	icc		
	C++	icpc		
MPI 並列 注 3	Fortran	mpiifort		
	C	mpiicc		
	C++	mpiicpc		

注 1: Miyabi-C 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は Intel MPI ライブラリの module のロードが必要です。

参考情報として、Intel oneAPI では LLVM ベースの新しい Intel oneAPI DPC++/C++ コンパイラを使用することができます。コンパイル/リンクのコマンド一覧は以下です。

表 4-6 (参考)Intel oneAPI DPC++/C++のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション注 1	OpenMP 注 2
逐次 (非 MPI)	Fortran	ifx	-axSAPPHIRERAPIDS,CORE-AVX512	-qopenmp
	C	icx		
	C++	icpx		
MPI 並列 注 3	Fortran	mpiifx		
	C	mpicx		
	C++	mpicpx		

注 1: Miyabi-C 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は Intel MPI ライブラリの module のロードが必要です。

Copyright 2025 FUJITSU LIMITED

4.4.1. 逐次プログラムのコンパイル/リンク（Intel OneAPI）

逐次プログラムを Miyabi-C 計算ノード向けにコンパイル/リンクする場合、*ifort/icc/icpc* コマンドを使用します。

$ ifort/icc/icpc [option] file [...]

例1) 逐次 Fortran プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ ifort -axSAPPHIRERAPIDS,CORE-AVX512 seq.f
```

例2) 逐次 C プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ icc -axSAPPHIRERAPIDS,CORE-AVX512 seq.c
```

4.4.2. スレッド並列プログラムのコンパイル/リンク（Intel OneAPI）

スレッド並列プログラムを Miyabi-C 計算ノード向けにコンパイル/リンクする場合、*ifort/icc/icpc* コマンドを使用します。

$ ifort/icc/icpc thread-option [option] file [...]

例1) Fortran プログラムを Miyabi-C 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@miyabi-c1 work]$ ifort -axSAPPHIRERAPIDS,CORE-AVX512 -parallel para.f
```

例2) C プログラムを Miyabi-C 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@miyabi-c1 work]$ icc -axSAPPHIRERAPIDS,CORE-AVX512 -parallel para.c
```

例3) OpenMP Fortran プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ ifort -axSAPPHIRERAPIDS,CORE-AVX512 -qopenmp omp.f
```

例4) OpenMP C プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ icc -axSAPPHIRERAPIDS,CORE-AVX512 -qopenmp omp.c
```

例5) OpenMP Fortran プログラムを Miyabi-C 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@miyabi-c1 work]$ ifort -axSAPPHIRERAPIDS,CORE-AVX512 -parallel -qopenmp
omp_para.f
```

Copyright 2025 FUJITSU LIMITED

例6) OpenMP C プログラムを Miyabi-C 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@miyabi-c1 work]$ icc -axSAPPHIRERAPIDS,CORE-AVX512 -parallel -qopenmp omp_para.c
```

4.4.3. MPI 並列プログラムのコンパイル/リンク（Intel OneAPI）

MPI 並列プログラムを Miyabi-C 計算ノード向けにコンパイル/リンクする場合は、*mpiifort*/*mpiicc*/*mpiicpc* コマンドを使用します。

```
$ mpiifort/mpiicc/mpiicpc [option] file [...]
```

例1) MPI Fortran プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ mpiifort -axSAPPHIRERAPIDS,CORE-AVX512 mpi.f
```

例2) MPI C プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ mpiicc -axSAPPHIRERAPIDS,CORE-AVX512 mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ mpiifort -axSAPPHIRERAPIDS,CORE-AVX512 -qopenmp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Miyabi-C 計算ノード向けにコンパイル/リンク

```
[username@miyabi-c1 work]$ mpiicc -axSAPPHIRERAPIDS,CORE-AVX512 -qopenmp mpi_omp.c
```

Copyright 2025 FUJITSU LIMITED

4.4.4. インテルコンパイラの最適化オプション(Fortran/C/C++)

計算ノード向けの最適化を促進するオプションとして代表的なものを以下に示します。

表 4-7 一般的な最適化オプション

オプション	説明
-ipo	複数のソースファイルにまたがるインライン展開やその他のプロシージャ間の最適化を許可します。
-O[0,1,2,3]	最適化のレベルを指定します。-O 未指定時は-O2 が有効です。
-no-prev-div	浮動小数点除算の精度を下げることによりパフォーマンスの向上を図ります。
-fp-model name	特定の最適化を制限することで、浮動小数点演算のパフォーマンス、精度、再現性を制御します。name の値には、fast=[1|2], precise, double, extended, source, except, strict を指定できます。 -fp-model 未指定時はパフォーマンスを重視する fast=1 が有効です。
-qopt-report[=n]	最適化レポートを生成します。デフォルトでは、レポートは.optrpt 拡張子を持つファイルに出力されます。n には、0(レポートなし)から 5(最も詳しい)の詳細レベルを指定します。デフォルトは 2 です。

上記の最適化オプションには副作用を伴うものが含まれているため、演算結果に影響を与える場合があります。使用する場合は、実行結果に問題がないかをご確認ください。

4.5. GCC のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-8 GCC のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション注 1	OpenMP 注 2
逐次 (非 MPI)	Fortran	gfortran	-O3	-fopenmp
	C	gcc		
	C++	g++		
MPI 並列 注 3	Fortran	mpifort		
	C	mpicc		
	C++	mpic++		

注 1: Miyabi-G 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は OpenMPI ライブラリの module のロードが必要です。

Copyright 2025 FUJITSU LIMITED

4.5.1. 逐次プログラムのコンパイル/リンク（GCC）

逐次プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*gfortran*/*gcc*/*g*++コマンドを使用します。

$ gfortran/gcc/g++ [option] file [...]

例1) 逐次 Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ gfortran -O3 seq.f
```

例2) 逐次 C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ gcc -O3 seq.c
```

4.5.2. スレッド並列プログラムのコンパイル/リンク（GCC）

スレッド並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合、*gfortran*/*gcc*/*g*++コマンドを使用します。

$ gfortran/gcc/g++ thread-option [option] file [...]

例1) OpenMP Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ gfortran -O3 -fopenmp omp.f
```

例2) OpenMP C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ gcc -O3   -fopenmp omp.c
```

4.5.3. MPI 並列プログラムのコンパイル/リンク（GCC + OpenMPI）

MPI 並列プログラムを Miyabi-G 計算ノード向けにコンパイル/リンクする場合は、*mpifort*/*mpicc*/*mpic*++コマンドを使用します。

$ mpifort/mpicc/mpic++ [option] file [...]

例1) MPI Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpifort -O3 mpi.f
```

Copyright 2025 FUJITSU LIMITED

例2) MPI C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpicc -O3 mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpifort -O3 -fopenmp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Miyabi-G 計算ノード向けにコンパイル/リンク

```
[username@miyabi-g1 work]$ mpicc -O3 -fopenmp mpi_omp.c
```

Copyright 2025 FUJITSU LIMITED

4.6. ライブラリ

4.6.1. NVIDIA 数学ライブラリ(Miyabi-G 用)

NVIDIA 数学ライブラリを利用する場合は、コンパイル/リンク時に以下のオプションを指定します。MAGMA を利用する際には、Environment-moudles で magma-gpu を load する必要があります。

表 4-9 cuBlas/cuFFTW/MAGMA オプション一覧

ライブラリ	並列性	指定オプション	備考
cuBlas	逐次	-cudalib=cublas	
cuFFT	逐次	-cudalib=cufft	
MAGMA	逐次	-lmagma_sparse -lmagma	

例1) C++プログラムから逐次版 cuBLAS を使用

```
[username@miyabi-c1 work]$ nvc++ -cudalib=cublas sample.cxx
```

例2) C++プログラムから逐次版 cuFFTW を使用

```
[username@miyabi-c1 work]$ nvc++ -cudalib=cufft sample.cxx
```

例3) C++プログラムから逐次版 MAGMA を使用

```
[username@miyabi-c1 work]$ module load magma-gpu
[username@miyabi-c1 work]$ nvc++ -lmagma_sparse -lmagma
 sample.cxx
```

上記以外の NVIDIA の数学ライブラリにつきましては、下記 URL を参照ください。

https://docs.nvidia.com/hpc-sdk/index.html#math-libraries

Copyright 2025 FUJITSU LIMITED

4.6.2. インテル MKL(Miyabi-C 用)

インテル MKL は、主要な計算分野における包括的な機能を提供します。

- BLAS(レベル 1、2、3) および LAPACK 線形代数ルーチン
- ScaLAPACK 分散処理線形代数ルーチン、BLACS および PBLAS
- インテル MKL PARDISO、反復法スパースソルバー、スパース BLAS(レベル 1、2、3)
- 高速フーリエ変換(FFT)関数
- ベクトルマス(VM)ルーチン
- ベクトル統計(VS)ルーチン
- データ・フィッティング・ライブラリ
- 拡張固有値ソルバ
- C 言語インターフェイスのディープ・ニューラル・ネットワーク(DNN)プリミティブ関数

BLAS/LAPACK/ScaLAPACK を利用する場合には、コンパイル/リンク時に以下のオプションを指定します。

表 4-10 BLAS/LAPACK/ScaLAPACK オプション一覧

ライブラリ	並列性	指定オプション	備考
BLAS/LAPACK	逐次	-qmkl=sequential	
	スレッド並列	-qmkl=parallel	
ScaLAPACK	MPI 並列	-qmkl=cluster	

例1)　Fortran プログラムから逐次版 BLAS/LAPACK を利用する。

```
[username@miyabi-c1 work]$ ifort -qmkl=sequential sample.f
```

例2)　Fortran プログラムからスレッド並列版 BLAS/LAPACK を利用する。

```
[username@miyabi-c1 work]$ ifort -qmkl=parallel sample.f
```

例3)　Fortran プログラムから ScaLAPACK を利用する(逐次版 BLAS/LAPACK をリンク)。

```
[username@miyabi-c1 work]$ mpiifort -qmkl=cluster sample.f
```

上記以外の組み合わせについては、下記 URL を参照してください。

https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-link-line-advisor.html

Copyright 2025 FUJITSU LIMITED

4.6.3. FFTW

Fortran/C プログラムにて FFTW(http://www.fftw.org/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.3.1. リンクオプション

FFTW ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-11 FFTW（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	精度	並列性	指定オプション	備考
FFTW	単精度	逐次	-lfftw3f	
		スレッド並列	-lfftw3f_omp	同時に-lfftw3f を指定する。
		MPI 並列	-lfftw3f_mpi	同時に-lfftw3f を指定する。
	倍精度	逐次	-lfftw3	
		スレッド並列	-lfftw3_omp	同時に-lfftw3 を指定する。
		MPI 並列	-lfftw3_mpi	同時に-lfftw3 を指定する。

4.6.3.2. 利用方法

Miyabi-G と Miyabi-C の利用方法を記載します。Miyabi-G の FFTW は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.3.2.1. Miyabi-G CPU 版

FFTW(Miyabi-G 用 逐次、スレッド並列)の環境設定

```
[username@miyabi-g1 work]$ module load fftw
```

FFTW(Miyabi-G 用 MPI 並列)の環境設定

```
[username@miyabi-g1 work]$ module load mpi-fftw
```

例1) Miyabi-G 計算ノード向けに Fortran プログラムから逐次版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ nvfortran sample.f -lfftw3
```

Copyright 2025 FUJITSU LIMITED

例2) Miyabi-G 計算ノード向けに C プログラムからスレッド並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ nvc -mp sample.c -lfftw3_omp -lfftw3
```

例3) Miyabi-G 計算ノード向けに C プログラムから MPI 並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ mpicc sample.c -lfftw3_mpi -lfftw3
```

例4) Miyabi-G 計算ノード向けに Fortran プログラムから逐次版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ nvfortran sample.f -lfftw3
```

例5) Miyabi-G 計算ノード向けに C プログラムからスレッド並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ nvc -mp sample.c -lfftw3_omp -lfftw3
```

例6) Miyabi-G 計算ノード向けに C プログラムから MPI 並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-g1 work]$ mpicc sample.c -lfftw3_mpi -lfftw3
```

4.6.3.2.2. Miyabi-C/プリポスト

FFTW(Miyabi-C 用 逐次、スレッド並列)の環境設定

```
[username@miyabi-c1 work]$ module load fftw
```

FFTW(Miyabi-C 用 MPI 並列)の環境設定

```
[username@miyabi-c1 work]$ module load intel
[username@miyabi-c1 work]$ module load impi
[username@miyabi-c1 work]$ module load mpi-fftw
```

例1) Miyabi-C 計算ノード向けに Fortran プログラムから逐次版 FFTW(倍精度)を利用する。

```
[username@miyabi-c1 work]$ ifort sample.f -lfftw3
```

例2) Miyabi-C 計算ノード向けに C プログラムからスレッド並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-c1 work]$ icc -mp sample.c -lfftw3_omp -lfftw3
```

例3) Miyabi-C 計算ノード向けに C プログラムから MPI 並列版 FFTW(倍精度)を利用する。

```
[username@miyabi-c1 work]$ mpiicc sample.c -lfftw3_mpi -lfftw3
```

Copyright 2025 FUJITSU LIMITED

4.6.4. GSL

C プログラムにて GSL(https://www.gnu.org/software/gsl/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.4.1. リンクオプション

GSL ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-12 GSL（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
GSL	逐次	-lgsl -lgslcblas	

4.6.4.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。Miyabi-G の GSL は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.4.2.1. Miyabi-G CPU 版

GSL(Miyabi-G 用)の環境設定

```
[username@miyabi-g1 work]$ module load gsl
```

例1) Miyabi-G 計算ノード向けに C プログラムから GSL を利用する。

```
[username@miyabi-g1 work]$ nvfortran sample.c -lgsl -lgslcblas
```

例2) Miyabi-G 計算ノード向けに C プログラムから GSL を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lgsl -lm
```

4.6.4.2.2. Miyabi-C/プリポスト

GSL(Miyabi-C 用)の環境設定

```
[username@miyabi-c1 work]$ module load gsl
```

Copyright 2025 FUJITSU LIMITED

例1)　Miyabi-C 計算ノード向けに C プログラムから GSL を利用する。

```
[username@miyabi-c1 work]$ icc -I$GSL_INC/gsl sample.c -lgsl -lgslcblas
```

例2) Miyabi-C 計算ノード向けに C プログラムから GSL および MKL を利用する。

```
[username@miyabi-c1 work]$ icc -I$GSL_INC/gsl sample.c -lgsl -qmkl
```

4.6.5. SuperLU/SuperLU_MT/SuperLU_DIST

Fortran/C プログラムにて SuperLU/SuperLU_MT（pthread 版）/SuperLU_DIST (http://crd-legacy.lbl.gov/~xiaoye/SuperLU/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.5.1. リンクオプション

SuperLU/SuperLU_MT/SuperLU_DIST ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-13 SuperLU/SuperLU_MT/SuperLU_DIST（Miyabi-G 用）リンクオプション一覧

利用ライブラリ	指定オプション	備考
SuperLU	-lsuperlu	同時に-lblas を指定する。
SuperLU_MT	-lsuperlu_mt	同時に-lblas を指定する。
SuperLU_DIST	-lsuperlu_dist	ParMETIS の利用環境も設定する。 同時に-lblas -lparmetis -lmetis を指定する。

表 4-14 SuperLU/SuperLU_MT/SuperLU_DIST（Miyabi-C 用）リンクオプション一覧

利用ライブラリ	指定オプション	備考
SuperLU	-lsuperlu	同時に-qmkl=sequential を指定する。
SuperLU_MT	-lsuperlu_mt	同時に-qmkl=sequential を指定する。
SuperLU_DIST	-lsuperlu_dist	ParMETIS の利用環境も設定する。 同時に-qmkl=cluster -lparmetis -lmetis を指定する。

Copyright 2025 FUJITSU LIMITED

4.6.5.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の SuperLU/SuperLU_MT/SuperLU_DIST は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.5.2.1. Miyabi-G CPU 版

SuperLU（Miyabi-G 用）の環境設定

```
[username@miaybi-g1 work]$ module load superlu
```

SuperLU_MT（Miyabi-G 用）の環境設定

```
[username@miaybi-g1 work]$ module load superlu_mt
```

SuperLU_DIST（Miyabi-G 用）の環境設定

```
[username@miaybi-g1 work]$ module load metis parmetis
[username@miaybi-g1 work]$ module load superlu_dist
```

例1) Miyabi-G 計算ノード向けに C プログラムから SuperLU を利用する。

```
[username@miaybi-g1 work]$ nvc sample.c -lsuperlu -lblas
```

例2) Miyabi-G 計算ノード向けに C プログラムから SuperLU_MT を利用する。

```
[username@miaybi-g1 work]$ nvc sample.c -lsuperlu_mt -lpthread -lblas
```

例3) Miyabi-G 計算ノード向けに C プログラムから SuperLU_DIST 利用する。

```
[username@miaybi-g1 work]$ mpicc sample.c -lsuperlu_dist -lparmetis -lmetis -lblas
```

4.6.5.2.2. Miyabi-C/プリポスト

SuperLU（Miyabi-C 用）の環境設定

```
[username@miaybi-c1 work]$ module load superlu
```

Copyright 2025 FUJITSU LIMITED

SuperLU_MT（Miyabi-C 用）の環境設定

```
[username@miaybi-c1 work]$ module load superlu_mt
```

SuperLU_DIST（Miyabi-C 用）の環境設定

```
[username@miaybi-c1 work]$ module load metis parmetis
[username@miaybi-c1 work]$ module load superlu_dist
```

例1) Miyabi-C 計算ノード向けに C プログラムから SuperLU を利用する。

```
[username@miaybi-c1 work]$ icc -qmkl=sequential sample.c -lsuperlu
```

例2) Miyabi-C 計算ノード向けに C プログラムから SuperLU_MT を利用する。

```
[username@miaybi-c1 work]$ icc -qmkl=sequential sample.c -lsuperlu_mt -lpthread
```

例3) Miyabi-C 計算ノード向けに C プログラムから SuperLU_DIST 利用する。

```
[username@miaybi-c1 work]$ mpiicc -qmkl=cluster sample.c -lsuperlu_dist -lparmetis -lmetis
```

4.6.6. METIS/ParMETIS

Fortran/C プログラムにて METIS/ParMETIS (http://glaros.dtc.umn.edu/gkhome/views/metis/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.6.1. リンクオプション

METIS/ParMETIS ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-15 METIS/ParMETIS（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
METIS	逐次	-lmetis	
ParMETIS	MPI 並列	-lparmetis	同時に-lmetis を指定する。

Copyright 2025 FUJITSU LIMITED

4.6.6.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の METIS/ParMETIS は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.6.2.1. Miyabi-G CPU 版

METIS（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load metis
```

ParMETIS（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load metis
[username@miyabi-g1 work]$ module load parmetis
```

例1) Miyabi-G 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lmetis
```

例2) Miyabi-G 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@miyabi-g1 work]$ mpicc sample.c -lparmetis -lmetis
```

4.6.6.2.1. Miyabi-C/プリポスト

METIS（Miyabi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load metis
```

ParMETIS（Miyabi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load metis
[username@miyabi-c1 work]$ module load parmetis
```

Copyright 2025 FUJITSU LIMITED

例1) Miyabi-C 計算ノード向けに C プログラムから METIS を利用する。

```
[username@miyabi-c1 work]$ icc sample.c -lmetis
```

例2) Miyabi 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@miyabi-c1 work]$ mpiicc sample.c -lparmetis -lmetis
```

Copyright 2025 FUJITSU LIMITED

4.6.7. Scotch/PT-Scotch

Fortran/C プログラムにて Scotch/PT-SCotch(http://www.labri.fr/perso/pelegrin/scotch/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.7.1. リンクオプション

Scotch/PT-Scotch ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-16 Scotch/PT-Scotch（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
Scotch	逐次	-lscotch -lscotcherr	
PT-Scotch	MPI 並列	-lptscotch -lscotch -lptscotcherrexit	

4.6.7.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の Scotch/PT-Scotch は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.7.2.1. Miyabi-G CPU 版

Scotch（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load scotch
```

PT-Scotch（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load scotch
[username@miyabi-g1 work]$ module load pt-scotch
```

Copyright 2025 FUJITSU LIMITED

例1) Miyabi-G 計算ノード向けに C プログラムから Scotch を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lscotch -lscotcherr
```

例2) Miyabi-G 計算ノード向けに C プログラムから PT-Scotch を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lptscotch -lscotch -lptscotcherrexit
```

4.6.7.2.2. Miyabi-C/プリポスト

Scotch（Miaybi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load scotch
```

PT-Scotch（Miyabi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load scotch
[username@miyabi-c1 work]$ module load pt-scotch
```

例1) Miyabi-C 計算ノード向けに C プログラムから Scotch を利用する。

```
[username@miyabi-c1 work]$ icc sample.c -lscotch -lscotcherr
```

例2) Miyabi-C 計算ノード向けに C プログラムから PT-Scotch を利用する。

```
[username@miyabi-c1 work]$ mpiicc sample.c -lptscotch -lscotch -lptscotcherrexit
```

Copyright 2025 FUJITSU LIMITED

4.6.8. HDF5/Parallel HDF5

Fortran/C/C++プログラムにて HDF5 (http://www.hdfgroup.org/HDF5/)を利用することが可能です。また、Fortran/C プログラムにて Parallel HDF5 を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.8.1. リンクオプション

HDF5, Parallel HDF5 ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-17 HDF5, Parallel HDF5（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	並列性	使用コンパイラ	指定オプション	備考
HDF5	逐次	C	-lhdf5	
			-lhdf5_hl	high level library
		C++	-lhdf5_cpp	
			-lhdf5_hl_cpp	high level library
		Fortran	-lhdf5_fortran	
			-lhdf5hl_fortran	high level library
PHDF5	MPI 並列	C	-lhdf5	
			-lhdf5_hl	high level library
		Fortran	-lhdf5_fortran	
			-lhdf5hl_fortran	high level library

4.6.8.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の HDF5/Parallel HDF5 は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.8.2.1. Miyabi-G CPU 版

HDF5（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load hdf5
```

Copyright 2025 FUJITSU LIMITED

Parallel HDF5（Miyabi-G 用）の環境設定

```
[username@miyabi-g1 work]$ module load phdf5
```

例1) Miyabi-G 計算ノード向けに C プログラムから HDF5 を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lhdf5
```

例2) Miyabi-G 計算ノード向けに C++プログラムから HDF5 を利用する。

```
[username@miyabi-g1 work]$ nvc++ sample.c -lhdf5_cpp -lhdf5
```

例3) Miyabi-G 計算ノード向けに Fortran プログラムから Parallel HDF5 を利用する。

```
[username@miyabi-g1 work]$ nvfortran sample.f90 -lhdf5_fortran -lhdf5 -I${PHDF5_INC}
```

4.6.8.2.2. Miyabi-C/プリポスト

HDF5（Miyabi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load hdf5
```

Parallel HDF5（Miyabi-C 用）の環境設定

```
[username@miyabi-c1 work]$ module load phdf5
```

例1) Miyabi-C 計算ノード向けに C プログラムから HDF5 を利用する。

```
[username@miyabi-c1 work]$ icc sample.c -lhdf5
```

例2) Miyabi-C 計算ノード向けに C++プログラムから HDF5 を利用する。

```
[username@miyabi-c1 work]$ icpc sample.c -lhdf5_cpp -lhdf5
```

例3) Miyabi-C 計算ノード向けに Fortran プログラムから Parallel HDF5 を利用する。

```
[username@miyabi-c1 work]$ mpiifort sample.f90 -lhdf5_fortran -lhdf5
```

Copyright 2025 FUJITSU LIMITED

4.6.9. netCDF

Fortran/C/C++プログラムにて netCDF(http://www.unidata.ucar.edu/software/netcdf/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。なお、netCDF 利用時には、HDF5 の利用環境の設定が必要です。HDF5 の設定の詳細については、4.6.8 を参照してください。

4.6.9.1. リンクオプション

netCDF ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-18 netCDF（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	並列性	使用コンパイラ	指定オプション	備考
netCDF	逐次	C	-lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。
		C++	-lnetcdf_c++4 -lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。
		Fortran	-lnetcdff -lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。

4.6.9.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の netCDF は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.9.2.1. Miyabi-G CPU 版

netCDF(Miyabi-G 用、C 言語版)の環境設定

```
[username@miyabi-g1 work]$ module load netcdf
[username@miyabi-g1 work]$ module load hdf5
```

netCDF(Miyabi-G 用、Fortran 版)の環境設定

```
[username@miyabi-g1 work]$ module load netcdf-fortran
[username@miyabi-g1 work]$ module load hdf5
```

Copyright 2025 FUJITSU LIMITED

netCDF(Miyabi-G 用、C++版)の環境設定

```
[username@miyabi-g1 work]$ module load netcdf-cxx
[username@miyabi-g1 work]$ module load hdf5
```

netCDF 並列版(Miyabi-G 用、MPI 並列版)の環境設定

```
[username@miyabi-g1 work]$ module load netcdf-parallel
[username@miyabi-g1 work]$ module load phdf5
```

例1) Miyabi-G 計算ノード向けに C プログラムから netCDF を利用する。

```
[username@miyabi-g1 work]$ nvc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

例2) Miyabi-G 計算ノード向けに Fortran プログラムから netCDF を利用する。

```
[username@miyabi-g1 work]$ nvfortran sample.f -lnetcdff -lnetcdf -lhdf5_hl -lhdf5 -I${NETCDF_FORTRAN_INC}
```

例3) Miyabi-G 計算ノード向けに C++プログラムから netCDF を利用する。

```
[username@miyabi-g1 work]$ nvc++ sample.f90 -lnetcdf_c++4 -lnetcdf -lhdf5_hl -lhdf5
```

例4) Miyabi-G 計算ノード向けに C プログラムから netCDF 並列版を利用する。

```
[username@miyabi-g1 work]$ mpicc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

4.6.9.2.2. Miyabi-C/プリポスト

netCDF(Miyabi-C 用、C 言語版)の環境設定

```
[username@miyabi-c1 work]$ module load netcdf
[username@miyabi-c1 work]$ module load hdf5
```

netCDF(Miyabi-C 用、Fortran 版)の環境設定

```
[username@miyabi-c1 work]$ module load netcdf-fortran
[username@miyabi-c1 work]$ module load hdf5
```

netCDF(Miyabi-C 用、C++版)の環境設定

```
[username@miyabi-c1 work]$ module load netcdf-cxx
[username@miyabi-c1 work]$ module load hdf5
```

Copyright 2025 FUJITSU LIMITED

netCDF 並列版(Miyabi-C 用、MPI 並列版)の環境設定

```
[username@miyabi-c1 work]$ module load netcdf-parallel
[username@miyabi-c1 work]$ module load phdf5
```

例1) Miyabi-C 計算ノード向けに C プログラムから netCDF を利用する。

```
[username@miyabi-c1 work]$ icc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

例2) Miyabi-C 計算ノード向けに Fortran プログラムから netCDF を利用する。

```
[username@miyabi-c1 work]$ ifort sample.f -lnetcdff -lnetcdf -lhdf5_hl -lhdf5
```

例3) Miyabi-C 計算ノード向けに C++プログラムから netCDF を利用する。

```
[username@miyabi-c1 work]$ icpc sample.f90 -lnetcdf_c++4 -lnetcdf -lhdf5_hl -lhdf5
```

例4) Miyabi-C 計算ノード向けに C プログラムから netCDF 並列版を利用する。

```
[username@miyabi-g1 work]$ mpiicc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

Copyright 2025 FUJITSU LIMITED

4.6.10. Parallel netCDF

Fortran/C プログラムにて Parallel netCDF(https://parallel-netcdf.github.io/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.10.1. リンクオプション

Parallel netCDF ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-19 Parallel netCDF（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	指定オプション	備考
Parallel netCDF	-lpnetcdf	

4.6.10.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の Parallel netCDF は CPU 上で動作するライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.10.2.1. Miyabi-G CPU 版

Parallel netCDF(Miyabi-G 用)の環境設定

```
[username@miyabi-g1 work]$ module load pnetcdf
```

例1) Miyabi-C 計算ノード向けに C プログラムから Parallel netCDF を利用する。

```
[username@miyabi-g1 work]$ mpicc sample.c -lpnetcdf
```

4.6.10.2.2. Miyabi-C/プリポスト

Parallel netCDF(Miyabi-C 用)の環境設定

```
[username@miyabi-c1 work]$ module load pnetcdf
```

例1) Miyabi-C 計算ノード向けに C プログラムから Parallel netCDF を利用する。

```
[username@miyabi-c1 work]$ mpiicc sample.c -lpnetcdf
```

Copyright 2025 FUJITSU LIMITED

4.6.11. PETSc

C/C++/Fortran コンパイラから PETSc(http://www.mcs.anl.gov/petsc/)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

4.6.11.1. リンクオプション

PETSc ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-20 Parallel netCDF（Miyabi-G/Miyabi-C 共通）リンクオプション一覧

利用ライブラリ	指定オプション	備考
PETSc	-lpetsc	

4.6.11.2. 利用方法

Miyabi-G と Miyabi-C での利用方法を記載します。

Miyabi-G の PETSc は GPU 上での動作に対応したライブラリを提供しています。ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Miyabi-G では、NIVIDA HPC SDK 利用環境、Miyabi-C では、Intel oneAPI 利用環境が自動で設定されます。以下の利用方法は、NIVIDA HPC SDK 利用環境、および、Intel oneAPI 利用環境が自動設定された状態での利用例を記載しています。

4.6.11.2.1. Miyabi-G GPU 版

PETSc(Miyabi-G 用)の環境設)

```
[username@miyabi-g1 work]$ module load petsc-gpu
```

例 1）KSP の tutorials/ex1.c をコンパイルする場合

・ KSP の tutorials をコピー

```
[username@miyabi-g1 ~]$ cp -r $PETSC_DIR/share/petsc/examples/src/ksp/ksp/tutorials .
[username@miyabi-g1 ~]$ cd tutorials
[username@miyabi-g1 tutorials]$ make ex1
```

Copyright 2025 FUJITSU LIMITED

4.6.11.2.2. Miyabi-C/プリポスト

PETSc(Miyabi-C 用)の環境設定

```
[username@miyabi-c1 work]$ module load petsc
```

例 1）KSP の tutorials/ex1.c をコンパイルする場合(Miyabi-G/Miyabi-C 共通)

・ KSP の tutorials をコピー

```
[username@miyabi-g1 ~]$ cp -r $PETSC_DIR/share/petsc/examples/src/ksp/ksp/tutorials .
[username@miyabi-g1 ~]$ cd tutorials
[username@miyabi-g1 tutorials]$ make ex1
```

Copyright 2025 FUJITSU LIMITED

5. バッチジョブ・インタラクティブジョブ

5.1. ジョブ管理システム概要

Miyabi システムでは、ジョブ管理システムによりバッチジョブおよびインタラクティブジョブの実行が制御されます。ユーザーはジョブ実行に必要なキュー名、ノード数、経過時間などを指定し、ジョブ管理システムにジョブ実行を依頼します。依頼されたジョブはジョブ管理システムにより資源の空き状況やジョブ間の優先度などを元に自動で実行開始されます。

Miyabi システムで利用可能なジョブ形式は以下の 2 種類です。

表 5-1 ジョブの形式

ジョブ形式	用途
バッチジョブ	バッチ形式でジョブを実行する。 ノードダウンなどの異常発生時、オプションによりジョブの自動再実行が可能。
インタラクティブジョブ	ユーザー端末からのデータ入力により、会話形式でジョブを実行する。 主にデバッグ等での利用を想定。 ノードダウンなどの異常発生時、ジョブは再実行されない。

バッチジョブは、以下の 3 種類の種別に分類されます。

表 5-2 バッチジョブの種別

バッチジョブ種別	用途	投入形式
通常ジョブ	スクリプト単位でジョブを実行する	「5.4.3.1」を参照
チェーンジョブ	投入した複数のジョブを 1 つのまとまりとして扱い、その中で実行順序、依存関係をもつジョブ	「5.4.3.2」を参照
アレイジョブ	1 度のジョブ投入で、複数の通常ジョブを生成し、実行するジョブ	「5.4.3.3」を参照

ジョブ操作のためのコマンドを表 5-3 に示します。

表 5-3 ジョブ操作コマンド一覧

機能	コマンド名
ジョブ投入	qsub
ジョブ参照	qstat
ジョブ削除	qdel
ジョブ保留	qhold
ジョブ保留の解除	qrls

Copyright 2025 FUJITSU LIMITED

5.2. ジョブ実行リソース

ジョブ管理システムは、キューという単位で計算ノードを管理します。ユーザーは、ジョブを投入する際にどのキューでジョブを実行するか指定します。

5.2.1. バッチジョブ用キュー

Miyabi-G/Miyabi-C で指定可能なキューを表 5-5、表 5-6、表 5-7 に示します。ジョブ投入時にルーティングキュー「regular-g/regular-c」を指定した場合は、ノード数に応じて自動的にキューが選択されます。

Miyabi-G のキューは、ノード専有利用と MIG 利用の 2 つが存在します。ノード専有利用とはジョブに割り当てるリソースを 1 ノード単位とする運用、MIG 利用とはジョブに割り当てるリソースを MIG インスタンス単位とする運用です。MIG 利用のキューでは、NVIDIA GH200 Grace Hopper Superchip を 4 分割した MIG インスタンスを提供いたします。MIG インスタンス当たりに割り当てられるリソース情報は表 5-4 のとおりです。GPU リソース（SM 数）の少ない MIG インスタンス#3 を回避するジョブ投入オプションはありません。

表 5-4 MIG インスタンス情報

	MIG インスタンス#0	MIG インスタンス#1	MIG インスタンス#2	MIG インスタンス#3
GPU リソース	32 SMs、24 GB	32 SMs、24 GB	32 SMs、24 GB	26 SMs、24 GB
CPU リソース	18 コア、25 GiB	18 コア、25 GiB	18 コア、25 GiB	18 コア、25 GiB

表 5-5 Miyabi-G バッチジョブ用キュー(ノード専有)

ルーティングキュー	キュー	割り当て ノード数	最小 ノード数	最大 ノード数	ジョブあたりの 実行時間制限	ノードあたりの 最大メモリ容量
debug-g		48	1	16	30 分	100GiB
short-g		24	1	8	8 時間	
regular-g	small-g	1024	1	16	48 時間	
	medium-g		17	64	48 時間	
	large-g		65	128	48 時間	
	x-large-g		129	256	24 時間	

表 5-6 Miyabi-G バッチジョブ用キュー(MIG 利用)

キュー	割り当てノード数 (総 MIG インスタンス数)	指定可能 MIG インスタンス数	ジョブあたりの 実行時間制限	MIG インスタンスあたりの 最大メモリ容量
debug-mig	12(48)	1,2,4	30 分	25GiB
short-mig	4(16)	1,2,4	8 時間	
regular-mig	8(32)	1,2,4	48 時間	

Copyright 2025 FUJITSU LIMITED

表 5-7 Miyabi-C バッチジョブ用キュー

ルーティングキュー	キュー	割り当て ノード数	最小 ノード数	最大 ノード数	ジョブあたりの 実行時間制限	ノードあたりの 最大メモリ容量
debug-c		16	1	4	30 分	118GiB
short-c		6	1	2	8 時間	
regular-c	small-c	168	1	16	48 時間	
	medium-c		17	32	48 時間	
	large-c		33	64	48 時間	

申請された基本セットによりに表 5-8、表 5-9 示す各種制限があります。Miyabi-G/Miyabi-C 共にアレイジョブについては、サブジョブを 1 ジョブとして取り扱い制限します。

表 5-8 Miyabi-G バッチジョブのグループ(プロジェクト)毎の制限

制限	グループコース
ジョブ同時受付制限数	ジョブ同時実行制限数 × 8
ジョブ同時実行制限数	申し込み 2 セットまで 4 本、 4 セットまで 8 本、 8 セットまで 16 本、 以降、8 セット単位で 2 本ずつ追加
ノード同時使用制限数	256
GPU(MIG インスタンス)同時使用制限数	24
ジョブ 1 本あたりの最大ノード数	256

表 5-9 Miyabi-C バッチジョブのグループ(プロジェクト)毎の制限

制限	グループコース
ジョブ同時受付制限数	ジョブ同時実行制限数 × 2
ジョブ同時実行制限数	申し込み 16 セットまで 2 本、 以降、16 セット単位で 1 本ずつ追加し、最大 8 本まで
ノード同時使用制限数	64
ジョブ 1 本あたりの最大ノード数	64

5.2.2. インタラクティブジョブ用キュー

インタラクティブジョブ実行時に指定可能なキューを表 5-10、表 5-11、表 5-12 に示します。インタラクティブジョブ用キューは以下の特徴を有します。

- インタラクティブジョブとして利用可能です。
- ジョブを実行してもトークンが消費されません。
- 所属プロジェクトの数によらず、1 ユーザーあたり 1 本のジョブのみ投入・実行が可能です。

Copyright 2025 FUJITSU LIMITED

表 5-10 Miyabi-G インタラクティブジョブ用キュー(ノード専有)

ルーティングキュー	キュー	割り当てノード数	最小ノード数	最大ノード数	ジョブあたりの実行時間制限	ノードあたりの最大メモリ容量
interact-g	intract-g_n1	48	1	1	2 時間	100GiB
	intract-g_n8		2	8	10 分	

表 5-11 Miyabi-G インタラクティブジョブ用キュー(MIG 利用)

ルーティングキュー	キュー	割り当てノード数 (総 MIG インスタンス数)	指定可能 MIG インスタンス数	ジョブあたりの実行時間制限	MIG インスタンスあたりの最大メモリ容量
interact-mig	intract-mig_g1	12(48)	1	2 時間	25GiB

表 5-12 Miyabi-C インタラクティブジョブ用キュー

ルーティングキュー	キュー	割り当てノード数	最小ノード数	最大ノード数	ジョブあたりの実行時間制限	ノードあたりの最大メモリ容量
interact-c	intract-c_n1	16	1	1	2 時間	118GiB
	intract-c_n2		2	2	10 分	

5.2.3. プリポスト処理用キュー

ログインノードで行う作業のうち負荷が高い処理を実行する場合、プリポスト処理用キューを利用してください。プリポスト処理用キューは以下の特徴を有します。

- インタラクティブジョブとして利用可能です。
- ログインノード(Miyabi-C)と同一の環境で処理を行うことができます。
- ログインノード(Miyabi-G)と同一環境のプリポスト環境は用意しておりません。
- ジョブを実行してもトークンは消費されません。
- 所属プロジェクトの数によらず、1 ユーザーあたり 1 本のみジョブの投入・実行が可能です。
- 利用支援ポータルから日時指定でリソースを予約することができます。

表 5-13 プリポスト処理用キュー

キュー	割り当てノード数	最小ノード数	最大ノード数	ジョブあたりの実行時間制限	ノードあたりの最大メモリ容量
prepost	3	1	1	6 時間	240GiB
prepost1_n1 注 1	1	1	1	1～6 時間	
prepost2_n1 注 1	1	1	1	1～6 時間	
prepost1_n2 注 1	2	1	2	1～6 時間	

注 1: 利用支援ポータルのプリポスト予約から予約することで利用可能になります。

Copyright 2025 FUJITSU LIMITED

5.3. ジョブ投入オプション

ジョブ投入時は、ジョブの実行に応じて、以下の 2 つのオプションを指定します。Miyabi-G/Miyabi-C 共通で利用できるオプションです。

- 基本オプション
- ジョブ資源オプション

5.3.1. 基本オプション

ジョブ投入時に指定する主な基本オプションを表 5-14 に示します。赤文字のオプションは、必ず指定する必要があります。

表 5-14 主なジョブ投入基本オプション

オプション		説明
-q <name>		キュー名/ルーティングキュー名を指定
-W		ジョブ属性を指定
	group_list=<*projectcode*>	ジョブ実行時にトークンを消費するプロジェクトを指定
	depend=<*dependency list*>	チェーンジョブの依存ルールを指定
-o <*filename*>		ジョブの標準出力を指定されたファイルに出力 同名のファイルが存在する場合は、既存ファイルに追記で出力する。
-e <*filename*>		ジョブの標準エラー出力を指定されたファイルに出力 同名のファイルが存在する場合は、既存ファイルに追記で出力する。
-j oe		ジョブの標準エラー出力を標準出力へ出力
-N <*jobname*>		ジョブ名を指定
-I [-X]		インタラクティブジョブとしてジョブを投入(-X で X11 転送を許可)
-J <range> <start>-<end>[:step]		アレイジョブとしてジョブを投入
-m <mail events>		メール送信条件を指定
	n	メールを送信しない
	a	ジョブ異常終了時にメールを送信 (デフォルト)
	b	ジョブ開始時にメールを送信
	e	ジョブ終了時にメールを送信
-V		ジョブ投入時の環境変数をジョブ実行環境に継承
-r <*y/n*>		ジョブが再実行可能か指定

Miyabi のジョブ管理システムは、マルチバイト文字がサポートされていません。そのため、ジョブに引き継ぐ環境変数やディレクトリ名等にマルチバイト文字が含まれている場合、ジョブの投入に失敗します。

Copyright 2025 FUJITSU LIMITED

5.3.2. ジョブ資源オプション

ジョブが利用する資源に関する主要オプションを表 5-15 に示します。ジョブ資源オプションは-l オプションを用いて指定します。赤文字のオプションは、必ず指定する必要があります。各資源は":"区切りで指定します。

表 5-15 Miyabi-G 用ジョブ資源オプション

オプション		説明
-l		ジョブ実行に必要な資源値を指定
	select=<*num*>	ノード数の指定（ノード専有利用の場合） MIG インスタンス数を指定（MIG 利用の場合） ルーティングキューにジョブを投入した場合、本設定により自動でキューが選択されます。
	walltime=<*elapselimit*>	経過時間制限値を指定（[[hour:]minute:]second で指定） 適切な経過時間を指定することでバックフィル機能(*1)によりジョブの実行開始が早まる可能性があります。 (*1) 先に投入したジョブの実行開始予定を遅らせることなく、空いているリソースにジョブを割り当てて実行する機能
	mpiprocs=<*num*>	ノードおよび MIG インスタンスあたりの MPI プロセス数を指定
	ompthreads=<num>	OpenMP スレッド数を指定
	mem=limit	ジョブに割り当てられたノード単位、または、MIG インスタンス単位の使用メモリ制限を指定
	filesystem=name	ジョブを実行するファイルシステム名を指定。 ジョブ投入時にファイルシステム状態が異常の場合、ジョブ実行を抑止する（HELD 状態でジョブ投入する）。
	mail_power_info=true	ジョブ終了時のメール通知内容に電力情報を含める。
	pcore=limit	プロセス当たりの core ファイルサイズを指定

ジョブ投入例) debug-g キューに、2 ノードを使用し、総 MPI 並列数 8 のジョブを投入

```
[username@miyabi-g1 work]$ qsub -q debug-g -l select=2:mpiprocs=4 ¥
                                -W group_list=group   run.sh
123456.opbs
```

Copyright 2025 FUJITSU LIMITED

5.4. バッチジョブ実行

5.4.1. ジョブスクリプト作成

バッチジョブを投入するためには、実行する計算プログラムとは別に「ジョブスクリプト」を作成します。ジョブスクリプトには利用するキュー、ノード数、経過時間などの資源や利用するプロジェクトを記載したオプションを記述した上で、実行するコマンド列を記載します。

(1) 先頭行は〝#!〟に続けて、シェルのパスを指定します。

[記述例]

```
#!/bin/bash                                    bash を利用
```

ログインシェルが *bash* 以外で、*sh* で記述されたジョブスクリプト内で *module* コマンドを実行する場合は、「#!/bin/sh -l」を指定する必要があり、*bash* で記述されたジョブスクリプト内で *module* コマンドを実行する場合は、「#!/bin/bash -l」を指定する必要があります。

(2) 2 行目以降にジョブ投入オプションをディレクティブ"**#PBS**"を用いて指定します。

[記述例]

```
#PBS -q debug-g                 キューを指定
#PBS -l select=1                ノード数を指定
#PBS -W group_list=group1       グループを指定
#PBS -j oe                      標準エラーを標準出力マージして出力
```

(3) ジョブ投入オプションに続けて、実行時の環境変数設定と、プログラム実行を指定します。

[記述例]

```
cd ${PBS_O_WORKDIR}             ジョブ投入ディレクトリに移動
./a.out                         プログラムを実行
```

Copyright 2025 FUJITSU LIMITED

5.4.2. ジョブ環境変数

ジョブスクリプト内で利用可能な環境変数を表 5-16 に示します。

表 5-16 ジョブスクリプト内で参照可能な環境変数（Miyabi-G/Miyabi-C 共通）

環境変数	説明
PBS_ENVIRONMENT	バッチジョブでは"BATCH"、会話型ジョブでは"INTERACT"
PBS_JOBID	ジョブ ID (<シーケンス番号>.opbs)
PBS_JOBNAME	ジョブ名
PBS_O_WORKDIR	qsub コマンド実行時のカレントディレクトリ
PBS_NODEFILE	ホストファイル
PBS_ARRAY_INDEX	アレイジョブのインデックス番号
MPI_PROC	総 MPI 並列数 (MPI ライブラリの module を load したときのみ参照可能)
MPI_PROC_PER_NODE	ノード内、MPI 並列数 (MPI ライブラリの module を load したときのみ参照可能)

ログインノード上で設定した環境変数は、指示がない限り、ジョブ実行環境には引き継がれません。ジョブ実行環境に引き継ぐためには、*qsub* コマンドの-V オプションを指定します。

5.4.3. バッチジョブ投入

バッチジョブを投入する場合、*qsub* コマンドの引数にバッチジョブとして実行するジョブスクリプトを指定します。

5.4.3.1. 通常ジョブ投入

通常ジョブを投入する場合、*qsub* コマンドの引数にバッチジョブとして実行するジョブスクリプトを指定します。

```
$ qsub [option] [job-script]
```

- ジョブスクリプトを指定しない場合、標準入力から実行命令を読み込みます。
- ジョブ投入が完了後、ジョブに対して識別用 ID(ジョブ ID)が割り当てられます。

例) 通常ジョブ投入例

```
[username@miyabi-g1 work]$ qsub run.sh
123456.opbs
```

Copyright 2025 FUJITSU LIMITED

5.4.3.2. チェーンジョブ投入

チェーンジョブは、複数のバッチジョブを 1 つのまとまりとして扱い、その中で実行の順序関係や依存関係を指定することで、ジョブチェーン機能を実現するジョブモデルです。チェーンジョブのイメージは「図 5-1 チェーンジョブイメージ」に記載しています。

チェーンジョブの投入形式は以下のとおりです。投入するジョブと依存関係になる投入済のジョブ（依存ジョブ）ID とその依存関係(type)を指定します。

```
$ qsub -W depend=<type>:<依存ジョブ ID>[,<type>:<依存ジョブ ID> ...] jobscript
```

表 5-17 ステップジョブ依存関係式

type	説明
after:<依存ジョブ ID>	依存ジョブがすべて開始後にスケジューリングされる
afterok:<依存ジョブ ID>	依存ジョブがすべてエラーなしで終了したあとにスケジューリングされる
afternotok:<依存ジョブ ID>	依存ジョブがすべてエラーで終了したあとにスケジューリングされる
afterany:<依存ジョブ ID>	依存ジョブすべて終了した後(エラーは問わない)にスケジューリングされる
before:<依存ジョブ ID>	投入したジョブが開始したら、依存ジョブがスケジューリングされる
beforeok:<依存ジョブ ID>	投入したジョブがエラーなしで終了したあとに、依存ジョブがスケジューリングされる
beforenotok:<依存ジョブ ID>	投入したジョブがエラーで終了した後に、依存ジョブがスケジューリングされる
beforeany:<依存ジョブ ID>	投入したがジョブが終了した後(エラーは問わない)に、依存ジョブがスケジューリングされる

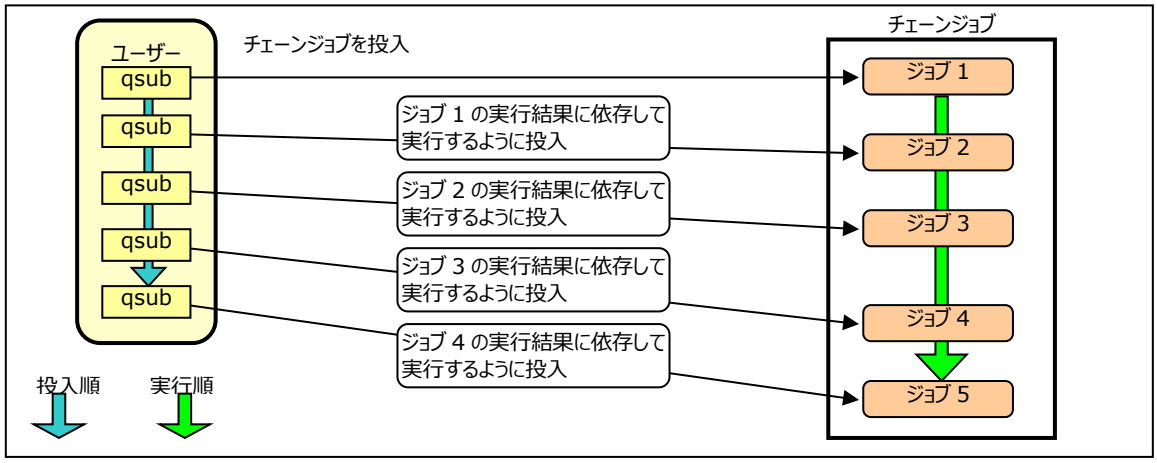


図 5-1 チェーンジョブイメージ

例1) 投入順に実行される 3 本のチェーンジョブ

```
[username@miyabi-g1work]$ qsub step1.sh
123456.opbs
[username@miyabi-g1 work]$ qsub -W depend=afterok:123456.opbs step2.sh
123457.opbs
[username@miyabi-g1 work]$ qsub -W depend=afterok:123457.opbs step3.sh
123458.opbs
```

Copyright 2025 FUJITSU LIMITED

5.4.3.3. アレイジョブ投入

アレイジョブは、複数の同じバッチジョブを同時に投入し、実行するジョブです。例えば、ジョブのパラメタを変えて、それぞれの実行結果を確認したい場合、通常のバッチジョブであれば、ジョブを 1 つ 1 つ投入する必要がありますが、アレイジョブを使えば、1 度に複数パターンの投入ができます。

アレイジョブの投入形式は以下のとおりです。アレイジョブを使用する場合、-r y オプションの指定が必須です。

```
$ qsub -r y -J <range> [%<max subjobs>] jobscript
```

例）8 つのサブジョブが input.1～input.8 の入力ファイルをそれぞれ読み込み、出力ファイル output.1～output.8 をそれぞれ出力する

```
[username@miyabi-g1 work]$ ls input.*
input.1 input 2 input.3 input.4 input.5 input.6 input.7 input.8
[username@miyabi-g1 work]$ vi run.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out < input.${PBS_ARRAY_INDEX}

[username@miyabi-g1 work]$ qsub -r y -J 1-8 array.sh
123456[].opbs
```

Copyright 2025 FUJITSU LIMITED

5.4.4. バッチジョブの標準出力と標準エラー出力

バッチジョブの標準出力ファイルと標準エラー出力ファイルは、ジョブ投入ディレクトリもしくは、ジョブ投入時に指定されたファイルに出力されます。

標準出力ファイルにはジョブ実行中の標準出力、標準エラー出力ファイルにはジョブ実行中のエラーメッセージが出力されます。ジョブ投入時に、標準出力ファイル、標準エラー出力ファイルを指定しなかった場合は、以下のファイルに出力されます。

- **ジョブ名.**oXXXXX　---　標準出力ファイル
- **ジョブ名.**eXXXXX　---　標準エラー出力ファイル
  (XXXXX はジョブ投入時に割り当てられたジョブのシーケンス番号)

Copyright 2025 FUJITSU LIMITED

5.4.5. Miyabi-G(NVIDIA HPC SDK)用バッチジョブスクリプト例

Miyabi-G へのバッチジョブ実行時に自動で NVIDIA HPC SDK (nvidia 及び nv-hpcx)が load されるため、バッチジョブスクリプト内での記述(*module load nvidia*, *module load nv-hpcx*)は不要です。

5.4.5.1. 単一ノード逐次ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-2 に示します。

・キュー : debug-g
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi seq.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-2 Miyabi-G 単一ノード逐次ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.5.2. 単一 MIG インスタンス逐次ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-3 に示します。

・キュー : debug-mig
・MIG インスタンス数 : 1
・プロセス数 : 1 プロセス(オプション不要)
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi seq_mig.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-mig
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-3 Miyabi-G 単一 MIG インスタンス逐次ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.5.3. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-4 に示します。

・キュー : short-g
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 1 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi para.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q short-g
#PBS -l select=1:ompthreads=12
#PBS -l walltime=01:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-4 単一ノードスレッド並列ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.5.4. 単一 MIG インスタンス スレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-5 に示します。

・キュー : debug-mig
・MIG インスタンス数 : 1
・プロセス数 : 1 プロセス(オプション不要)
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi para_mig.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-mig
#PBS -l select=1:ompthreads=12
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-5 単一 MIG インスタンス スレッド並列ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.5.5. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-6 に示します。

・キュー : regular-g
・ノード数 : 1 ノード
・プロセス数 : 48 プロセス
・経過時間 : 24 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=1:mpiprocs=48
#PBS -l walltime=24:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-6 単一ノード MPI 並列ジョブ用スクリプト

mpiprocs には、ノード内の MPI 並列数を指定します。

Copyright 2025 FUJITSU LIMITED

5.4.5.6. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-7 に示します。

・キュー : short-g
・ノード数 : 1 ノード
・プロセス数 : 4 プロセス
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 4 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi single-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q short-g
#PBS -l select=1:mpiprocs=4:ompthreads=12
#PBS -l walltime=04:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-7 単一ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、
qsub コマンドの-l select=<num>:mpiprocs=<num>:ompthreads=<num>
オプションの指定が必要です。

Copyright 2025 FUJITSU LIMITED

5.4.5.7. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-8 に示します。

・キュー : regular-g
・ノード数 : 2 ノード
・プロセス数 : 96 プロセス
・経過時間 : 12 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi multi-mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=48
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-8 複数ノード MPI 並列ジョブ用スクリプト

MPI 並列ジョブを実行する場合、qsub コマンドの mpiprocs オプションを指定してください。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.5.8. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-9 に示します。

・キュー : regular-g
・ノード数 : 2 ノード
・プロセス数 : 8 プロセス
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi multi-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=4:ompthreads=12
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-9 複数ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、qsub コマンドの mpiprocs オプション、ompthreads オプションの指定が必要です。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.6. Miyabi-C(Intel OneAPI)用バッチジョブスクリプト例

Miyabi-C へのバッチジョブ実行時に自動で Intel OneAPI (intel 及び impi)が load されるため、バッチジョブスクリプト内での記述(*module load intel*, *module load impi*)は不要です。

5.4.6.1. 単一ノード逐次ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-10 に示します。

・キュー : debug-c
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi seq.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-c
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-10 Miyabi-C 単一ノード逐次ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.6.2. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-11 に示します。

・キュー : short-c
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・1 プロセスあたりのスレッド数 : 64 スレッド
・経過時間 : 1 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi para.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-c
#PBS -l select=1:ompthreads=64
#PBS -l walltime=01:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
./a.out
```

図 5-11 単一ノードスレッド並列ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.6.3. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-12 に示します。

・キュー : debug-c
・ノード数 : 1 ノード
・プロセス数 : 112 プロセス
・経過時間 : 15 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-c
#PBS -l select=1:mpiprocs=112
#PBS -l walltime=15:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpiexec.hydra ./a.out
```

図 5-12 単一ノード MPI 並列ジョブ用スクリプト

mpiprocs には、ノード内の MPI 並列数を指定します。

Copyright 2025 FUJITSU LIMITED

5.4.6.4. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-13 に示します。

・キュー : short-c
・ノード数 : 1 ノード
・プロセス数 : 4 プロセス
・1 プロセスあたりのスレッド数 : 8 スレッド
・経過時間 : 4 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi single-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q short-c
#PBS -l select=1:mpiprocs=4:ompthreads=8
#PBS -l walltime=04:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpiexec.hydra ./a.out
```

図 5-13 単一ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、
qsub コマンドの-l select=<num>:mpiprocs=<num>:ompthreads=<num>
オプションの指定が必要です。

Copyright 2025 FUJITSU LIMITED

5.4.6.5. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-14 に示します。

・キュー : regular-c
・ノード数 : 2 ノード
・プロセス数 : 96 プロセス
・経過時間 : 12 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi multi-mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-c
#PBS -l select=2:mpiprocs=48
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpiexec.hydra ./a.out
```

図 5-14 複数ノード MPI 並列ジョブ用スクリプト

MPI 並列ジョブを実行する場合、qsub コマンドの mpiprocs オプションを指定してください。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.6.6. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-15 に示します。

・キュー : regular-c
・ノード数 : 2 ノード
・プロセス数 : 8 プロセス
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 24 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-c1 work]$ vi multi-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-c
#PBS -l select=2:mpiprocs=4:ompthreads=12
#PBS -l walltime=24:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpiexec.hydra ./a.out
```

図 5-15 複数ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、qsub コマンドの mpiprocs オプション、ompthreads オプションの指定が必要です。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.7. GCC（OpenMPI 使用含む）用バッチジョブスクリプト例

Miyabi-G へのバッチジョブ実行時に自動的に NVIDIA HPC SDK の環境、Miyabi-C へのバッチジョブ実行時には自動的に Intel OneAPI の環境となるため、GCC 環境を使用するためには、*module* コマンドで GCC 環境に切り替える必要がございます。以下のジョブスクリプト例は Miyabi-G での実行ですがキューを変更することで Miyabi-C でも同様に実行することができます。

5.4.7.1. 単一ノード逐次ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-16 に示します。

・キュー : debug-g
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・経過時間 : 30 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi seq.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc

cd ${PBS_O_WORKDIR}
./a.out
```

図 5-16 単一ノード逐次ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.7.2. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-17 に示します。

・キュー : short-g
・ノード数 : 1 ノード
・プロセス数 : 1 プロセス(オプション不要)
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 1 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi para.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1:ompthreads=12
#PBS -l walltime=01:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc

cd ${PBS_O_WORKDIR}
./a.out
```

図 5-17 単一ノードスレッド並列ジョブ用スクリプト

Copyright 2025 FUJITSU LIMITED

5.4.7.3. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-18 に示します。

・キュー : regular-g
・ノード数 : 1 ノード
・プロセス数 : 48 プロセス
・経過時間 : 24 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=1:mpiprocs=48
#PBS -l walltime=24:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc
module load ompi

cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-18 単一ノード MPI 並列ジョブ用スクリプト

MPI 並列ジョブを実行する場合、qsub コマンドの mpiprocs オプションを指定してください。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.7.4. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-19 に示します。

・キュー : debug-g
・ノード数 : 1 ノード
・プロセス数 : 4 プロセス
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 15 分
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi single-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1:mpiprocs=4:ompthreads=12
#PBS -l walltime=15:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc
module load ompi

cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-19 単一ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、
qsub コマンドの-l select=<num>: mpiprocs=<num>: ompthreads=<num>
オプションの指定が必要です。

Copyright 2025 FUJITSU LIMITED

5.4.7.5. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-20 に示します。

・キュー : regular-g
・ノード数 : 2 ノード
・プロセス数 : 96 プロセス
・経過時間 : 24 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi multi-mpi.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=2:mpiprocs=48
#PBS -l walltime=24:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc
module load ompi

cd ${PBS_O_WORKDIR}
mpirun  ./a.out
```

図 5-20 複数ノード MPI 並列ジョブ用スクリプト

MPI 並列ジョブを実行する場合、qsub コマンドの mpiprocs オプションを指定してください。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.4.7.6. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-21 に示します。

・キュー : regular-g
・ノード数 : 2 ノード
・プロセス数 : 8 プロセス
・1 プロセスあたりのスレッド数 : 12 スレッド
・経過時間 : 4 時間
・課題番号 : group1
・標準エラー出力を標準出力にマージ : あり

```
[username@miyabi-g1 work]$ vi multi-hybrid.sh
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=4:ompthreads=12
#PBS -l walltime=04:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
module purge
module load gcc
module load ompi

cd ${PBS_O_WORKDIR}
mpirun ./a.out
```

図 5-21 複数ノードハイブリッド並列ジョブ用スクリプト

ハイブリッド並列ジョブを実行する場合、qsub コマンドの mpiprocs オプション、ompthreads オプションの指定が必要です。mpiprocs オプションには、総 MPI 並列数 = mpiprocs 指定数 x 要求ノード数となるように指定してください。

Copyright 2025 FUJITSU LIMITED

5.5. インタラクティブジョブ実行

インタラクティブジョブを実行するためには、qsub コマンドに-I オプションを付与して実行します。ジョブ管理システムがインタラクティブジョブに対して計算資源を割り当てることでジョブの実行が開始され、会話処理が可能になります。

インタラクティブジョブ投入時は、ジョブ投入オプションを引数として指定します。

```
$ qsub -I [option...]
```

5.5.1. Miyabi-G 用インタラクティブジョブ実行例

Miyabi-G へのインタラクティブジョブは、実行時に自動で NVIDIA HPC SDK が load されるため、インタラクティブジョブ実行中の *module load nvidia, module load nv-hpcx* は実行不要です。

例1) Miyabi-G インタラクティブジョブ逐次実行

```
[username@miyabi-g1 work]$ qsub -I -l select=1 -W group_list=group1 ¥
        -q interact-g -l walltime=01:00:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mg0001 ~]$ cd ${PBS_O_WORKDIR}
[username@mg0001 work]$ nvc hello_world.c
[username@mg0001 work]$ ./a.out
Hello world
[username@mg0001 work]$ exit
logout
qsub: job 123456.opbs completed
```

例2) Miyabi-G インタラクティブジョブ単一ノード MPI 並列実行

```
[username@miyabi-g1 work]$ qsub -I -l select=1:mpiprocs=48 ¥
        -W group_list=group1 -q interact-g -l walltime=01:00:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mg0001 ~]$ cd ${PBS_O_WORKDIR}
[username@mg0001 work]$ mpicc mpi_hello.c
[username@mg0001 work]$ mpirun ./a.out
Hello world from rank 2 process
Hello world from rank 12 process
Hello world from rank 40 process
(省略)
[username@mg0001 work]$ exit
logout
qsub: job 123456.opbs completed
```

例3) Miyabi-G インタラクティブジョブ複数ノード MPI 並列実行

Copyright 2025 FUJITSU LIMITED

```
[username@miyabi-g1 work]$ qsub -I -l select=2:mpiprocs=48 ¥
        -W group_list=group1 -q interact-g -l walltime=00:10:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mg0001 ~]$ cd ${PBS_O_WORKDIR}
[username@mg0001 work]$ mpicc mpi_hello.c
[username@mg0001 work]$ mpirun ./a.out
Hello world from rank 2 process
Hello world from rank 12 process
Hello world from rank 40 process
(省略)
[username@mg0001 work]$ exit
logout
qsub: job 123456.opbs completed
```

5.5.2. Miyabi-C 用インタラクティブジョブ実行例

Miyabi-C へのインタラクティブジョブは、実行時に自動で Intel OneAPI が load されるため、インタラクティブジョブ実行中の *module load intel, module load impi* は実行不要です。

例4) Miyabi-C インタラクティブジョブ逐次実行

```
[username@miyabi-c1 work]$ qsub -I -l select=1 -W group_list=group1 ¥
        -q interact-c -l walltime=01:00:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mc001 ~]$ cd ${PBS_O_WORKDIR}
[username@mc001 work]$ icc hello_world.c
[username@mc001 work]$ ./a.out
Hello world
[username@mc001 work]$ exit
logout
qsub: job 123456.opbs completed
```

例5) Miyabi-C インタラクティブジョブ単一ノード MPI 並列実行

```
[username@miyabi-c1 work]$ qsub -I -l select=1:mpiprocs=48 ¥
        -W group_list=group1 -q interact-c -l walltime=01:00:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mc001 ~]$ cd ${PBS_O_WORKDIR}
[username@mc001 work]$ mpiicc mpi_hello.c
[username@mc001 work]$ mpiexec.hydra -n ${MPI_PROC} ¥
> ./a.out
Hello world from rank 2 process
Hello world from rank 12 process
Hello world from rank 40 process
(省略)
[username@mc001 work]$ exit
logout
qsub: job 123456.opbs completed
```

Copyright 2025 FUJITSU LIMITED

例6) Miyabi-C インタラクティブジョブ複数ノード MPI 並列実行

```
[username@miyabi-g1 work]$ qsub -I -l select=2:mpiprocs=48 ¥
        -W group_list=group1 -q interact-g -l walltime=00:10:00
qsub: waiting for job 123456.opbs to start
qsub: job 123456.opbs ready
[username@mc001 ~]$ cd ${PBS_O_WORKDIR}
[username@mc001 work]$ mpiicc mpi_hello.cm
[username@mc001 work]$ mpiexec.hydra ./a.out
Hello world from rank 2 process
Hello world from rank 12 process
Hello world from rank 40 process
(省略)
[username@mc001 work]$ exit
logout
qsub: job 123456.opbs completed
```

Copyright 2025 FUJITSU LIMITED

5.6. MPI プログラムの実行

5.6.1. Miyabi-G での MPI プログラムの実行

MPI プログラムを実行する場合、*mpirun* コマンドを使用します。

```
$ mpirun [global_option] [local_option] 実行モジュール : ....
```

Miyabi-G 向けの *mpirun* コマンドの主なオプションを表 5-18 に示します。

表 5-18 *mpirun* コマンドの主なオプション（Miyabi-G 向け）

オプション	説明
-n *proc*	MPI プログラムの並列プロセス数を指定します。
-x *env*	リモートノードに引き継ぐ環境変数を指定します。注 1
-hostfile *hostfile*	使用するホストファイルを指定します。
--map-by node	ノード毎にラウンドロビンでプロセスを生成します。
--map-by ppr:*n*:node	各ノードに生成するプロセス数を指定します。

注 1: Miyabi-G にて MPI 環境(NVIDIA HPC SDK、OpenMPI)を、Environment-modules を使用して設定した場合、環境変数 OMPI_MCA_mca_base_env_list に環境変数 PATH と LD_LIBRALY_PATH が自動設定されます。そのため、-x env オプションを使用する場合は、環境変数 OMPI_MCA_mca_base_env_list を削除する必要があります。

例) -x *env* でリモートノードに環境変数 MIYABI を引き継ぐジョブスクリプトの例

```
[username@miyabi-g1 work]$ vi run.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=48
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
unset OMPI_MCA_mca_base_env_list
export MIYABI=G

cd ${PBS_O_WORKDIR}
mpirun -x PATH -x LD_LIBRARY_PATH ¥
        -x MIYABI ¥
        ./a.out
```

Copyright 2025 FUJITSU LIMITED

5.6.2. Miyabi-C での MPI プログラムの実行(Intel MPI 用)

Intel MPI でプログラムを実行する場合、mpiexec.hydra コマンドを使用します。

```
$ mpiexec.hydra [global_option] [local_option] 実行モジュール : …
```

表 5-19 *mpiexec.hydra* コマンドの主なオプション

オプション	説明
-n *proc*	MPI プログラムの並列プロセス数を指定します。
-rr	ノード毎にラウンドロビンでプロセスを生成します。
-s spec	標準入力を指定された MPI プロセスにリダイレクトします(デフォルト: 0)。
-prepend-rank	並列プロセスの標準出力の行の先頭に、ランク番号を付加します。

5.6.2.1. MPI 実行に関する環境変数（Intel MPI 用）

MPI プログラム実行時に使用可能な主な環境変数を表 5-20 に示します。

表 5-20 MPI プログラム実行に利用可能な主な環境変数（Intel MPI 用）

環境変数	説明
I_MPI_PIN	プロセスのピニングを制御します。
I_MPI_PIN_DOMAIN	MPI/OpenMP ハイブリッドアプリケーションのプロセスピニングを制御します。
I_MPI_FABRICS_LIST	使用するファブリックリストを定義します。
I_MPI_FABRICS	ノード内通信およびノード間通信で使用するファブリックを指定します。
I_MPI_FALLBACK	使用できるファブリックにフォールバックするかを指定します。
I_MPI_PERHOST	ノード毎に割り当てるプロセス数を指定します。 ジョブ投入時に指定されたノード数とプロセス数を元に自動で設定されます。
I_MPI_EAGER_THRESHOLD	全てのデバイスの eager/rendezvous メッセージサイズの閾値を変更します。
I_MPI_DEBUG	MPI プログラムの実行が開始するとデバッグ情報を出力します。

5.6.2.2. MPI 実行に関する注意事項

ノード内にプロセスを複数生成する場合、ノード内のプロセス数が増えるにつれて MPI 初期化処理に時間がかかるようになります。

Copyright 2025 FUJITSU LIMITED

5.7. 高度なジョブ実行

5.7.1. Miyabi-G での高度なジョブ実行

5.7.1.1. SHARP 利用

Miyabi-G では、NVIDIA Scalable Hierarchical Aggregation and Reduction Protocol (SHARP)を利用できます。SHARP を利用する際は以下の環境変数を設定する必要があります。

- SHARP_SMX_ENABLED_PROTOCOLS=1
- SHARP_SMX_PROTOCOL=1

例1) NVIDIA HPC SDK の HPC-X を利用するジョブスクリプト

```
[username@miyabi-g1 work]$ vi sharp_hello.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=48
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
export SHARP_SMX_ENABLED_PROTOCOLS=1
export SHARP_SMX_PROTOCOL=1
unset OMPI_MCA_mca_base_env_list
mpirun -x PATH -x LD_LIBRARY_PATH ¥
        -x SHARP_SMX_ENABLED_PROTOCOLS ¥
        -x SHARP_SMX_PROTOCOL ¥
        ./a.out
```

Copyright 2025 FUJITSU LIMITED

5.7.1.2. 複数 MIG インスタンスの利用

Miyabi-G の MIG 利用のジョブにて複数の MIG インスタンスを利用する際は以下のようなラッパスクリプトを作成し、プログラム実行時のコマンドラインに指定します。

同一 GPU 内、あるいは、同一ノード内の MIG デバイス間の GPUDirect P2P はサポートされていません。また、回避策もございません。MIG 利用に関する考慮事項は以下をご参照ください。

https://docs.nvidia.com/datacenter/tesla/mig-user-guide/

「7.2. Application Considerations」

例1) 複数 MIG インスタンスを利用するラッパスクリプトの作成

```
[username@miyabi-g1 work]$ vi set_mig_device.sh
#!/bin/sh
MIG_UUID=(${CUDA_VISIBLE_DEVICES//,/ })
MIG_RANK=$((OMPI_COMM_WORLD_LOCAL_RANK/MPI_PROC_PER_MIG))
export CUDA_VISIBLE_DEVICES=${MIG_UUID[$MIG_RANK]}
$@
[username@miyabi-g1 work]$ chmod +x ./set_mig_device.sh
```

例2) 4MIG インスタンス、48 プロセスでプログラムを実行するジョブスクリプト例

```
[username@miyabi-g1 work]$ vi run_mig.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-mig
#PBS -l select=4:mpiprocs=12
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./set_mig_device.sh ./a.out
```

Copyright 2025 FUJITSU LIMITED

5.7.1.3. MPS 利用

Miyabi-G では MPS(Multi-Process Service)を利用できます。MPS を利用する際は nvidia-cuda-mps-control コマンドを使用し MPS デーモンを起動する必要があります。

```
$ nvidia-cuda-mps-control -d
```

MPS デーモンを停止する際は以下のコマンドを実行します。

```
$ echo quit | nvidia-cuda-mps-control
```

例1) 1 ノード 32 プロセスで MPS を利用してプログラムを実行するジョブスクリプト例

```
[username@miyabi-g1 work]$ vi run_mps.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=1:mpiprocs=32
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
nvidia-cuda-mps-control -d
mpirun ./a.out
echo quit | nvidia-cuda-mps-control
```

表 5-21 MPS に関連する環境変数

環境変数	説明
CUDA_MPS_PIPE_DIRECTORY	MPS のプロセス間通信に必要なパイプおよびソケットファイルを配置するディレクトリを指定します。
CUDA_MPS_LOG_DIRECTORY	MPS サーバおよび MPS クライアントのログを保存するディレクトリを指定します。

これらの環境変数は各ユーザごとに一意のパス設定が必要ですが、本システムでは、ジョブ実行時に自動設定されるため個別の設定は不要です。ただし、複数の MIG インスタンス上で MPS を利用する場合は各 MIG インスタンスごとに一意のパスになるように設定する必要があります。複数 MIG インスタンスを使用した場合の MPS 利用方法は、5.7.1.3.3 章を参照ください。

5.7.1.3.1. MPS クライアント数制限

同時に起動可能な MPS クライアント数の最大値はノード専有利用の場合は 48、MIG 利用の場合は 1MIG インスタンスあたり 14 となります。

Copyright 2025 FUJITSU LIMITED

5.7.1.3.2. 複数ノードでの MPS 利用

複数ノードで MPS を利用してプログラムを実行する際は以下のようなラッパスクリプトを作成し、プログラム実行時のコマンドラインに指定します。

例1) 複数ノードで MPS を利用するラッパスクリプトの作成

```
[username@miyabi-g1 work]$ vi set_mps.sh
#!/bin/sh
atexit () {
    trap - EXIT
    if [ $OMPI_COMM_WORLD_LOCAL_RANK -eq 0 ]; then
        ps aux | grep nvidia-cuda-mps-control | grep -v grep > /dev/null
        [ $? -eq 0 ] && echo quit | nvidia-cuda-mps-control
        [ -d $CUDA_MPS_PIPE_DIRECTORY ] && rm -rf $CUDA_MPS_PIPE_DIRECTORY
        [ -d $CUDA_MPS_LOG_DIRECTORY ] && rm -rf $CUDA_MPS_LOG_DIRECTORY
    fi
}

if [ $OMPI_COMM_WORLD_LOCAL_RANK -eq 0 ]; then
    trap atexit EXIT HUP INT QUIT TERM USR1 USR2
    nvidia-cuda-mps-control -d
fi

$@
[username@miyabi-g1 work]$ chmod +x ./set_mps.sh
```

例2) 2 ノード、64 プロセスで MPS を利用してプログラムを実行するジョブスクリプト例

```
[username@miyabi-g1 work]$ vi run_multi_mps.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-g
#PBS -l select=2:mpiprocs=32
#PBS -l walltime=12:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
unset OMPI_MCA_mca_base_env_list
mpirun -x PATH -x LD_LIBRARY_PATH ¥
        -x CUDA_MPS_PIPE_DIRECTORY ¥
        -x CUDA_MPS_LOG_DIRECTORY ¥
        ./set_mps.sh ./a.out
```

Copyright 2025 FUJITSU LIMITED

5.7.1.3.3. 複数 MIG インスタンスでの MPS 利用

複数の MIG インスタンス上で MPS を利用してプログラムを実行する際は以下のようなラッパスクリプトを作成し、プログラム実行時のコマンドラインに指定します。

例1) 複数 MIG インスタンスで MPS を利用するラッパスクリプトの作成

```
[username@miyabi-g1 work]$ vi set_mps_mig.sh
#!/bin/sh
atexit () {
    trap - EXIT
    if [ $LOCAL_RANK_MIG -eq 0 ]; then
        ps aux | grep nvidia-cuda-mps-control | grep -v grep > /dev/null
        [ $? -eq 0 ] && echo quit | nvidia-cuda-mps-control
        [ -d $CUDA_MPS_PIPE_DIRECTORY ] && rm -rf $CUDA_MPS_PIPE_DIRECTORY
        [ -d $CUDA_MPS_LOG_DIRECTORY ] && rm -rf $CUDA_MPS_LOG_DIRECTORY
    fi
}

MIG_UUID=(${CUDA_VISIBLE_DEVICES//,/ })
MIG_RANK=$((OMPI_COMM_WORLD_LOCAL_RANK/MPI_PROC_PER_MIG))
LOCAL_RANK_MIG =$((OMPI_COMM_WORLD_LOCAL_RANK%MPI_PROC_PER_MIG))
export CUDA_MPS_PIPE_DIRECTORY=$CUDA_MPS_PIPE_DIRECTORY-$MIG_RANK
export CUDA_MPS_LOG_DIRECTORY=$CUDA_MPS_LOG_DIRECTORY-$MIG_RANK
export CUDA_VISIBLE_DEVICES=${MIG_UUID[$MIG_RANK]}

if [ $LOCAL_RANK_MIG -eq 0 ]; then
    trap atexit EXIT HUP INT QUIT TERM USR1 USR2
    nvidia-cuda-mps-control -d
fi

$@
[username@miyabi-g1 work]$ chmod +x ./set_mps_mig.sh
```

例2) 2MIG インスタンス、16 プロセスで MPS を利用してプログラムを実行するジョブスクリプト例

```
[username@miyabi-g1 work]$ vi run_multi_mps_mig.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q regular-mig
#PBS -l select=2:mpiprocs=8
#PBS -l walltime=2:00:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
mpirun ./set_mps_mig.sh ./a.out
```

Copyright 2025 FUJITSU LIMITED

5.7.2. Miyabi-C での高度なジョブ実行

5.7.2.1. OpenMP スレッド並列プログラム実行

5.7.2.1.1. Intel コンパイラの OpenMP スレッド並列実行に関する環境変数

OpenMP スレッド並列実行時に指定可能な主な環境変数を表 5-22 に示します。

表 5-22 OpenMP スレッド並列実行に利用可能な主な環境変数

環境変数	説明
OMP_NUM_THREADS	OpenMP もしくは自動並列によりスレッド並列化されたプログラムのスレッド数を指定します。デフォルトではジョブ投入時に指定されたスレッド数が設定されますが、ジョブスクリプト中で再設定した場合は、その値が有効になります。
KMP_AFFINITY	スレッドのコアバインドを制御します。
KMP_STACKSIZE	プライベートスタックとして使用する各 OpenMP スレッドに割り当てるバイト数を指定します。
KMP_BLOCKTIME	parallel region の実行が終了した後、スレッドがスリープ状態になるまでスレッドが待機する時間(ミリ秒単位)を指定します。

5.7.2.1.2. OpenMP スレッド並列実行におけるコア割り当て

Intel コンパイラの OpenMP を用いたスレッド並列プログラムでは、環境変数 KMP_AFFINITY の設定により、物理コアへのスレッド割り当てが変更されます。

表 5-23 環境変数 KMP_AFFINITY

設定値	説明
balanced	可能な限り異なる物理コアを使用し、ノード内のコア全体にスレッドを均等に分配します。scatter と異なり、物理コアに割り当てられるスレッドが互いに隣接します。
scatter	可能な限り異なる物理コアを使用し、ノード内のコア全体にスレッドを均等に分配します。scatter は、compact の逆の動作です。
compact	可能な限り同じ物理コアを使用します。
none	スレッドはバインドされません。(デフォルト)

Copyright 2025 FUJITSU LIMITED

例） KMP_AFFINITY 環境変数に compact を指定してプログラムを実行するジョブスクリプト

```
#!/bin/sh

#------ qsub option --------#
#PBS -q regular-c
#PBS -l select=1:ompthreads=12
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe

#------- Program execution -------#
cd ${PBS_O_WORKDIR}
export KMP_AFFINITY=compact

./a.out
```

Copyright 2025 FUJITSU LIMITED

5.8. ジョブ状態表示

投入したジョブ状態やリソース情報を確認する場合、*qstat* コマンドを使用します。

```
$ qstat [option] [JOBID[JOBID…]]
```

表 5-24 qstat コマンドオプション一覧

オプション名	説明
なし	実行待ち、実行中のジョブ情報を表示する。
-a	全ユーザのジョブ情報を表示する。 -v オプション、-t オプションおよび--system のみ併用可能とする。
-A	所属プロジェクト全員のジョブ情報を表示する。システム管理者の場合、全ユーザ情報を表示する。グループ管理者、システム管理者のみ使用可能とする。
--system	システム毎にジョブ情報を表示する。
-H	処理が終了したジョブ情報を表示する。過去 3 日に終了したジョブで直近最大 100 件を表示する。
--hday day	表示期間（日）を指定する。
--hnum num	最大表示件数を指定する。
-t	アレイジョブ情報(サブジョブ)を表示する。
-v	標準形式で出力されない、いくつかのジョブ情報を追加で表示する。 ・ジョブの経過時間制限値 ・ジョブの補足情報(実行開始時刻、スケジューリング待ち理由・ジョブ操作メッセージなど)
-f	-v オプションで出力される情報のほか、資源使用状況、資源制限値などの詳細情報を表示する。引数にジョブ ID の指定を必須とする。 ・チェーンジョブの依存情報 ・投入・開始・終了時刻、終了コード etc
--rsc	キューの情報を表示する。ユーザーが実行を許可されていない情報は表示しない。
-x	キューの制限値を表示する。
-b	キューのジョブ情報（ジョブ情報毎の統計）を表示する。
--limit	プロジェクトのジョブ投入数・実行制限値を表示する。
--rscuse	ノード専有キューと MIG 利用キューのノードおよび MIG インスタンス使用量を表示する。
-v	ルーティングキューについて、ルーティングキューに設定された各キュー情報を個別に表示する。
--nodeuse	ノード専有キューのノード使用量を表示する。
-v	ルーティングキューについて、ルーティングキューに設定された各キュー情報を個別に表示する。
--miguse	MIG 利用キューの MIG インスタンス使用量を表示する。
-v	ルーティングキューについて、ルーティングキューに設定された各キュー情報を個別に表示する。
--scal	システム停止、起動の予定時刻を表示する。
-l[l] | -w[w]	ジョブ名をロングフォーマット形式で表示する。

Copyright 2025 FUJITSU LIMITED

5.8.1. ジョブ状態表示

qstat コマンドを実行すると、現在実行中もしくは実行待ちのジョブ状態を表示します。

```
[username@miyabi-g1 work]$ qstat
Miyabi scheduled stop time: 2025/01/22(Web) 09:00:00 (Remain:   -days --:--:--)

JOB_ID          JOB_NAME     STATUS    PROJECT       QUEUE              START_DATE           ELAPSE
TOKEN            NODE   MIG
12345            job.sh         RUNNING  group1           debug-g      12/09  22:39:56      00:26:27
-               1     8
12346          job.sh       RUNNING group1        debug-g     (12/09 23:18:55)   --:--:--              -
1     8
12347          job.sh       RUNNING group1        debug-g     (12/09 23:47:08)   --:--:--              -
1     8
```

START_DATE に出力されるジョブの実行開始予測時刻は目安であるため、システムの混雑状況や課題の優先度により変動することがあります。

表 5-25 ジョブ情報の表示項目

項目	説明
JOB_ID	ジョブ ID
JOB_NAME	ジョブ名（10 文字以上は切り捨て） -a オプションを指定した場合、全ジョブを表示する。 -l オプションを指定した場合、ジョブ名をロングフォーマットで表示する。
STATUS	ジョブの状態(表 5-26 参照)
USER	ユーザー名(-A オプション指定時のみ)
PROJECT	プロジェクト名 -a オプションを指定した場合、自ジョブのみ表示する。
QUEUE	キュー名
START_DATE	ジョブの開始時刻（MM/DD HH:MM:SS）。 ジョブが実行前の場合は開始予測時刻（"（）”で表示）、実行中および実行後の場合は実際に実行を開始した時刻を表示する。
ELAPSE	ジョブの経過時間
TOKEN	使用トークン時間 ジョブ実行開始前の場合はトークン予測使用量(ノード数と経過時間制限を元に算出)
NODE	ジョブの要求ノード数
MIG	ジョブの要求 MIG インスタンス数

Copyright 2025 FUJITSU LIMITED

表 5-26 ジョブの状態一覧

状態	説明
QUEUED	ジョブ実行待ち
RUNNING	ジョブ実行中
BEGUN	アレイジョブに実行中のサブジョブが少なくとも 1 つが存在
EXITING	ジョブ終了中
FINISH	ジョブ終了
HELD	ジョブ保留

5.8.2. 詳細ジョブ情報の表示(-v オプション)

qstat コマンドの-v オプションを指定すると、詳細なジョブ情報を表示します。

```
[username@miyabi-g1 work]$ qstat -v
Miyabi scheduled stop time: 2025/01/22(Web) 09:00:00 (Remain:   -days --:--:--)

JOB_ID            JOB_NAME      STATUS    PROJECT        QUEUE                START_DATE             ELAPSE
ELAPSE_LIM        TOKEN             NODE   MIG COMMENT
12345            job.sh        RUNNING  group1          debug-g     12/09  23:11:24     00:03:58    48:00:00
-              1     8 -
12346            job.sh        RUNNING  group1          debug-g    (12/09  23:18:55)    --:--:--    48:00:00
-              1     8 -
12347            job.sh        RUNNING  group1          debug-g    (12/09  23:47:08)    --:--:--    48:00:00
-              1     8 -
```

表 5-27 詳細ジョブ情報の表示項目(追加項目のみ)

オプション名	説明
ELAPSE_LIM	ジョブの経過時間制限
COMMENT	通知メッセージ

Copyright 2025 FUJITSU LIMITED

5.8.3. 終了ジョブ情報の表示(-H オプション)

qstat コマンドの-H オプションを指定すると、実行が終了したジョブを最大 100 件表示します。デフォルトでは過去 3 日間で実行が終了したジョブが対象となりますが、--hday オプションにより最大 31 日間で終了したジョブが対象となります。

```
[username@miyabi-g1 work]$ qstat -H
Miyabi scheduled stop time: 2025/01/22(Web) 09:00:00 (Remain:   -days --:--:--)

JOB_ID          JOB_NAME     STATUS   PROJECT      QUEUE            START_DATE          ELAPSE
TOKEN             NODE   MIG
7597             run.sh        END       group1       debug-g      12/09  20:43:12      00:10:00
-               2    -
7598             run.sh        END       group1       debug-g      12/09  20:47:37      00:10:01
-               2    -
7599             run.sh        END       group1       debug-g      12/09  20:48:44      00:10:02
-               2    -
```

5.8.4. キューの表示(--rsc オプション)

qstat コマンドの--rsc オプションを指定すると、利用可能なキューを表示します。

```
[username@miyabi-g1 work]$ qstat --rsc

SYSTEM: Miyabi-G
QUEUE                        STATUS                    NODE
debug-g                     [ENABLE, START]            48
short-g                     [ENABLE, START]            24
coupler-g                   [ENABLE, START]          1024
tutorial-g                  [ENABLE, STOP]              0
lecture0-g                  [ENABLE, STOP]              0
lecture1-g                  [ENABLE, STOP]              0
lecture2-g                  [ENABLE, STOP]              0
(省略)

SYSTEM: Miyabi-C
QUEUE                        STATUS                    NODE
debug-c                     [ENABLE, START]            16
short-c                     [ENABLE, START]            6
coupler-c                   [ENABLE, START]          168
tutorial-c                  [ENABLE, STOP]              0
lecture0-c                  [ENABLE, STOP]              0
lecture1-c                  [ENABLE, STOP]              0
lecture2-c                  [ENABLE, STOP]              0
lecture3-c                  [ENABLE, STOP]              0
lecture4-c                  [ENABLE, STOP]              0
(省略)
```

Copyright 2025 FUJITSU LIMITED

表 5-28 キューの表示項目

項目	説明
QUEUE	キュー名
STATUS	キューの状態 ENABLE: ジョブの新規投入可能 DISABLE: ジョブの新規投入不可 START: 待ちジョブを新規実行可能 STOP: 待ちジョブの新規実行抑止
NODE	キューのノード数
MIG	キューの MIG インスタンス数

qstat コマンド--rsc オプションに加え-b オプションを指定すると、全ジョブのキュー毎のジョブ数を状態別に表示します。

```
[username@miyabi-g1 work]$ qstat --rsc -b

SYSTEM: Miyabi-G
QUEUE                STATUS          TOTAL RUNNING QUEUED  HELD OTHER   NODE
debug-g              [ENABLE, START]     0       0      0     0     0      48
short-g              [ENABLE, START]     0       0      0     0     0      24
coupler-g            [ENABLE, START]     0       0      0     0     0    1024
tutorial-g           [ENABLE, STOP]      0       0      0     0     0       0
lecture0-g           [ENABLE, STOP]      0       0      0     0     0       0
lecture1-g           [ENABLE, STOP]      0       0      0     0     0       0
lecture2-g           [ENABLE, STOP]      0       0      0     0     0       0
(省略)

SYSTEM: Miyabi-C
QUEUE                STATUS          TOTAL RUNNING QUEUED  HELD OTHER   NODE
debug-c              [ENABLE, START]     0       0      0     0     0      16
short-c              [ENABLE, START]     0       0      0     0     0       6
coupler-c            [ENABLE, START]     0       0      0     0     0     168
tutorial-c           [ENABLE, STOP]      0       0      0     0     0       0
lecture0-c           [ENABLE, STOP]      0       0      0     0     0       0
(省略)
```

表 5-29 キュー毎の状態別ジョブ数の表示項目

項目	説明
QUEUE	キュー名
STATUS	キューの状態
TOTAL	キューに投入されたジョブの総数
RUNNING	実行中のジョブ数
QUEUED	ジョブ実行待ち状態のジョブ数
HOLD	保留状態のジョブ数
OTHER	ジョブ状態が RUNNING,QUEUED,HOLD のジョブ数
NODE	キューのノード数
MIG	キューの MIG インスタンス数

Copyright 2025 FUJITSU LIMITED

qstat コマンドの--rsc オプションに加え-x オプションを指定すると、各キューのノード数制限や経過時間制限を表示します。

```
[username@miyabi-g1 work]$ qstat --rsc -x

SYSTEM: Miyabi-G
QUEUE                STATUS          MIN_NODE MAX_NODE MAX_ELAPSE REMAIN_ELAPSE     MEM
PROJECT
debug-g              [ENABLE, START]      0        0    00:30:00     1193:49:22       0 group1
short-g              [ENABLE, START]      0        0    08:00:00     1193:49:22       0 group1
coupler-g            [ENABLE, START]      0        0    02:00:00     1193:49:22       0 group1
tutorial-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
lecture0-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
lecture1-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
lecture2-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
lecture3-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
lecture4-g           [ENABLE, STOP]       0        0    00:15:00     1193:49:22       0 group1
(省略)
```

表 5-30 キューの表示項目

項目	説明
QUEUE	キュー名
STATUS	キューの状態
MIN_NODE	ジョブの最小割り当てノード数
MAX_NODE	ジョブの最大割り当てノード数
AVAIL_MIG	ジョブの割り当て MIG インスタンス数(Miyabi-G のみ)
MAX_ELAPSE	ジョブの最大経過時間制限
REMAIN_ELAPSE	定期保守までの残り時間
MEM(GB)	ジョブの最大メモリ制限(GB)
PROJECT	プロジェクト名

Copyright 2025 FUJITSU LIMITED

5.8.5. ノード利用状況の表示(--nodeuse オプション)

qstat コマンドの--nodeuse オプションを指定すると、ノードの利用状況を表示します。割り当てノードが共通のキューは同一行に表示されます。

```
[username@miyabi-g1 work]$ qstat --nodeuse

SYSTEM: Miyabi-G
QUEUE                                                    Ratio Used/Total(Node)
debug-g                    --------------------          0%          0/   48
short-g                    --------------------         0%          0/   24
coupler-g                  --------------------         0%          0/1024
tutorial-g                 --------------------         0%          0/    0
lecture0-g                 --------------------         0%          0/    0
```

表 5-31 ノード利用状況の表示項目

項目	説明
QUEUE	キュー名
Ratio	ノード使用率
Used	使用している計算ノード数 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた計算ノード総数

Copyright 2025 FUJITSU LIMITED

5.8.6. MIG インスタンス利用状況の表示(--miguse オプション)

qstat コマンドの--miguse オプションを指定すると、MIG 利用キューの利用状況を表示します。割り当てノードが共通のキューは同一行に表示されます。

```
[username@miyabi-g1 work]$ qstat --miguse

SYSTEM: Miyabi-G
QUEUE                                                Ratio Used/Total(MIG)
debug-mig                    --------------------     0%        0/   96
short-mig                    --------------------     0%        0/    0
regular-mig                  --------------------     0%        0/    0
tutorial-mig                 --------------------     0%        0/    0
lecture0-mig                 --------------------     0%        0/    0
lecture1-mig                 --------------------     0%        0/    0
lecture2-mig                 --------------------     0%        0/    0
lecture3-mig                 --------------------     0%        0/    0
lecture4-mig                 --------------------     0%        0/    0
(省略)
```

表 5-32 MIG 利用状況の表示項目

項目	説明
QUEUE	キュー名
Ratio	ノード使用率
Used	使用している MIG インスタンス数 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた MIG インスタンス総数

Copyright 2025 FUJITSU LIMITED

5.8.7. ノードおよび MIG 利用状況の表示(--rscuse オプション)

qstat コマンドの--rscuse オプションを指定すると、ノードおよび MIG インスタンスの利用状況を表示します。割り当てノードが共通のキューは同一行に表示されます。

```
[username@miyabi-g1 work]$ qstat --rscuse

SYSTEM: Miyabi-G
QUEUE                                                Ratio Used/Total(Node)
debug-g                    --------------------      0%        0/   48
short-g                    --------------------      0%        0/   24
coupler-g                  --------------------      0%        0/1024
tutorial-g                 --------------------      0%        0/    0
(省略)

QUEUE                                                Ratio Used/Total(MIG)
debug-mig                  --------------------      0%        0/   96
short-mig                  --------------------      0%        0/    0
regular-mig                --------------------      0%        0/    0
tutorial-mig               --------------------      0%        0/    0
(省略)
```

表 5-33 ノードおよび MIG 利用状況の表示項目

項目	説明
QUEUE	キュー名
Ratio	ノード使用率
Used	使用している計算ノード数、または MIG インスタンス数 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた計算ノード総数、または MIG インスタンス総数

Copyright 2025 FUJITSU LIMITED

5.8.8. ジョブ本数制限、ノード制限の表示(`--limit` オプション)

qstat コマンドの--limit オプションを指定すると、各種制限値と現在の状況を表示します。

```
[username@miyabi-g1 work]$ qstat --limit

SYSTEM: Miyabi-G
PROJECT                          ACCEPT        RUN        NODE       MIG
group1                           5/2240      5/1120     19/1120      1/24

SYSTEM: Miyabi-C
PROJECT                          ACCEPT        RUN        NODE
group1                           17/384       1/192      32/192
```

表 5-34 各種制限値の表示項目

項目	説明
PROJECT	プロジェクト名
ACCEPT	投入ジョブ本数/バッチジョブ同時受付制限
RUN	実行中のジョブ本数/バッチジョブ同時実行制限
NODE	実行中のジョブの使用ノード数/プロジェクト単位の同時ノード使用制限 同時使用ノード数制限がないプロジェクトは、”-“を表示する。
MIG	実行中のジョブの使用 MIG インスタンス数/プロジェクト単位の同時 MIG インスタンス使用制限

5.9. ジョブキャンセル

ジョブをキャンセルする場合、*qdel* コマンドを実行します。

$ qdel JOBID [JOBID…]

例) ジョブ ID12345 をキャンセル

```
[username@miyabi-g11 work]$ qdel 12345
12345 canceled.
```

Copyright 2025 FUJITSU LIMITED

5.10. ジョブ保留

ジョブの実行を保留する場合、*qhold* コマンドを実行します。

$ qhold JOBID [JOBID…]

例) ジョブ ID12345 を保留

```
[username@miyabi-g1 work]$ qhold 12345
12345 held.
```

5.11. ジョブ保留の解除

保留されたジョブを解除する場合、*qrls* コマンドを実行します。

$ qrls JOBID [JOBID…]

例) ジョブ ID12345 の保留を解除

```
[username@miyabi-g1 work]$ qrls 12345
12345 released.
```

Copyright 2025 FUJITSU LIMITED

5.12. ジョブ履歴の参照

ジョブの状態遷移を表示する場合、tracejob コマンドを使用します。

```
$ tracejob [option] JOBID [JOBID…]
```

表 5-35 ジョブの状態遷移表示コマンドオプション一覧

オプション	説明
-a	アカウント情報を報告しない
-s	サーバの情報を報告しない
-l	スケジューラの情報を報告しない
-w W	ジョブ情報を表示する幅の文字数を指定する(デフォルト: 80)
-n N	表示期間を指定 (デフォルト: 当日)
-z	重複するメッセージを表示しない
-c C	過剰なメッセージの表示をカウント制限する（デフォルト: 15）

例) ジョブ ID12345 の状態遷移を表示

```
[username@miyabi-g1 work]$ tracejob 12345

Job: 12345.opbs

12/09/2024 14:49:20 S Job Queued at request of z30216@miyabi-g1, owner = z30216@miyabi-g1, job name = run.sh, queue = debug-g
12/17/2024 14:49:21 S 'RunjobCheck' hook set job's Variable_List = PBS_O_HOME=XXXXXX…(省略)
12/09/2024 14:49:21 S Job Run at request of Scheduler@opbs1 on exec_vnode (mg0389:mem=104857600kb:ncpus=72:ngpus=1:nvnodes-g=1)+(mg0390:mem=104857600kb:ncpus=72:ngpus=1:nvnodes-g=1)+(mg0391:mem=104857600kb:ncpus=72:ngpus=1:nvnodes-g=1)+(mg0392:mem=104857600kb:ncpus=72:ngpus=1:nvnodes-g=1)
12/09/2024 14:49:27 S Exit_status=0 resources_used.cpupercent=351 resources_used.cput=00:00:05 resources_used.mem=11075136kb resources_used.ncpus=288 resources_used.vmem=11075136kb resources_used.walltime=00:00:02
```

Copyright 2025 FUJITSU LIMITED

5.13. 計算ノードから外部リソースへのアクセス

Miyabi-G の計算ノードからバッチジョブ・インタラクティブジョブを使用して外部リソース(サーバ、ストレージ、センサーなど)に直接アクセスすることが可能です。ただし、外部リソースへのアクセスは以下のアクセス方法のみとなります。なお、外部リソースから各計算ノードへのアクセスはできません。

表 5-36 各計算ノードからの外部リソースへのアクセス方法一覧

システム	アクセス方法	アクセス設定
Miyabi-G	・HTTP/HTTPS ・SSH	NAT を使用し外部へアクセス

Copyright 2025 FUJITSU LIMITED

6. マニュアル参照

コンパイラやデバッガ、各種ソフトウェアのマニュアルは、利用支援ポータルで公開しています。

(1) ブラウザで下記 URL にアクセスします。

https://miyabi-www.jcahpc.jp/

(2) 利用支援ポータルの画面を開き、ポータルにログインします。

図 6-1 利用支援ポータルログイン画面

(3) [ドキュメント閲覧]メニューへアクセスします。

図 6-2 利用規約の確認

(4) 必要なマニュアルを表示します。

Copyright 2025 FUJITSU LIMITED

7. コンテナ環境

コンテナでの実行環境として、singularity を利用することができます。

7.1. Singularity の利用方法

例) Singularity を利用する場合のジョブスクリプト

```
[username@miyabi-g1 work]$ vi run.sh
#!/bin/sh
#------ qsub option --------#
#PBS -q debug-g
#PBS -l select=1
#PBS -l walltime=30:00
#PBS -W group_list=group1
#PBS -j oe
#------- Program execution -------#

module load singularity/4.2.1
singulariy <command>
```

Copyright 2025 FUJITSU LIMITED

8. Open OnDemand

Open OnDemand を利用して以下の機能を Web ブラウザから利用可能です。デスクトップ環境、および、JupyterLab 環境はバッチジョブシステムと連携し、バッチジョブにて利用環境を提供します。

- ファイル操作
- デスクトップ環境
- JupyterLab 環境
- Shell 機能

8.1. 認証方式

Open OnDemand のログインには、パスワード認証とワンタイムパスコード認証(OTP 認証)を併用する 2 要素認証を採用しています。ここで使用するパスワードは利用支援ポータルにログインする際に使用するパスワードと 2.3.2.2 利用ポータルの初回ログイン時に設定した、ワンタイムパスワードを使用します。

Copyright 2025 FUJITSU LIMITED

8.2. Open OnDemand への接続

(1) ブラウザで下記 URL にアクセスします。

https://miyabi-c7.jcahpc.jp/

(2) ログイン画面が表示されますので、Miyabi システムのユーザーID でログインし、2.3.2.2 利用ポータルの初回ログイン時に設定した、ワンタイムパスワードを入力します。

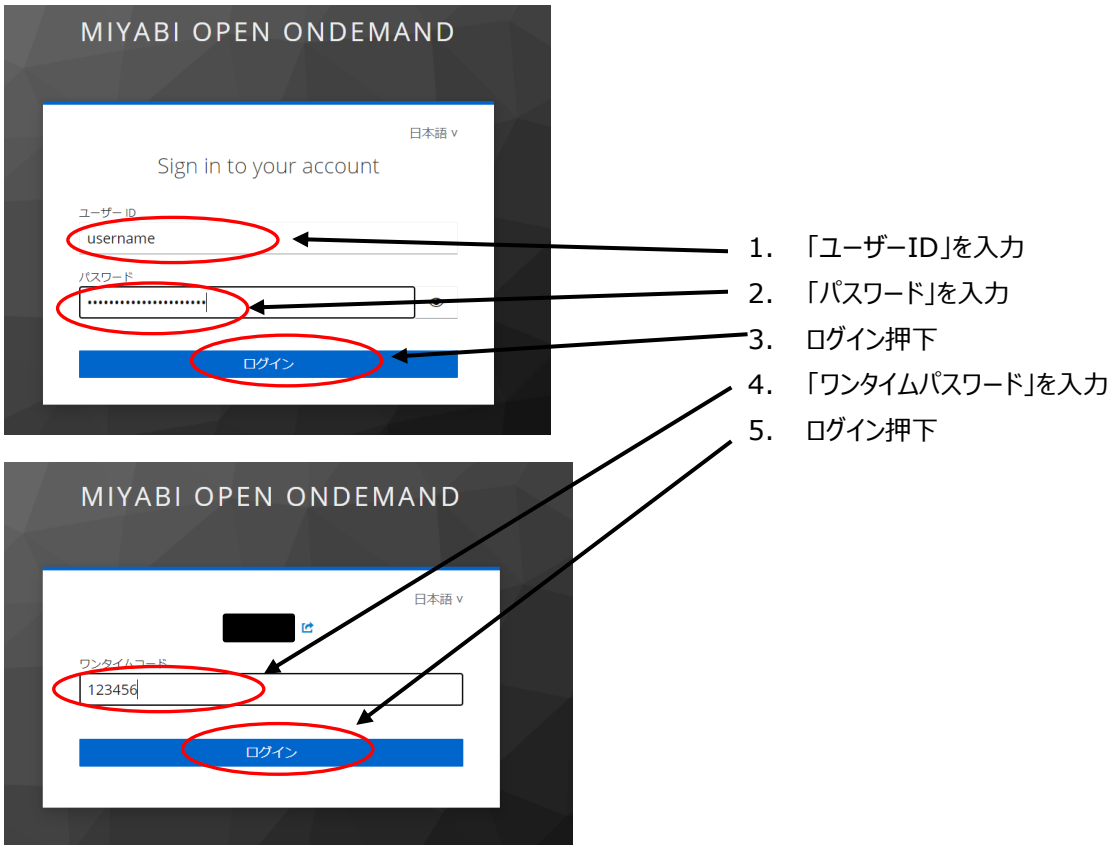


図 8-1 ログイン画面

ワンタイムパスワードを設定していない場合は、ワンタイムパスワード設定画面が表示されます。
ワンタイムパスワードの設定方法については、2.3.2.2 を参照ください。

Copyright 2025 FUJITSU LIMITED

(3) ログインに成功すると HOME 画面が表示されます。

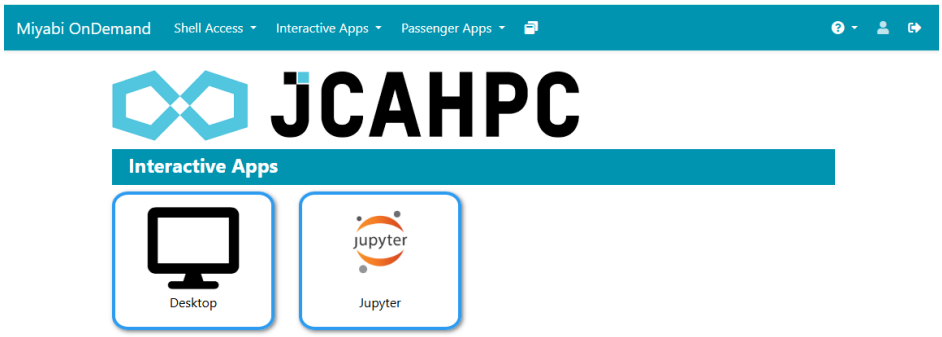


図 8-2 ログイン後の HOME 画面

Open OnDemand にログインすると、work ディレクトリ(/work/<プロジェクト名>/<ユーザ名>/)に「ondemand」とういディレクトリが作成されます。
ジョブ投入などで使用するディレクトリであるため、削除しないでください。

8.3. ログアウト

ログアウトする場合は、画面右上の「Log Out」ボタンをクリックします。

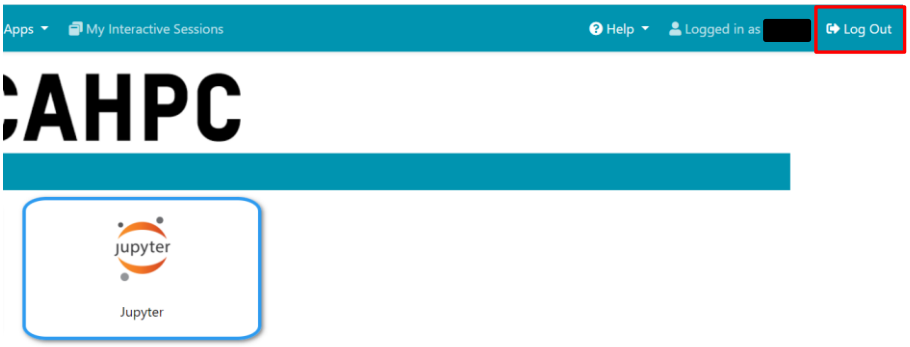


図 8-3 ログアウト

Copyright 2025 FUJITSU LIMITED

8.4. ファイル操作機能の利用

Open Ondemand のファイル操作機能では、ファイル/ディレクトリの作成・削除・参照・移動やファイルのアップロードとダウンロードを Web ブラウザ上で実施できます。

(1) ログイン後の画面上部のタブの Passenger Apps から、「Home Directory」を選択します。

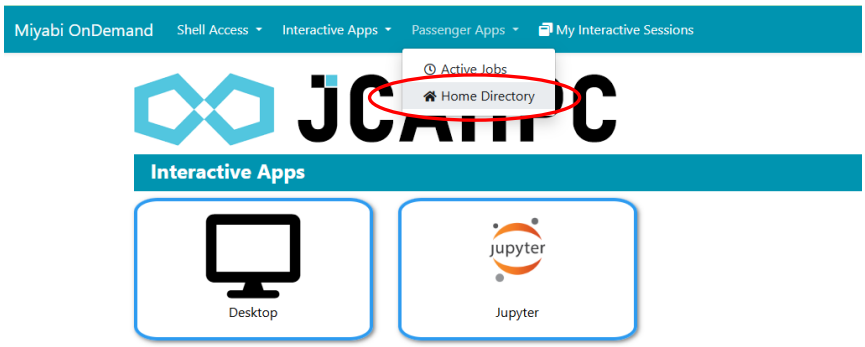


図 8-4 ファイル操作の選択

(2) ファイル操作画面が開きますので、各種操作を行うことができます。

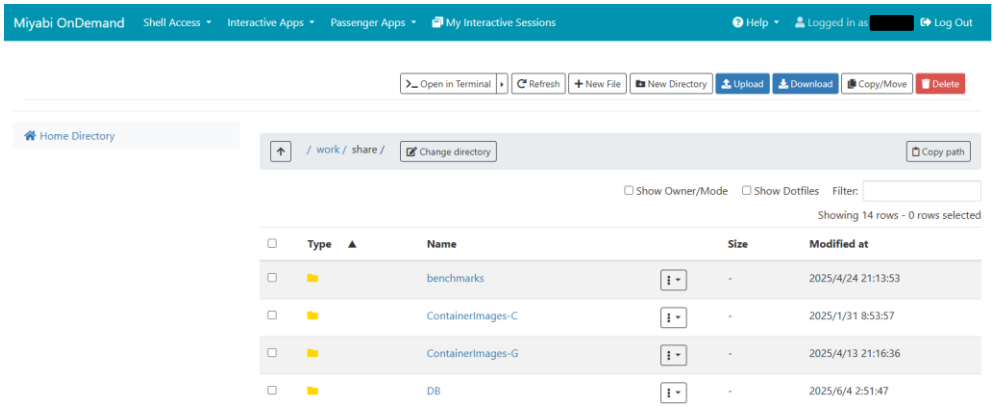


図 8-5 ファイル操作画面

Copyright 2025 FUJITSU LIMITED

8.5. Desktop 環境の利用

(1) ログイン後の HOME 画面の Interactive Apps から「Desktop」をクリックします。

図 8-6 Desktop 利用の選択

(2) 使用する Desktop 環境をプルダウンで選択して「Launch」ボタンで Desktop 環境を利用するためのジョブが投入されます。

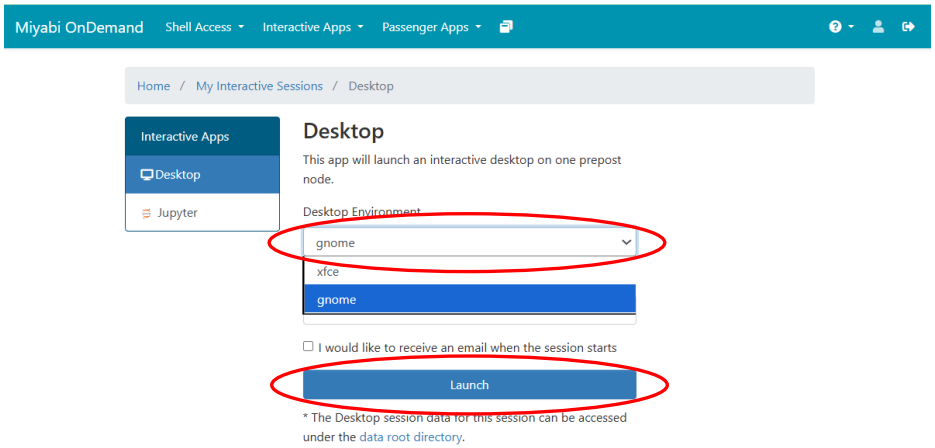


図 8-7 Desktop 利用の選択

(3) Desktop 環境を使用するためのインタラクティブジョブが投入されます。ジョブのステータスが Runing になったら「Launch Desktop」ボタンをクリックしデスクトップ環境にアクセスします。

Copyright 2025 FUJITSU LIMITED

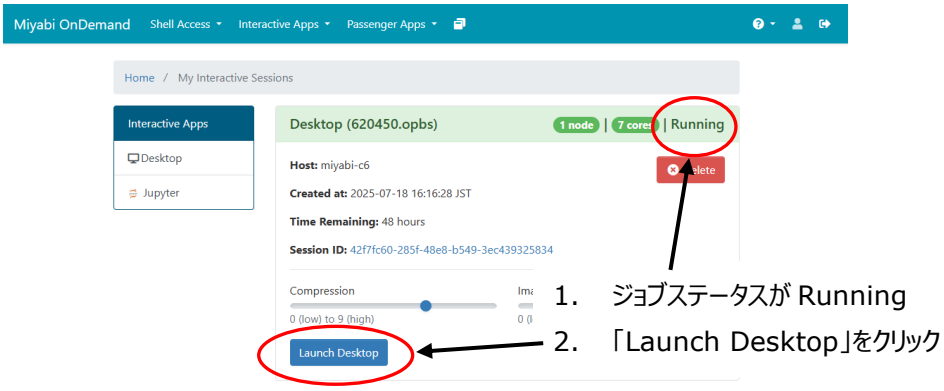


図 8-8 Desktop 環境へのアクセス

(4) デスクトップ環境が表示され、各種操作を行うことができます。

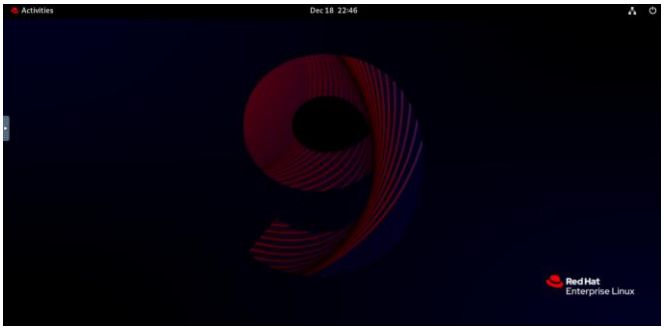


図 8-9 Desktop 環境（gnome）へのアクセス

(5) デスクトップ環境終了時は画面右上の電源ボタンをクリックし、「Log Out」を選択します。

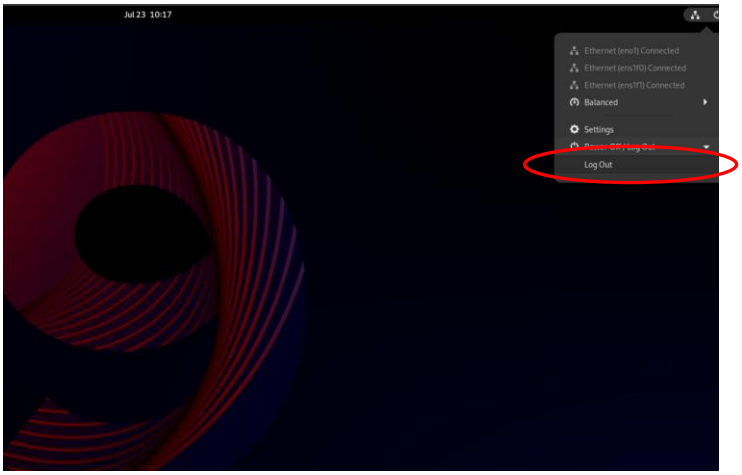


図 8-10 Desktop 環境からのログアウト

Copyright 2025 FUJITSU LIMITED

8.6. Jupyter 環境の利用

(1) ログイン後の HOME 画面の Interactive Apps から Jupyter を選択します。

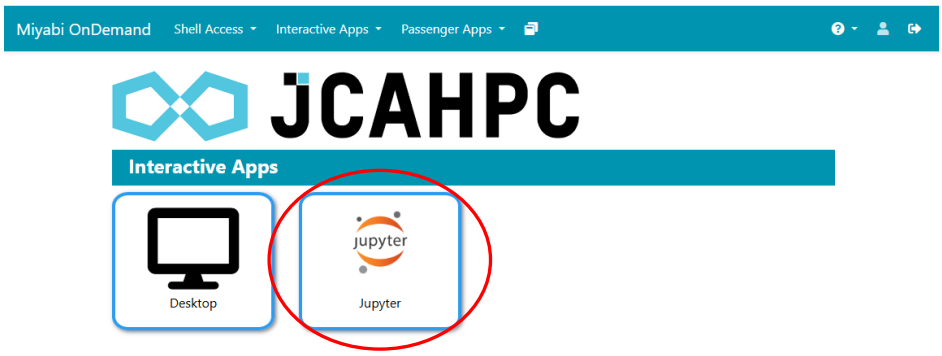


図 8-11 Jupyter 利用の選択

(2) Jupyter サーバをジョブで起動するためのリソースを選択し、Launch ボタンをクリックすることでジョブが投入されます。リソースの選択で設定する項目については、表 8-1 に示します。

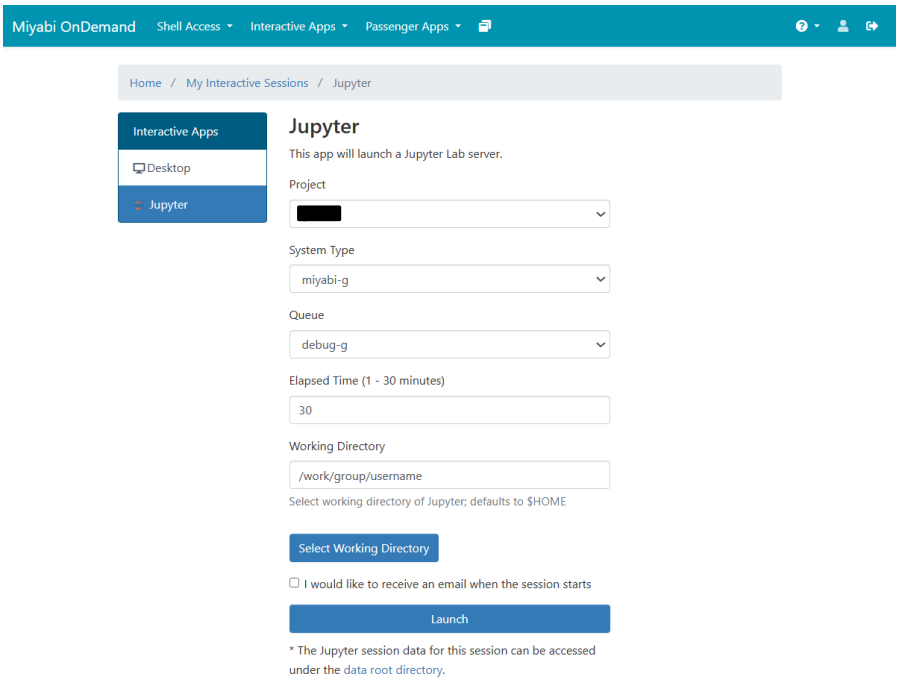


図 8-12 リソースの選択

Copyright 2025 FUJITSU LIMITED

表 8-1 リソースの選択項目一覧

項目	説明
Project	ジョブ実行時にトークンを消費するプロジェクトを指定
System Type	ジョブ投入するシステムを選択(Miyabi-G/Miyabi-C)
Queue	キュー名/ルーティングキュー名を指定
Elapsed time	ジョブの実行時間を分(minutes)または時間(hours)で選択
Working Directory	Jupyter サーバが起動する作業ディレクトリを指定 (デフォルトは/work/<グループ名>/<ユーザ名>)

(3) ジョブのステータスが Running になったら、「Connect to Jupyter」をクリックし、JupyterLab にアクセスします。

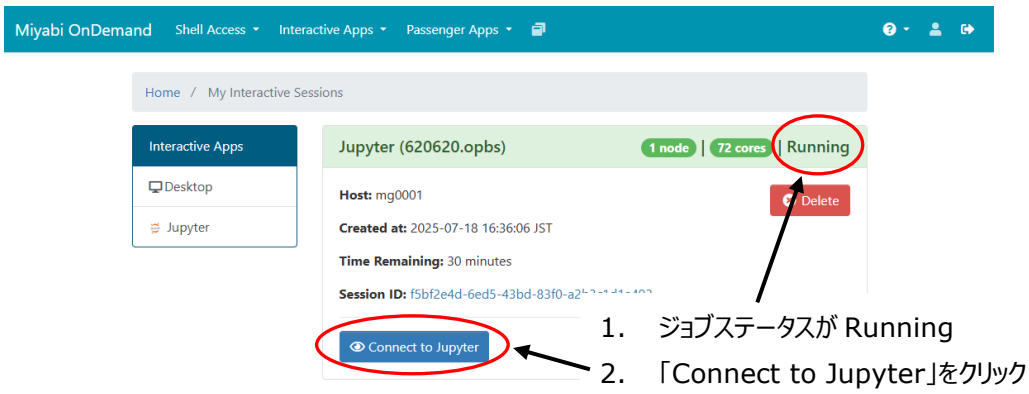


図 8-13 JupyterLab へのアクセス

(4) JupyterLab サーバにアクセスし、各種操作を行うことができます。

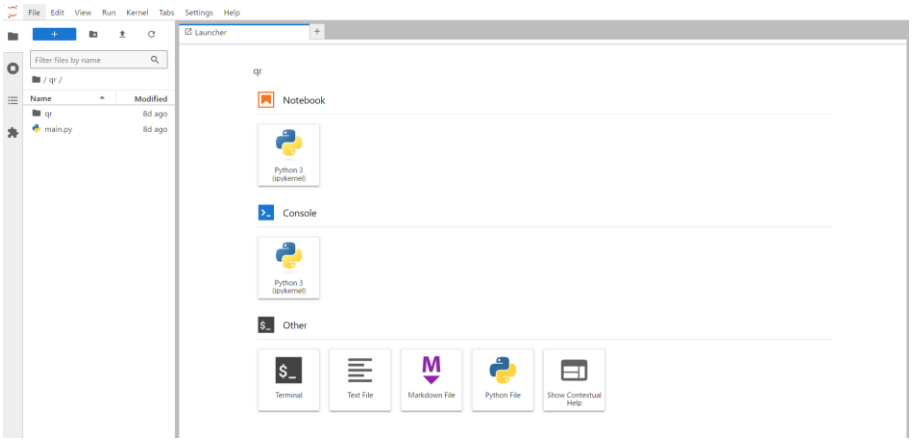


図 8-14 JupyterLab 画面

Copyright 2025 FUJITSU LIMITED

(5) JupyterLab 終了時は「File」タブから「Shut Down」を選択することで、JupyterLab が終了します。

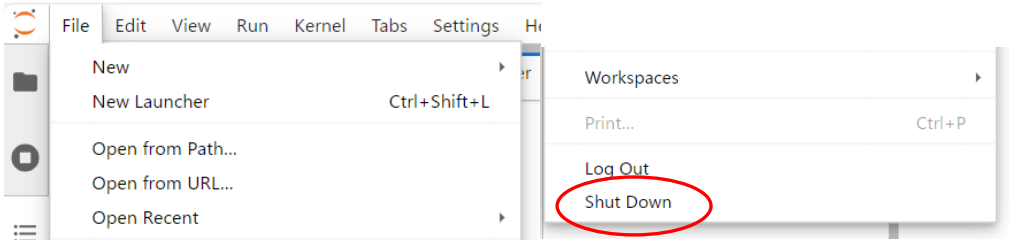


図 8-15 ログアウト

JupterLab サーバが起動中(ジョブ実行中)は、トークンが消費されます。
利用終了後は Shut Down で終了させるか、8.7 のアクティブなジョブ確認機能より該当ジョブを削除してください。

Copyright 2025 FUJITSU LIMITED

8.7. Shell 機能

Open OnDemand の Shell 機能では、ログインノードのログインシェルにアクセスし、Web ブラウザ上からコマンドラインインタフェースによる操作が可能です。

(1) ログイン後の画面上部のタブの Shell Access から、「Miyabi-C」(または「Miyabi-G」)を選択します。

図 8-16 Shell 機能の選択

(2) Miyabi-C のログインノードのシェルにアクセスし、各種操作を行うことが可能です((1)で「Miyabi-G」を選択した場合は Miyabi-G のログインノードにアクセスします)。

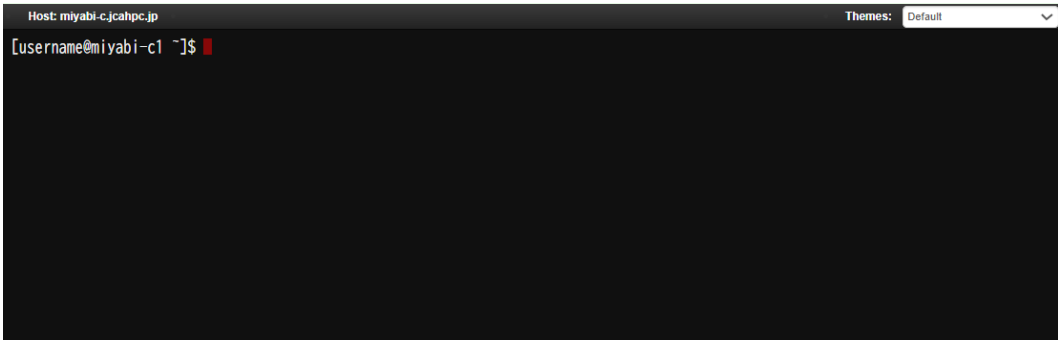


図 8-17 ログインシェルへのアクセス

Copyright 2025 FUJITSU LIMITED

8.8. アクティブなジョブの確認機能の利用

アクティブなジョブの確認機能では、Open OnDemad から投入したバッチジョブ(デスクトップ環境、および、JupyterLab 環境)の確認および、ジョブの削除を行うことができます。

(1) ログイン後の画面上部のタブの Passenger Apps から、「Active Jobs」を選択します。

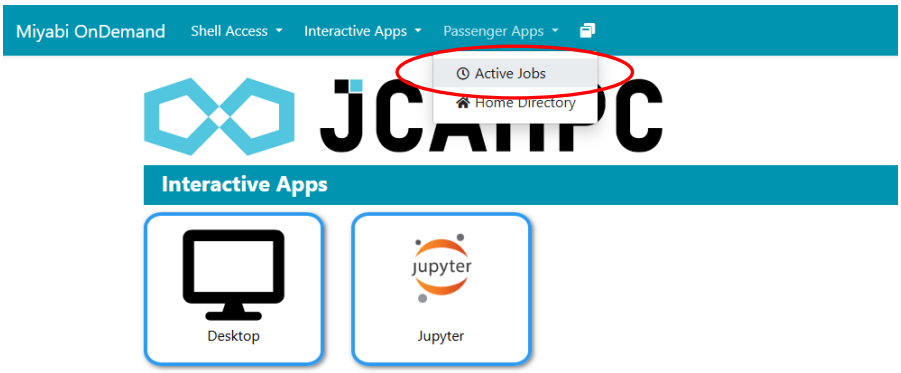


図 8-18 アクティブなジョブの選択

(2) アクティブなジョブの一覧が表示され、各種操作を行うことができます。

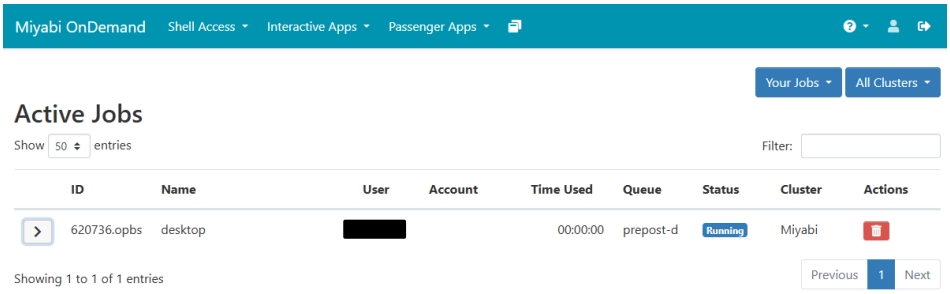


図 8-19 アクティブなジョブ一覧

(3) ジョブ ID 左側のボタンをクリックすることで、ジョブ詳細情報を表示できます。

Copyright 2025 FUJITSU LIMITED

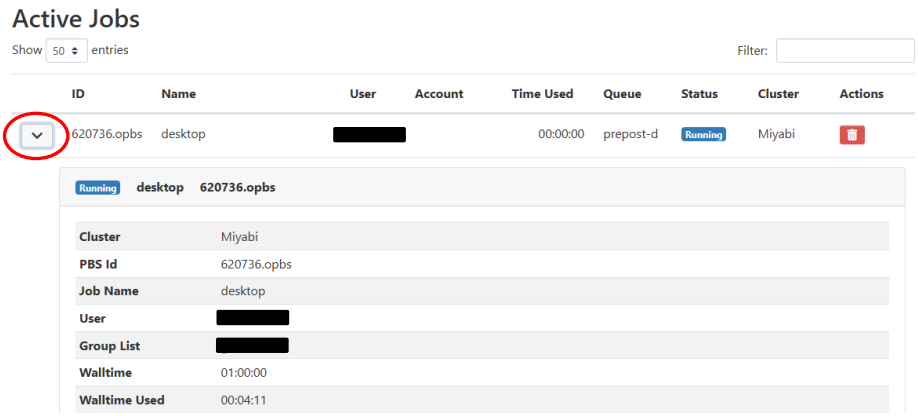


図 8-20 ジョブ詳細情報の表示

(4) 「ゴミ箱」ボタンをクリックすることでジョブを削除することができます。

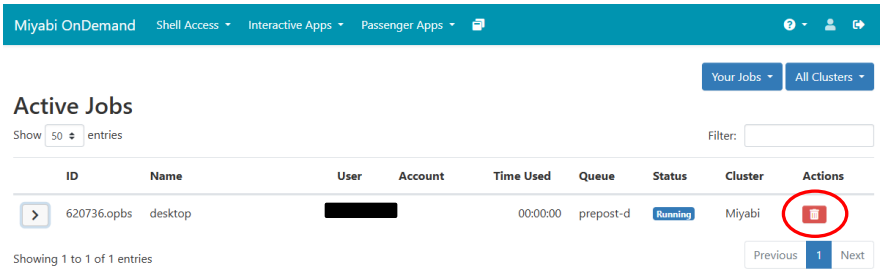


図 8-21 ジョブの削除

Copyright 2025 FUJITSU LIMITED